# 第7章: 書籍管理アプリ — 一覧表示と登録機能の実装

> いよいよアプリの本体を作ります！この章では、**CRUD**（クラッド／create・read・update・delete の頭文字を取った言葉。データに対する基本4操作のこと）のうち、**R（Read：一覧表示）** と **C（Create：新規登録）** を実装します。

## CRUD の4つの操作とは

| 文字 | 英語 | 日本語 | 例 |
|---|---|---|---|
| C | Create | 作成 | 新しい書籍を登録する |
| R | Read | 読み取り | 書籍の一覧を表示する／1冊の詳細を見る |
| U | Update | 更新 | 書籍の評価やステータスを変更する |
| D | Delete | 削除 | 不要になった書籍を消す |

ほとんどのアプリ（SNS・ECサイト・メモアプリなど）は、結局この4操作の組み合わせでできています。この章ではまず **C** と **R** を作ります。

### ▼ CRUD は「画面」「コード」「データベース」で呼び名が変わるだけ

同じ「本を登録する」という操作でも、どこから見るかによって呼び名が変わります。この対応表を最初に頭に入れておくと、この先のコードがぐっと読みやすくなります。

| CRUD | 画面でやること（この本のアプリ） | supabase-js で書くこと | SQL（データベースへの命令） | HTTP メソッド（通信の種類） | 扱う章 |
|---|---|---|---|---|---|
| C | 登録フォームから本を追加する | `.insert({...})` | `INSERT` | `POST` | **この章（4節）** |
| R | 一覧ページで本を眺める | `.select("*")` | `SELECT` | `GET` | **この章（1節）** |
| U | 評価やメモを書き直す | `.update({...}).eq("id", id)` | `UPDATE` | `PATCH` | 第8章 |
| D | 不要になった本を消す | `.delete().eq("id", id)` | `DELETE` | `DELETE` | 第8章 |

> **用語:** HTTP メソッド（エイチティーティーピー・メソッド）とは、ブラウザやプログラムがサーバーに送る「お願いの種類」のこと。`GET` ＝ください、`POST` ＝新しく受け取って、`PATCH` ＝一部を書き換えて、`DELETE` ＝消して、という意味です。supabase-js は、私たちが書いた `.insert()` などを裏側でこの HTTP のお願いに変換して Supabase に送っています。

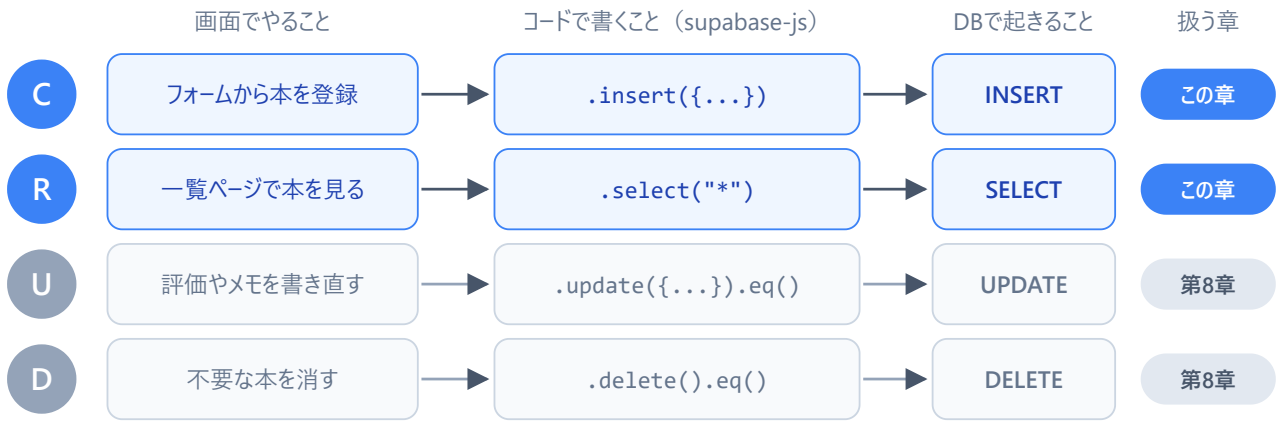



図7-1: CRUD の4操作と、画面・コード・データベースでの呼び名の対応（青がこの章で作る部分）

## この章で作るもの

この章を終えると、以下の2つの画面が動くようになります。

| 画面 | できること | CRUDのどれ？ |
|---|---|---|
| **書籍一覧ページ**（トップページ） | Supabaseに保存された書籍をカード形式で表示する | **R**（Read：読み取り） |
| **書籍登録ページ** | フォームに入力して新しい書籍を登録する | **C**（Create：作成） |

## ユーザー体験の流れ

ユーザーの視点で、完成後の操作の流れを確認しましょう。

1. アプリを開く → **書籍一覧ページ**が表示される（登録済みの書籍がカード形式で並ぶ）
2. 「＋ 新規登録」ボタンを押す → **登録フォーム**に移動する
3. タイトル・著者・評価などを入力して「登録する」を押す → データがSupabaseに保存される
4. 自動的に一覧ページに戻る → 今登録した書籍が一覧に追加されている

> **用語メモ:** - **Server Component**（サーバーコンポーネント／server component）: サーバー側でHTMLを組み立ててからブラウザに送る部品。データベース接続などに向いている。 - **Client Component**（クライアントコンポーネント／client component）: ブラウザの中で動く部品。クリックや入力などインタラクティブな処理に向いている。 - **revalidation**（リバリデーション／再検証）:「キャッシュ済みのデータをもう一度サーバーから取り直す」こと。データ更新後に画面を最新化するため使う。 - **楽観的UI**（らっかんてきユーアイ／optimistic UI）: サーバーの返事を待たずに「成功したつもり」で画面を先に更新するUI手法。体感速度を上げる代わりに失敗時のロールバックが必要になる。この章では使わないが、第8章以降で登場する。

### 作成するコンポーネント（部品）一覧

| コンポーネント名 | 役割 | なぜ分けるのか |
|---|---|---|
| StatusBadge | 「読書中」「読了」などのステータスを色付きバッジで表示 | 複数の画面で使い回すため |
| RatingStars | 1〜5の評価を星マーク（★）で表示 | 一覧・詳細の両方で使うため |
| BookCard | 1冊分の書籍情報をカード型で表示 | 一覧画面でN冊分繰り返し表示するため |
| BookList | BookCardを並べて表示するコンテナ（入れ物） | レイアウトの責任を分離するため |
| BookForm | 書籍の登録/編集に使うフォーム | 登録と編集で同じフォームを使い回すため |
| LoadingSpinner | データ読み込み中の表示 | ユーザーに「読み込み中です」と伝えるため |

> **なぜコンポーネントを分けるの？** 1つの巨大なファイルに全てを書くこともできますが、部品に分けることで「見通しが良くなる」「バグを見つけやすくなる」「同じ部品を使い回せる」というメリットがあります。これは**コンポーネント設計**と呼ばれ、React開発の重要な考え方です。

## この章のゴールと地図

**たとえ話:** この章でやることは「本棚づくり」に似ています。まず本棚（一覧ページ）を組み立て、そこに並ぶ本の背表紙（カード）を整えます。次に「棚を準備中です」「棚が壊れています」という張り紙（読み込み中・エラー画面）を用意し、最後に新しい本を受け付ける窓口（登録フォーム）を作ります。窓口で受け付けた本が、ちゃんと本棚に並べば完成です。

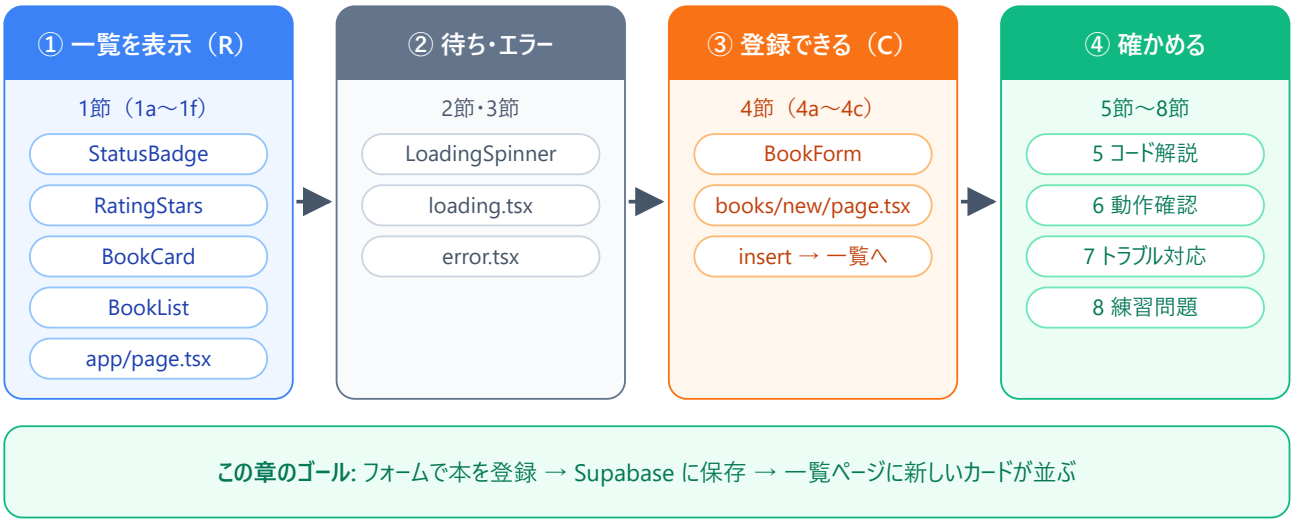



図7-2: この章の地図（①→④の順に進みます）

**この章を終えると、次のことができるようになります。**

- Server Component の中で `await` を使って Supabase からデータを取得し、一覧として表示できる
- 小さな部品（StatusBadge・RatingStars・BookCard・BookList）を組み合わせて1つの画面を作れる
- 0冊のとき（空状態）・読み込み中・エラーのとき、それぞれの画面を用意できる
- フォームの入力を `useState` で管理し、送信前にチェック（バリデーション）できる
- Supabase に新しい行を INSERT し、一覧に戻って最新の状態を表示できる

**前提知識:** 第3章（React の props と useState）、第4章（App Router と Server / Client Component）、第5章（books テーブルと RLS）、第6章（プロジェクトの準備）を終えていること。

**進め方のコツ:** 各節は「完成コード → 画面の説明 → 分解解説 → 動かして確かめる（Node.js だけで実行できる小さな例）」の順に並んでいます。分からない行があっても、まずはコードを写して画面を動かし、あとから分解解説を読む、という順番でも大丈夫です。

---

## 目次

0. 前提知識: 非同期処理（async / await）の超基礎
1. 一覧表示機能の実装
2. ローディング状態
3. エラーハンドリング
4. 登録機能の実装
5. コード解説
6. 動作確認手順
7. トラブルシューティング

8. 練習問題（解答・解説つき）

---

## 0. 前提知識: 非同期処理（async / await）の超基礎

この章のコードでは `async`（エイシンク）と `await`（アウェイト）が頻出します。これは「**時間がかかる処理を待つ**」ための仕組みです。Supabaseに「データを取ってきて」とお願いする処理は、結果が返ってくるまで数十〜数百ミリ秒かかります（インターネット越しに別のサーバーへ問い合わせるので、CPU内部で完結する計算より格段に遅い）。その「待ち時間」を扱うのが async/await です。

> **読み方と意味:** - **synchronous**（シンクロナス／同期）: 「時間軸が揃っている」という意味。前の処理が終わってから次の処理に進む。 - **asynchronous**（エイシンクロナス／非同期）: 「時間軸が揃っていない」。前の処理の終了を待たずに次の処理が始まることがある。

### 0.1 同期と非同期の違い

```
// 同期処理（synchronous）: 上から下へ順番に実行され、即座に結果が返る
// この計算はCPU内部だけで完結するのでマイクロ秒で終わる
const sum = 1 + 2;
// 3 と表示される。1 + 2 の結果がすでに確定しているため。
console.log(sum);

// 非同期処理（asynchronous）: 結果が返るのに時間がかかる
// ネット越しのリクエスト。返事が来るまで時間がかかる。
const data = supabase.from("books").select("*");
//    ❌ await を付けないので、data には「答え」ではなく「答えが入る約束（Promise）」が代入される。
```

### 0.2 Promise って何？

非同期処理の「結果が返ってくる予定の入れ物」のことを **Promise（プロミス、約束／英語の "promise" は『約束』）** と言います。

Promise は3つの状態を持ちます。

- **pending**（ペンディング／保留中）: まだ結果が出ていない。
- **fulfilled**（フルフィルド／成功）: 結果（値）が確定した。
- **rejected**（リジェクテッド／失敗）: エラーで失敗した。

```
今すぐ返せる値: ── 1 + 2 ── 即 3 が返る
非同期の値:    ── DB問い合わせ ── ⏳ Promise が返る ── 数百ms後に中身が確定
```

▼ たとえ話: Promise は「フードコートの呼び出しベル」

フードコートで注文すると、料理の代わりにまず「呼び出しベル」を渡されますね。ベルを持っている間、料理はまだ手元にありません（**pending**）。ベルが鳴ったら料理を受け取り（**fulfilled**）、品切れなら店員さんから「ごめんなさい」の連絡が来ます（**rejected**）。`supabase.from("books").select("*")` を呼んだ直後に返ってくるのは、料理（データ）ではなく、この「ベル」＝ Promise なのです。

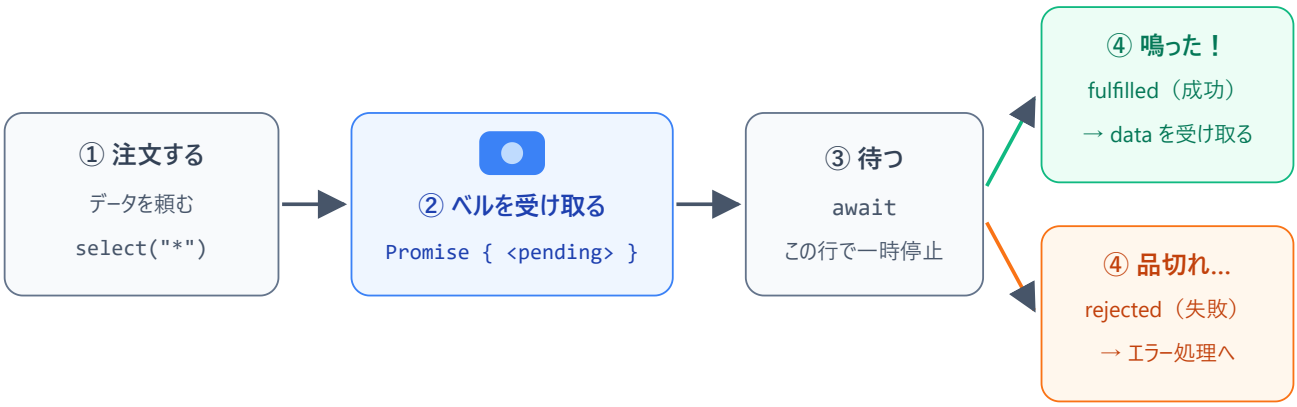



図7-3: Promise は「呼び出しベル」。await は「ベルが鳴るまで待つ」こと

## 0.3 async / await で「待つ」

`await` を関数呼び出しの前に書くと、「Promiseの中身が確定するまでこの行で待ってね」という意味になります。`await` を使う関数には **必ず** `async` を付ける必要があります。`async` の付いていない関数の中で `await` を書くと、TypeScript / JavaScript の文法エラーになります。

```
// ❌ await なし: data には Promise オブジェクトが入る
// 普通の関数（async が無い）
const fetchBooks = () => {
  // Promise が data に入る（中身はまだ無い）
  const data = supabase.from("books").select("*");
  // 本のデータではなく「あとで結果が入る入れ物」が表示される（中身はまだ無い）
  // ※ supabase-js では PostgrestFilterBuilder { ... } という Promise のなかまが表示される
  console.log(data);
};

// ✅ async/await あり: data に実際のデータが入る
// async を付けたので await が使える関数になる
const fetchBooks = async () => {
  const { data } = await supabase.from("books").select("*");
  // await により Promise の中身が確定するまで停止 → 確定した結果から data プロパティを取り出す（分割代入）
  // [{ id: 1, title: "..." }, ...] のような配列が出る
  console.log(data);
};
```

▼ 概念図:

```
async function fetchBooks() {
  const { data } = await supabase.from("books").select("*");
  //              └─ ここで「結果が来るまで待つ」（pendingからfulfilledへ変わる瞬間）
  console.log(data);  // ←──── 結果が来てからこの行が実行される
}
```

> **重要なポイント:** `await` を付けた関数を呼び出した瞬間、その関数自体も Promise を返します。つまり「async関数は常に Promise を返す」というルールがあります。`async function f() { return 1; }` を呼ぶと、戻り値は数値の 1 ではなく、中身が 1 の Promise になります。

### 動かして確かめる: await を付ける・付けないの違い

> **▼ このコードがやること（先に日本語で）:** 本物の Supabase の代わりに「0.3秒後に本の一覧を返す、にせものの関数」を作ります。それを `await` を付けずに呼んだ場合と、付けて呼んだ場合で、受け取れる値がどう違うかをターミナルに表示します。Supabase のアカウントが無くても、Node.js だけで試せます。

```ts
// 本物の Supabase の代わりに使う「にせものの取得関数」
// 0.3秒後に書籍の配列を返す（ネット越しの問い合わせをまねている）
function fakeSelectBooks(): Promise<{ title: string }[]> {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([{ title: "リーダブルコード" }, { title: "JavaScript Primer" }]);
    }, 300);
  });
}

// ❌ await を付けずに呼んだ場合
function withoutAwait() {
  const data = fakeSelectBooks();
  console.log("await なし:", data);
}

// ✅ async 関数の中で await を付けて呼んだ場合
async function withAwait() {
  const data = await fakeSelectBooks();
  console.log("await あり:", data);
}

// 2つを順番に試す
async function main() {
  withoutAwait();
  await withAwait();
  console.log("おしまい");
}

main();
```

▼ **実行結果**（`await-demo.ts` という名前で保存し、ターミナルで `node await-demo.ts` を実行）

```
await なし: Promise { <pending> }
await あり: [ { title: 'リーダブルコード' }, { title: 'JavaScript Primer' } ]
おしまい
```

> **ヒント:** Node.js 22.18 以降（2026年時点の LTS 版を含む）では、`.ts` ファイルを `node ファイル名.ts` で直接実行できます。ただし「型の部分を消してから実行する」だけなので、**型のまちがいはチェックされません。**型チェックは VS Code の赤線や `npx tsc --noEmit` が担当します。

## ▼ コードを1つずつ分解して解説

**解説1: Promise を返す「にせもの」の関数**

```
function fakeSelectBooks(): Promise<{ title: string }[]> {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([{ title: "リーダブルコード" }, { title: "JavaScript Primer" }]);
    }, 300);
  });
}
```

- 戻り値の型 `Promise<{ title: string }[]>` は「あとで『本の配列』が入る予定のベル」という意味です。
- `new Promise((resolve) => { ... })` でベルを作り、`setTimeout` で 300 ミリ秒（0.3秒）待ってから `resolve(...)` を呼びます。`resolve` を呼んだ瞬間にベルが鳴り、中身が確定（fulfilled）します。

> **用語:** `setTimeout(関数, ミリ秒)`（セット・タイムアウト）とは「指定した時間が経ってから関数を1回実行する」機能。ブラウザでも Node.js でも使えます。ここではネット越しの待ち時間をまねるために使っています。

**解説2: await なしだと「ベル」そのものが入る**

```
function withoutAwait() {
  const data = fakeSelectBooks();
  console.log("await なし:", data);
}
```

- `await` を付けないと、`data` には本の配列ではなく **Promise（ベル）そのもの**が入ります。実行結果の `Promise { <pending> }` は「まだ中身が無いベル」という表示です。
- この状態で `data.length` や `data.map(...)` と書いても、配列ではないので期待どおりに動きません。

**解説3: await ありなら「中身」が入る**

```
async function withAwait() {
  const data = await fakeSelectBooks();
  console.log("await あり:", data);
}
```

- `await` を付けると、ベルが鳴るまでこの行で待ち、**鳴ったあとの中身（本の配列）**が `data` に入ります。
- `main` の中で `await withAwait();` と書いているので、「おしまい」は必ず本の一覧が表示されたあとに出ます。もし `await` を外すと「おしまい」が先に表示されます（ぜひ試してみてください）。

> **補足（supabase-js の場合）**：supabase-js の `supabase.from("books").select("*")` が返すのは、正確には Promise と同じように `await` できる「Promise のなかま（thenable／ゼナブル）」です。しかも **`await`（または `.then()`）されるまで通信そのものが始まりません**。つまり `await` を書き忘れると、データが取れないだけでなく、Supabase への問い合わせ自体が行われないのです。

## 0.4 try / catch でエラーを捕まえる

非同期処理は「サーバーが落ちている」「ネットワーク切断」「権限が無い」などで失敗することがあります。`try / catch` で失敗を捕まえます。

```
// async関数の定義
const fetchBooks = async () => {
  // この中で起きたエラーは下の catch に飛ぶ
  try {
    // 非同期処理を await で待つ
    const { data, error } = await supabase.from("books").select("*");
    // Supabaseは「成功でも例外を投げない」設計。errorプロパティを自分で投げ直す。
    if (error) throw error;
    // 例外が出なかった＝成功。data を使える。
    console.log("成功:", data);
  // try ブロック内で throw された値や、await中の例外がここに来る
  } catch (err) {
    // 開発者ツール（コンソール）に赤文字でエラーを出す
    console.error("失敗:", err);
  }
};
```

**▼ 出力イメージ:**

成功時:

```
成功: [ { id: 1, title: "...", ... }, { id: 2, ... } ]
```

失敗時（テーブル名を `bookz` と間違えた場合）：

```
失敗: {
  code: 'PGRST205',
  details: null,
  hint: "Perhaps you meant the table 'public.books'",
  message: "Could not find the table 'public.bookz' in the schema cache"
}
```

> **補足:** 以前の Supabase では、同じまちがいが `{ code: '42P01', message: 'relation "bookz" does not exist' }` という形で返っていました。現在の Supabase（内部の PostgREST という部品の新しい版）では上のように `PGRST205`（テーブルが見つからない）が返り、`hint`（ヒント）に「もしかして `public.books` のこと？」という候補まで入ります。エラーが出たら `message` だけでなく `hint` も読む習慣をつけましょう。

> **本書での使い方:** Supabaseの `.from()` などはすべて非同期なので、`async` 関数の中で `await` を付けて呼びます。エラーは `try/catch` で捕まえ、画面に「エラーが発生しました」と表示します。

### 動かして確かめる: Supabase の error は catch に来ない

上のコードで `if (error) throw error;` と、わざわざ自分で投げ直していたのに気づきましたか？ これは **supabase-js が、失敗しても例外を投げない（throw しない）設計**だからです。失敗は `{ data: null, error: {...} }` という「戻り値」で知らされます。本当にそうなのか、にせものの関数で確かめてみましょう。

> **▼ このコードがやること（先に日本語で）:** supabase-js と同じく「失敗しても throw せず、`error` に理由を入れて返す」にせものの関数を作ります。それを ① `try / catch` だけで囲んだ書き方（NG）と、② `error` を自分で調べる書き方（OK）で呼び比べ、どちらが失敗に気づけるかを確かめます。

```ts
// supabase-js の戻り値と同じ「{ data, error } の形」を表す型
type Result = {
  data: { title: string }[] | null;
  error: { code: string; message: string } | null;
};

// にせものの取得関数。テーブル名が "books" 以外なら error を返す（throw はしない）
async function fakeFrom(table: string): Promise<Result> {
  if (table !== "books") {
    return {
      data: null,
      error: {
        code: "PGRST205",
        message: `Could not find the table 'public.${table}' in the schema cache`,
      },
    };
  }
  return { data: [{ title: "リーダブルコード" }], error: null };
}

// ❌ NG: try / catch だけで安心してしまう書き方
async function ngVersion(table: string) {
  try {
    const { data } = await fakeFrom(table);
    console.log("NG版: data =", data);
    console.log("NG版: 成功したつもりで先へ進んでしまう");
  } catch (err) {
    console.log("NG版: catch に来た", err);
  }
}

// ✅ OK: error を自分でチェックする書き方
async function okVersion(table: string) {
  const { data, error } = await fakeFrom(table);
  if (error) {
    console.log("OK版: エラーを検出 →", error.code, error.message);
    return;
  }
  const books = data ?? [];
  console.log("OK版: 取得件数 =", books.length);
}

async function main() {
  // テーブル名をわざと間違える（books → bookz）
  await ngVersion("bookz");
  await okVersion("bookz");
  // 正しいテーブル名
  await okVersion("books");
}
```

```
main();
```

▼ 実行結果（`node data-error-demo.ts`）

```
NG版: data = null
NG版: 成功したつもりで先へ進んでしまう
OK版: エラーを検出 → PGRST205 Could not find the table 'public.bookz' in the schema cache
OK版: 取得件数 = 1
```

## ▼ コードを1つずつ分解して解説

**解説1: 失敗を「戻り値」で知らせる関数**

```
type Result = {
  data: { title: string }[] | null;
  error: { code: string; message: string } | null;
};
```

- `data` と `error` は「どちらか片方だけに中身が入る」2つの箱です。成功なら `data` に配列・`error` は `null`、失敗なら `data` は `null`・`error` に理由が入ります。
- `fakeFrom` は失敗しても `throw` せず、この形のオブジェクトを `return` するだけです。本物の supabase-js も同じ振る舞いをします。

**解説2: NG 版は catch に来ない**

```
  try {
    const { data } = await fakeFrom(table);
    console.log("NG版: data =", data);
    console.log("NG版: 成功したつもりで先へ進んでしまう");
  } catch (err) {
```

- `catch` は「throw された例外」を捕まえる仕組みです。`fakeFrom` は何も throw していないので、**`catch` には一度も来ません。**
- 実行結果のとおり、`data` が `null` のまま「成功したつもり」で次の行へ進んでしまいます。この先で `data.map(...)` などを書いていたら、そこで初めて「null の map は呼べない」という分かりにくいエラーになります。

解説3: OK 版は if (error) で必ず調べる

```
  const { data, error } = await fakeFrom(table);
  if (error) {
    console.log("OK版: エラーを検出 →", error.code, error.message);
    return;
  }
  const books = data ?? [];
```

- 受け取った直後に `if (error)` で失敗かどうかを調べ、失敗ならその場で `return`（早期 return）します。
- 成功した場合も、念のため `data ?? []` で「null なら空配列」にしてから使います。これは 1e のトップページでも使う書き方です。

> **つまずきポイント:「try / catch で囲んだから安心」は supabase-js では通用しない**
>
> supabase-js の失敗は `error` という戻り値で届きます。`try / catch` だけでは気づけません。

```
// ❌ NG: error を受け取っていない。失敗しても books が null になるだけで気づけない
const { data: books } = await supabase.from("books").select("*");
```

```
// ✅ OK: error も受け取り、すぐに調べる
const { data: books, error } = await supabase.from("books").select("*");
if (error) {
  console.error("書籍の取得に失敗しました:", error);
}
```

> 実は supabase-js は、**通信そのものが切れたときでさえ throw せず**、`error` に `TypeError: Failed to fetch` のような理由を入れて返します。`try / catch` が役に立つのは、自分のコードのミス（`null` の中身を読もうとした等）や、環境変数の設定漏れでクライアント作成自体が失敗した、といった supabase-js の外側の例外です。この章の登録ページ（4b）では、`if (error)` と `try / catch` の**両方**を使って二重に備えています。

---

## 1. 一覧表示機能の実装

一覧表示は、データベースに保存された書籍をカード形式で並べて表示する機能です。以下のコンポーネント構成で実装します。

```
app/page.tsx（トップページ / Server Component）
  └── BookList（書籍一覧）
        └── BookCard（書籍カード）× N冊分
              ├── StatusBadge（ステータスバッジ）
              └── RatingStars（星評価）
```

この「入れ子」を絵にすると、次のようになります。外側の部品が内側の部品を**箱のように包み**、データ（props）は**外から内へ、一方通行で**渡されていきます。




図7-4: 一覧ページの部品の入れ子と、props（データ）が外から内へ渡っていく流れ

小さなコンポーネントから順に作っていきましょう。

---

## 1a. StatusBadge コンポーネント

**ファイル:** `components/StatusBadge.tsx`

書籍のステータス（読書中・読了・読みたい）をバッジ形式で表示するコンポーネントです。ステータスの値に応じて背景色とテキストが変わります。

> **▼ このコードがやること（先に日本語で）:** 「読書中」「読了」「読みたい」という**ステータスを、色付きの小さなラベル（バッジ）に変換して表示する**部品を作ります。カギは「ステータスごとの色や文字を1つの辞書（オブジェクト）にまとめておき、本体はそこから引くだけ」という作り。こうすると `if` 文で分岐を書かずに済み、ステータスを増やすときも辞書に1行足すだけで済みます。初心者は「**渡された status の値で見た目が変わる**」という1点を押さえれば十分です（細かい仕組みはコード内のコメントで説明します）。

```tsx
// ============================================================================
// ファイルパス: src/components/StatusBadge.tsx
// 役割       : 書籍のステータス（読書中／読了／読みたい）を、
//              色付きの小さなバッジとして画面に表示するための部品（コンポーネント）
// 親コンポーネント: BookCard が使う
// ============================================================================

/**
 * StatusBadge - 書籍の読書ステータスをバッジ形式で表示するコンポーネント
 *
 * 表示されるバッジの種類:
 *  - reading      : 「読書中」  （青色）
 *  - completed    : 「読了」    （緑色）
 *  - want_to_read : 「読みたい」（黄色）
 *
 * 受け取るデータ:
 *  - status: 上記3種類のうちどれか1つ（文字列）
 *
 * 出力（画面に出るもの）:
 *  - 角丸の小さなラベル（spanタグ）。文字色と背景色がステータスごとに違う。
 */

// ----------------------------------------------------------------------------
// (1) 「ステータスの型」を定義する
// ----------------------------------------------------------------------------
// TypeScript の「ユニオン型（|）」を使って
// 「BookStatus は文字列だけど、3つのうちのどれか」というルールを作る。
//
// こう書くことで、誤って "reading2" など typo のある文字列を渡したときに
// VS Code がコードを書いている最中に赤線で警告してくれる。
// 「export」を付けているので、ほかのファイル（BookCardなど）からも import して使える。
export type BookStatus = "reading" | "completed" | "want_to_read";

// ----------------------------------------------------------------------------
// (2) このコンポーネントが「親から受け取るプロパティ（Props）」の形を定義
// ----------------------------------------------------------------------------
// Reactでは、親コンポーネントから子コンポーネントに渡すデータを「Props」と呼ぶ。
// ここでは「status」というキーで BookStatus 型の値を1つ受け取る、と宣言している。
//
// 例: <StatusBadge status="reading" />  ← こう呼び出される。
type StatusBadgeProps = {
  status: BookStatus;
};

// ----------------------------------------------------------------------------
// (3) ステータスごとの表示設定をオブジェクトでまとめる
// ----------------------------------------------------------------------------
// もし if文 や switch文 で「reading の場合は青、completed の場合は緑...」と
// 書くと、新しいステータスを追加するたびに分岐を増やす必要があり保守が大変。
```

```
//
// そこで「ステータス → { 表示文字, 背景色, 文字色 }」という辞書（オブジェクト）を
// 作っておくと、コンポーネント本体はこの辞書を引くだけで済む。
//
// `Record<BookStatus, ...>` は「キーは BookStatus のどれか、値は { ... }」という
// TypeScriptの記法で、辞書全体の型を保証する。
//
// `bg-blue-100` などは Tailwind CSS のクラス名。
//   bg-       = background-color
//   blue-100  = 100段階で薄めの青（数字が大きいほど濃い）
//   text-blue-800 = テキストカラーを濃い青に
const statusConfig: Record<
  BookStatus,
  { label: string; bgColor: string; textColor: string }
> = {
  reading: {
    // バッジに表示する日本語
    label: "読書中",
    // 背景: 薄い青
    bgColor: "bg-blue-100",
    // 文字: 濃い青
    textColor: "text-blue-800",
  },
  completed: {
    label: "読了",
    // 背景: 薄い緑（完了感のある色）
    bgColor: "bg-green-100",
    textColor: "text-green-800",
  },
  want_to_read: {
    label: "読みたい",
    // 背景: 薄い黄
    bgColor: "bg-yellow-100",
    textColor: "text-yellow-800",
  },
};

// ---------------------------------------------------------------------------
// (4) コンポーネント本体
// ---------------------------------------------------------------------------
// `export default` で「このファイルの主役はこの関数だよ」と宣言。
// 関数の引数を `{ status }` と書いているのは「分割代入」と呼ばれる JS の文法。
//   - 引数として渡されたオブジェクト（StatusBadgeProps型）から
//     `status` という名前のプロパティだけを取り出す。
//   - 同等の書き方:
//       function StatusBadge(props: StatusBadgeProps) {
//         const status = props.status;
//         ...
//       }
export default function StatusBadge({ status }: StatusBadgeProps) {
```

```tsx
  // (4-1) 渡された status をキーにして、設定オブジェクトを引く。
  //
  // ?? は「Nullish Coalescing 演算子」と呼ばれ、
  // 左辺が null か undefined の場合に右辺を使うという意味。
  // 万が一 statusConfig に存在しないキー（不正な値）が来た場合のための保険。
  const config = statusConfig[status] ?? {
    label: "不明",
    bgColor: "bg-gray-100",
    textColor: "text-gray-800",
  };

  // (4-2) JSX を返す。これが画面に描かれるHTML（厳密にはReactが生成するDOM）。
  //
  // <span> はインライン要素（行の中で使う小さな箱）。
  // className に Tailwind のクラスを並べることで、CSSを別ファイルに書かずに
  // スタイルを当てられる。
  //
  // ${...} の部分はテンプレートリテラル。
  // configの値（変数）を文字列の中に埋め込んでいる。
  return (
    <span
      className={`
        /* 中身（テキスト）を上下中央に揃える */
        inline-flex items-center
        /* 内側の余白（padding x方向 / y方向）*/
        px-2.5 py-0.5
        /* 角を完全に丸めてカプセル型に */
        rounded-full
        /* 文字サイズを小さく、太さは中ぐらい */
        text-xs font-medium
        /* 背景色（ステータスにより変わる） */
        ${config.bgColor}
        /* 文字色（ステータスにより変わる） */
        ${config.textColor}
      `}
    >
      {/* {config.label} はJSXで「JS式を埋め込む」記法。
          ここでは "読書中" などの文字列が出力される */}
      {config.label}
    </span>
  );
}
// ============================================================================
// この StatusBadge.tsx を <StatusBadge status="reading" /> のように書くと:
//
// 出力されるHTML（概念）:
//   <span class="inline-flex ... bg-blue-100 text-blue-800">読書中</span>
//
// 画面の見た目:
//   ┌──────────┐
```

```
//   |  読書中  |  ← 青背景・青文字の角丸ラベル
//    ──────────
// ================================================================================
```

### ▼ どこから呼び出されるの？

このコンポーネントは、後で出てくる `BookCard.tsx` の中から次のように使われます。

```
import StatusBadge from "@/components/StatusBadge";

// ...
<StatusBadge status={book.status} />
// 例: book.status が "reading" のとき、青いバッジが描画される
```

**画面にはこのように表示されます:**

StatusBadge は小さな角丸のバッジとして表示されます。

- 「**読書中**」の場合: 薄い青色の背景に、濃い青色のテキストで「読書中」と表示されます。見た目は `[ 読書中 ]` のような角丸の小さなラベルです。
- 「**読了**」の場合: 薄い緑色の背景に、濃い緑色のテキストで「読了」と表示されます。完了を連想させる緑色です。
- 「**読みたい**」の場合: 薄い黄色の背景に、濃い黄色（オレンジ寄り）のテキストで「読みたい」と表示されます。「まだ読んでいない＝これから」を連想させる色です。

いずれも `text-xs`（12px 程度）の小さめのフォントサイズで、`rounded-full` により完全な角丸（カプセル型）になります。カードの中に配置すると、ステータスがひと目で分かるアクセントになります。

## ▼ コードを1つずつ分解して解説

上の `StatusBadge` には、初心者がつまずきやすい書き方がいくつか入っています。意味の塊ごとに、順番にていねいに見ていきましょう。

---

#### 解説1: ステータスの「型」をユニオン型で定義する

```
export type BookStatus = "reading" | "completed" | "want_to_read";
```

- `type`（タイプ）は TypeScript で「**新しい型に名前を付ける**」キーワードです。ここでは `BookStatus` という型を作っています。
- `"reading" | "completed" | "want_to_read"` の `|`（パイプ）は「**または**」を表す「ユニオン型」です。「この3つの文字列のうちのどれか1つ」という意味になります。
- こう書いておくと、誤って `"readign"`（タイプミス）のような値を渡したとき、VS Code がコードを書いている最中に赤線で警告してくれます。

- 先頭の `export`（エクスポート）は「**他のファイルからもこの型を使えるようにする**」印です。後で `BookCard` などがこの型を `import` して使います。

> **用語:** ユニオン型（union type）とは「いくつかの候補のうちのどれか」を表す型のこと。`"A" | "B" | "C"` のように `|` でつなぐと「決まった文字列のどれか」に限定でき、タイプミスを未然に防げる。

---

**解説2: 親から受け取る Props の形を定義する**

```
type StatusBadgeProps = {
  status: BookStatus;
};
```

- これは「このコンポーネントが**親から受け取る値（Props）の形**」を決めている部分です。
- `status: BookStatus` は「`status` という名前のプロパティを1つ受け取り、その値は `BookStatus` 型（＝解説1の3択）」という意味です。
- 例えば `<StatusBadge status="reading" />` のように呼び出されると、`status` に `"reading"` が渡ってきます。

> **用語:** Props（プロパティ）とは、親コンポーネントから子コンポーネントへ渡すデータのこと。HTML タグの属性（`<input type="text">` の `type` など）に似た書き方で渡す。

---

解説3: ステータスごとの表示設定を「辞書（オブジェクト）」にまとめる

```
const statusConfig: Record<
  BookStatus,
  { label: string; bgColor: string; textColor: string }
> = {
  reading: {
    label: "読書中",
    bgColor: "bg-blue-100",
    textColor: "text-blue-800",
  },
  completed: {
    label: "読了",
    bgColor: "bg-green-100",
    textColor: "text-green-800",
  },
  want_to_read: {
    label: "読みたい",
    bgColor: "bg-yellow-100",
    textColor: "text-yellow-800",
  },
};
```

- ここがこのコンポーネントの**心臓部**です。「ステータス → 表示文字・背景色・文字色」の対応表（辞書）をオブジェクトで作っています。
- もし `if` 文や `switch` 文で「`reading` なら青、`completed` なら緑…」と書くと、ステータスを追加するたびに分岐を増やす必要があり面倒です。辞書にしておけば、**新しいステータスは1ブロック足すだけ**で済みます。
- `Record<BookStatus, { ... }>` は「**キーは `BookStatus` のどれか、値は `{ label, bgColor, textColor }` という形**」という辞書全体の型です。これにより「3つのステータス全部の設定を書き忘れていないか」を TypeScript がチェックしてくれます。
- `bg-blue-100` や `text-blue-800` は Tailwind CSS のクラス名です。`bg-` は背景色、`text-` は文字色で、数字が大きいほど色が濃くなります。

> **用語:** `Record<K, V>`（レコード）とは「キーが `K` 型、値が `V` 型のオブジェクト」を表す TypeScript の型。ここでは「ステータス名をキーに、設定オブジェクトを値に持つ辞書」を型として表現している。

**解説4: コンポーネント本体で辞書を引く**

```
export default function StatusBadge({ status }: StatusBadgeProps) {
  const config = statusConfig[status] ?? {
    label: "不明",
    bgColor: "bg-gray-100",
    textColor: "text-gray-800",
  };
```

- `export default function StatusBadge(...)` で「**このファイルの主役の部品**」を1つ宣言しています。
- 引数の `{ status }` は「**分割代入**」です。親から渡された Props オブジェクトから `status` だけを取り出しています。`props.status` と書くのと同じ意味ですが、こちらのほうが短く書けます。
- `statusConfig[status]` で、渡された `status` をキーにして解説3の辞書から設定を引いています。例えば `status` が `"reading"` なら `{ label: "読書中", ... }` が取り出されます。
- `?? { label: "不明", ... }` の `??`（ヌル合体演算子）は「**左辺が `null` か `undefined` のときだけ右辺を使う**」という意味です。万が一、辞書にないステータスが来てもアプリが壊れないようにする「保険」です。

> **用語:** `??`（Nullish Coalescing 演算子・ヌル合体演算子）とは「左の値が `null` または `undefined` のときだけ右の値を使う」演算子。`||`（OR）と似ているが、`||` は `0` や `""` も右辺に切り替えてしまうのに対し、`??` は `null` / `undefined` のときだけ切り替える点が違う。

---

**解説5: JSX で色付きバッジを描画する**

```
  return (
    <span
      className={`
        inline-flex items-center
        px-2.5 py-0.5
        rounded-full
        text-xs font-medium
        ${config.bgColor}
        ${config.textColor}
      `}
    >
      {config.label}
    </span>
  );
}
```

- `<span>` は「行の中で使える小さな箱」を表す HTML 要素です。バッジのような小さなラベルにぴったりです。
- `className={ ... }` の中はテンプレートリテラル（バッククォート文字列）です。固定のクラス（`rounded-full` など）と、解説4で決めた `config.bgColor`・`config.textColor` を `${ }` で埋め込んで連結しています。

- これにより「ステータスによって背景色・文字色が変わるバッジ」が完成します。`rounded-full` で完全な角丸（カプセル型）に、`text-xs` で小さめの文字になります。
- `{config.label}` は「`config` の `label`（"読書中" など）を画面に表示する」部分です。JSX では `{ }` の中に JavaScript の値を埋め込めます。

> **用語:** テンプレートリテラルとは、バッククォート `` ` `` で囲む文字列のこと。中に `${変数}` と書くと、その変数の値を文字列に埋め込める。ここでは Tailwind のクラス名を動的に組み立てるために使っている。

---

## 1b. RatingStars コンポーネント

**ファイル:** `components/RatingStars.tsx`

書籍の評価（1〜5）を星マーク（★☆）で視覚的に表示するコンポーネントです。

> **▼ このコードがやること（先に日本語で）:** 「3」のような**数字の評価を、★★★☆☆ という星の見た目に変換して表示する**部品を作ります。やり方は「1から5までの星を並べ、評価値以下の星は黄色で塗り、それより大きい星はグレーにする」だけです。評価が `null`（未入力）のときは星ではなく「評価なし」と出す分岐も用意します。初心者は「**数字を見た目に変換しているだけ**」と捉えればOKです（星を並べる `map` や色分けの三項演算子はコード内コメントで解説します）。

```tsx
// components/RatingStars.tsx  ← このファイルのパス（コメントとして書いておくと分かりやすい）

/**
 * RatingStars - 1〜5の評価を星マークで表示するコンポーネント
 *
 * 例: rating=3 の場合 → ★★★☆☆
 *
 * rating が null の場合は「評価なし」と表示する。
 * これは「評価が未入力の本」を表現できるようにするため。
 */

// Props（プロパティ：親から渡される値）の型を定義。
// rating は数値、もしくは null（評価がないことを表す）のどちらかを受け取る。
// 「number | null」は TypeScript の「ユニオン型」で、「AかBどちらか」という意味。
type RatingStarsProps = {
  rating: number | null;
};

// export default = このファイルの主役の関数。他のファイルから import RatingStars from "./RatingStars" で読み込める。
// 引数で { rating } と分割代入しているので、props.rating ではなく rating だけで使える。
export default function RatingStars({ rating }: RatingStarsProps) {
  // 評価がない場合はテキストで表示する。
  // === は「型まで含めて完全一致」する比較演算子。==（緩い比較）よりも安全なので必ずこちらを使う。
  // null と undefined はどちらも「値が無い」ことを表すが、別物として扱われるため両方チェックしている。
  if (rating === null || rating === undefined) {
    return (
      // 早期 return で、以降の処理を実行せずにこの JSX を返して終わる。
      // text-sm = 小さめの文字、text-gray-400 = 薄いグレー（控えめなトーン）。
      <span className="text-sm text-gray-400">
        評価なし
      </span>
    );
  }

  // 1〜5 の範囲に収める（データベースの制約と合わせる）。
  // Math.max(1, rating)  = rating と 1 の大きい方 → 1未満なら 1 に押し上げる
  // Math.min(5, 上記結果) = 上記結果と 5 の小さい方 → 5を超えるなら 5 に押し戻す
  // 結果として「1以上5以下」に強制される。これを「クランプ（clamp、挟み込み）」と呼ぶ。
  const clampedRating = Math.min(5, Math.max(1, rating));

  return (
    // flex = 横並び、items-center = 縦方向中央揃え、gap-0.5 = 子要素間の隙間。
    // aria-label はスクリーンリーダー（視覚障害者向け読み上げ機能）に伝える説明文。
    // `${clampedRating}点` はテンプレートリテラル：バッククォート ` で囲み、${} で変数を埋め込める。
    <div className="flex items-center gap-0.5" aria-label={`評価: ${clampedRating}点`}>
      {/* 星を5つ並べる。rating以下のインデックスは塗りつぶし、それ以外は空の星。
```

```
          [1, 2, 3, 4, 5].map(...) は「配列の各要素に関数を適用して新しい配列を作る」メソッド。
          ここでは数値1〜5に対して <span> を作って配列にしている。 */}
        {[1, 2, 3, 4, 5].map((star) => (
          <span
            // key は React が「どの要素がどれか」を識別するための必須プロパティ。
            // map で要素を並べる時は必ず key を付ける（付けないと警告が出る）。
            key={star}
            // 三項演算子（条件 ? A : B）で「条件が真なら A、偽なら B」を選ぶ。
            // star（1〜5）が現在の評価値以下なら「塗りつぶし（黄色）」、それより大きいなら「空（グレー）」。
            className={`text-lg ${
              star <= clampedRating
                // 塗りつぶしの星（黄色）
                ? "text-yellow-400"
                // 空の星（グレー）
                : "text-gray-300"
            }`}
          >
            {/* 実は同じ「★」を出している。色だけクラスで切り替えることで塗りつぶし/空を表現。 */}
            ★
          </span>
        ))}
        {/* 数値も併記する（スクリーンリーダーや視認性のため）。
            ml-1 は left margin（左の外側余白）。星の右に少し離して数値を置く。 */}
        <span className="ml-1 text-sm text-gray-600">
          ({clampedRating})
        </span>
      </div>
    );
}
```

**画面にはこのように表示されます:**

- **rating=4** の場合: `★★★★☆ (4)` — 黄色い星4つとグレーの星1つ、その横に小さく `(4)` と数値が表示されます。
- **rating=null** の場合: グレーの文字で `評価なし` と表示されます。

### ▼ コードを1つずつ分解して解説

`RatingStars` には「未入力の分岐」「数値の範囲調整」「星を並べる map」など、押さえておきたい塊があります。順番に見ていきましょう。

---

**解説1: Props の型（数値 または null）**

```
type RatingStarsProps = {
  rating: number | null;
};
```

- 親から受け取る `rating` （評価）の型を定義しています。
- `number | null` は「**数値、または `null` （評価が無い）**」というユニオン型です。`|` は「または」を表します。
- なぜ `null` も許すのかというと、「**まだ評価を付けていない本**」を表現できるようにするためです。評価が無いことを `0` で表すと「最低評価」と区別がつかなくなるため、あえて `null` を使います。

> **用語:** `null` （ヌル）とは「値が存在しないこと」を意図的に表す特別な値。ここでは「評価が未入力」という状態を `null` で表している。

---

**解説2: 評価が無いときの早期 return**

```
export default function RatingStars({ rating }: RatingStarsProps) {
  if (rating === null || rating === undefined) {
    return (
      <span className="text-sm text-gray-400">
        評価なし
      </span>
    );
  }
```

- 引数の `{ rating }` は分割代入で、Props から `rating` だけを取り出しています。
- `if (rating === null || rating === undefined)` で「**評価が無い場合**」をまず判定しています。`===` は「型まで含めて完全に等しいか」を比べる厳密比較で、`==` より安全なので常にこちらを使います。
- 条件に当てはまれば、星ではなくグレーの文字で「評価なし」と表示し、その場で `return` して関数を終わらせます。これを「**早期 return（early return）**」と呼びます。先に例外的なケースを片付けることで、以降のコードを「評価が必ずある前提」でシンプルに書けます。

> **用語:** 早期 return（early return）とは、特殊な条件を関数の最初で処理して `return` で抜けてしまう書き方。残りのコードが「正常系（普通のケース）」だけに集中できるので読みやすくなる。

---

**解説3: 評価を1〜5に収める（クランプ）**

```
  const clampedRating = Math.min(5, Math.max(1, rating));
```

- 評価の値を「**必ず1以上5以下**」に強制する処理です。
- `Math.max(1, rating)` は「`rating` と `1` の大きいほう」を返すので、1未満の値が来たら `1` に押し上げます。
- その結果に `Math.min(5, ...)` を掛けると「5より大きければ `5` に押し戻す」ので、最終的に「1〜5の範囲」に収まります。
- このように「ある範囲に値を挟み込む」処理を「**クランプ（clamp）**」と呼びます。データベース側の制約（1〜5）と画面表示を確実に一致させる安全策です。

> **用語:** クランプ（clamp）とは、値を「下限と上限の範囲内に収める」操作のこと。`Math.min` と `Math.max` を組み合わせて実現する定番のテクニック。

---

**解説4: 星を5つ並べる（map と三項演算子）**

```
return (
  <div className="flex items-center gap-0.5" aria-label={`評価: ${clampedRating}点`}>
    {[1, 2, 3, 4, 5].map((star) => (
      <span
        key={star}
        className={`text-lg ${
          star <= clampedRating
            ? "text-yellow-400"
            : "text-gray-300"
        }`}
      >
        ★
      </span>
    ))}
```

- `[1, 2, 3, 4, 5].map((star) => ...)` は「**1〜5の数値それぞれに対して `<span>` を作り、星の配列を作る**」処理です。`map` は「配列の各要素を別のものに変換して新しい配列を作る」メソッドです。
- `key={star}` は、`map` で要素を並べるときに React が「どの星がどれか」を識別するための**必須の属性**です。付けないと警告が出ます。
- `star <= clampedRating ? "text-yellow-400" : "text-gray-300"` は三項演算子で、「**今の星番号が評価値以下なら黄色、そうでなければグレー**」を選んでいます。例えば評価が3なら、星1〜3は黄色、星4〜5はグレーになります。
- 実は塗りつぶしも空も同じ「★」を出しており、**色だけを切り替える**ことで「塗られた星／空の星」を表現しています。
- `aria-label={` 評価: ${clampedRating}点 `}` は、目で見えない人向けの読み上げ用テキストです。星の見た目だけでなく数値でも評価を伝えられます。

> **用語:** `aria-label`（エリア・ラベル）とは、スクリーンリーダー（画面読み上げソフト）に要素の意味を伝えるための属性。視覚的なUI（星マーク）を、音声でも分かるように補足する。

**解説5: 数値を併記する**

```
      <span className="ml-1 text-sm text-gray-600">
        ({clampedRating})
      </span>
    </div>
  );
}
```

- 星の右側に `(4)` のように**評価の数値そのもの**を小さく表示する部分です。
- `ml-1`（margin-left）で星から少し離し、`text-sm text-gray-600` で控えめな小さいグレー文字にしています。
- 星だけだと「4つなのか5つなのか」一瞬迷うことがあるため、数値を併記して**確実に伝わる**ようにしています。

> **用語:** `ml-1` の `ml` は margin-left（左外側の余白）の略。Tailwind では `m`（margin）+ 方向（`l`=left, `r`=right, `t`=top, `b`=bottom）+ 数値、という規則でクラス名が付く。

---

### 動かして確かめる: 評価を星の文字列に変えてみる

`RatingStars` の中身は「数字 → 星の見た目」の変換です。画面（JSX）を外して、変換のロジックだけを Node.js で動かすと、クランプや `map` の働きがよく見えます。

> **▼ このコードがやること（先に日本語で）:** `RatingStars` と同じルール（null なら「評価なし」、1〜5 に収める、評価値以下の番号は塗りつぶし）で、評価を `★★★★☆ (4)` のような文字列に変える関数を作ります。範囲外の値（0 や 9）や `null` も渡して、どう表示されるかを確かめます。

```ts
// RatingStars と同じ考え方で、評価を「★☆」の文字列に変える関数
function ratingToStars(rating: number | null): string {
  // 評価が無いときは早期 return
  if (rating === null) {
    return "評価なし";
  }
  // 1〜5 の範囲に収める（クランプ）
  const clamped = Math.min(5, Math.max(1, rating));
  // 1〜5 の番号ごとに「塗る★／塗らない☆」を決めて、1本の文字列につなげる
  const stars = [1, 2, 3, 4, 5]
    .map((star) => (star <= clamped ? "★" : "☆"))
    .join("");
  return `${stars} (${clamped})`;
}

// いろいろな評価値で試す（0 と 9 は範囲外、null は未入力）
const samples = [4, 1, 5, 0, 9, null];

for (const rating of samples) {
  console.log(String(rating).padStart(4), "→", ratingToStars(rating));
}
```

▼ 実行結果（`node stars-demo.ts`）

```
   4 → ★★★★☆ (4)
   1 → ★☆☆☆☆ (1)
   5 → ★★★★★ (5)
   0 → ★☆☆☆☆ (1)
   9 → ★★★★★ (5)
null → 評価なし
```

## ▼ コードを1つずつ分解して解説

**解説1: map で「番号の配列」を「星の配列」に変える**

```ts
  const stars = [1, 2, 3, 4, 5]
    .map((star) => (star <= clamped ? "★" : "☆"))
    .join("");
```

- `[1, 2, 3, 4, 5].map(...)` は、5つの番号それぞれを「★ か ☆」に変換し、`["★", "★", "★", "★", "☆"]` のような**新しい配列**を作ります（評価が 4 の場合）。
- `.join("")` は配列の要素を区切り文字なしでつなげて、1本の文字列 `"★★★★☆"` にします。
- `RatingStars` では `join` の代わりに、配列の各要素を `<span>` にして JSX の中に並べています。「配列を map で変換する」という骨組みは同じです。

**解説2: クランプで範囲外の値を丸める**

```
const clamped = Math.min(5, Math.max(1, rating));
```

- 実行結果の `0 → ★☆☆☆☆ (1)` と `9 → ★★★★★ (5)` に注目してください。範囲外の値が来ても、1〜5 に収められてから星になっています。
- データベースには CHECK 制約（第5章）があるので本来は 1〜5 しか入りませんが、「表示する側でも念のため守る」のが安全な作り方です。

> **用語:** `padStart(4)` （パッド・スタート）とは、文字列の**左側**を空白で埋めて、4文字の幅にそろえるメソッド。実行結果の矢印の位置をそろえるためだけに使っています。

---

## 1c. BookCard コンポーネント

**ファイル:** `components/BookCard.tsx`

1冊分の書籍情報をカード形式で表示するコンポーネントです。上で作った StatusBadge と RatingStars を組み合わせて使います。

> **▼ このコードがやること（先に日本語で）：書籍1冊分の情報（タイトル・著者・出版社・ステータス・評価など）を、1枚のカードにまとめて表示する**部品を作ります。ポイントは、さきほど作った `StatusBadge` と `RatingStars` を**子として組み合わせて使う**こと——小さな部品を寄せ集めて大きな部品を作る考え方（コンポジション）の実例です。さらにカード全体を `<Link>` で囲み、どこをクリックしても詳細ページへ飛ぶようにします。初心者は「**1冊分のデータを受け取り、カードの形に並べているだけ**」と押さえれば十分です（各部分の意味はコード内コメントにあります）。

```tsx
// ============================================================================
// ファイルパス: src/components/BookCard.tsx
// 役割       : 1冊分の書籍をカード形式で表示する。クリックで詳細ページへ遷移。
// 親         : BookList コンポーネントから1冊ずつ渡されて表示される
// ----------------------------------------------------------------------------
// このコンポーネントは、これまでに作った StatusBadge と RatingStars を
// 子として組み合わせて使う「コンポジション（合成）」の好例。
// ============================================================================

// (1) 必要なものを import
//     Next.js の Link: クライアントサイド遷移用のリンク要素（フルリロードしない）
//     ./StatusBadge  : 同フォルダの StatusBadge.tsx（拡張子は省略可能）
//     `type BookStatus` : 値ではなく「型」だけを取り込む記法
import Link from "next/link";
import StatusBadge, { type BookStatus } from "./StatusBadge";
import RatingStars from "./RatingStars";

/**
 * BookCard - 書籍1冊分の情報をカード形式で表示するコンポーネント
 *
 * 表示する情報:
 *   - タイトル
 *   - 著者
 *   - 出版社（あれば）
 *   - ステータスバッジ
 *   - 評価（星）
 *
 * カード全体が <Link> で囲まれており、どこをクリックしても /books/{id} へ遷移する。
 */

// (2) Book 型の定義
//     データベースの books テーブルの1行に対応する型。
//     publisher など null OK のカラムは「string | null」のユニオン型で表す。
//     export しているので他ファイルから import { type Book } で使える。
export type Book = {
  // UUID（主キー）
  id: string;
  // 必須
  title: string;
  // 必須
  author: string;
  // 任意
  publisher: string | null;
  // ISO日付文字列（"2024-01-15" など）
  published_date: string | null;
  // 1〜5 または null
  rating: number | null;
  // "reading" | "completed" | "want_to_read"
  status: BookStatus;
```

```
  // 感想メモ
  memo: string | null;
  // 作成日時（自動）
  created_at: string;
  // 更新日時（自動）
  updated_at: string;
};

// (3) Props の型
//     親（BookList）から book を1つ受け取る、というだけの単純な型。
type BookCardProps = {
  book: Book;
};

// (4) コンポーネント本体
//     export default で「このファイルの主役」を1つだけ宣言。
//     関数引数 ({ book }: BookCardProps) は分割代入＋型注釈。
export default function BookCard({ book }: BookCardProps) {
  return (
    // (5) Link はカード全体を「クリック可能なリンク」にする要素
    //     href={`/books/${book.id}`} のテンプレートリテラルで動的にURLを組み立てる
    //     /books/123e4567-e89b-12d3-a456-426614174000 のような遷移先になる
    //
    //     className のクラス文字列の意味（Tailwind CSS）:
    //       block              ← <Link>（普段はインライン）をブロック要素にする
    //       bg-white           ← 背景色を白に
    //       rounded-lg         ← 角丸（大きめ）
    //       shadow-md          ← 中ぐらいのドロップシャドウ
    //       hover:shadow-lg    ← マウスを乗せたら影を強くする
    //       transition-shadow  ← 影の変化を滑らかにアニメーション
    //       duration-200       ← アニメーション時間 200ms
    //       p-6                ← 全方向の内側余白 24px
    //       border             ← 1px の枠線を表示
    //       border-gray-200    ← 枠線の色
    //       hover:border-blue-300 ← ホバー時に枠線の色を変える
    <Link
      href={`/books/${book.id}`}
      className="
        block
        bg-white
        rounded-lg
        shadow-md
        hover:shadow-lg
        transition-shadow
        duration-200
        p-6
        border
        border-gray-200
        hover:border-blue-300
      "
```

```
    >
      {/*
        (6) 1行目: タイトル と StatusBadge を左右に配置
            flex                ← フレックスレイアウト
            items-start         ← 上端揃え（タイトルが2行になっても揃う）
            justify-between     ← 左右の端に寄せる
            gap-2               ← 子要素間の隙間
            mb-3                ← 下方向の外側余白
      */}
      <div className="flex items-start justify-between gap-2 mb-3">
        {/*
          line-clamp-2 = テキストが長くても2行で「...」省略する Tailwind 補助クラス
          flex-1       = 余白を最大限取る（バッジに押し出されないようにする）
        */}
        <h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
          {book.title}
        </h2>

        {/*
          (7) 子コンポーネント StatusBadge を呼び出す。Props として status を渡す
              <StatusBadge status="reading" /> のように展開される。
        */}
        <StatusBadge status={book.status} />
      </div>

      {/* (8) 著者名 */}
      <p className="text-sm text-gray-600 mb-1">
        <span className="font-medium text-gray-500">著者:</span>{" "}
        {book.author}
      </p>

      {/*
        (9) 「book.publisher && ...」は短絡評価。
            publisher が null/空 のときは何も表示せず、値があるときだけ <p> を描画。
      */}
      {book.publisher && (
        <p className="text-sm text-gray-600 mb-1">
          <span className="font-medium text-gray-500">出版社:</span>{" "}
          {book.publisher}
        </p>
      )}

      {/* (10) 出版日（null チェックは publisher と同じパターン） */}
      {book.published_date && (
        <p className="text-sm text-gray-600 mb-3">
          <span className="font-medium text-gray-500">出版日:</span>{" "}
          {book.published_date}
        </p>
      )}
```

```
        {/*
          (11) 評価（星）
               mt-3 pt-3 border-t border-gray-100 で
               「カード本体と評価エリアの間に薄い区切り線を入れる」効果。
        */}
        <div className="mt-3 pt-3 border-t border-gray-100">
          <RatingStars rating={book.rating} />
        </div>

        {/* (12) メモが存在すれば、最大2行までプレビュー表示 */}
        {book.memo && (
          <p className="mt-2 text-xs text-gray-400 line-clamp-2">
            {book.memo}
          </p>
        )}
      </Link>
    );
}
```

**1枚のカードはこのように表示されます:**

カードは白い背景（`bg-white`）に薄いグレーのボーダー（`border-gray-200`）と影（`shadow-md`）がついた角丸の矩形です。マウスカーソルを乗せると影が濃くなり（`hover:shadow-lg`）、ボーダーが薄い青色（`hover:border-blue-300`）に変わります。これにより「クリックできる」ことが視覚的に伝わります。

カードの内部構成は上から順に:

1. **1行目（タイトル行）**: 左にタイトルが太字で表示され、右端にステータスバッジが配置されます。タイトルが長い場合は2行まで表示され、それ以降は `...` で省略されます（`line-clamp-2`）。
2. **2行目**: 「著者: ○○」のように著者名が小さめのフォントで表示されます。
3. **3行目**（出版社がある場合）: 「出版社: ○○」と表示されます。
4. **4行目**（出版日がある場合）: 「出版日: 2024-01-15」のように表示されます。
5. **区切り線の下**: 薄いグレーの区切り線の下に星評価（例: `★★★★☆ (4)`）が表示されます。
6. **最下部**（メモがある場合）: メモの先頭部分が薄いグレーで2行まで表示されます。

カード全体が `<Link>` で囲まれているため、どこをクリックしても `/books/[id]` の詳細ページに遷移します。

### ▼ コードを1つずつ分解して解説

`BookCard` は、これまで作った `StatusBadge` と `RatingStars` を**子として組み合わせて使う**点が新しいところです。塊ごとに見ていきましょう。

---

**解説1: 必要なものを import する**

```
import Link from "next/link";
import StatusBadge, { type BookStatus } from "./StatusBadge";
import RatingStars from "./RatingStars";
```

- `import`（インポート）は「**別のファイルで作ったものを、このファイルで使えるように読み込む**」命令です。
- `Link` は Next.js が用意するリンク用の部品です。普通の `<a>` よりも画面遷移が速くなります（移動先を先読みする「プリフェッチ」機能のため）。
- `StatusBadge, { type BookStatus }` は「`StatusBadge` という部品（デフォルトエクスポート）と、`BookStatus` という型を同時に読み込む」書き方です。`type` を付けると「**型だけを読み込む**」という意味になり、ビルド時に消えるので無駄がありません。

> **用語:** デフォルトエクスポートと名前付きエクスポート。`import StatusBadge from "./StatusBadge"` のように名前を自由に付けて読み込めるのが「デフォルトエクスポート」（1ファイル1つ）。`import { type BookStatus }` のように `{ }` で囲み、決まった名前で読み込むのが「名前付きエクスポート」（1ファイルに複数可）。

---

**解説2: Book 型の定義**

```
export type Book = {
  id: string;
  title: string;
  author: string;
  publisher: string | null;
  published_date: string | null;
  rating: number | null;
  status: BookStatus;
  memo: string | null;
  created_at: string;
  updated_at: string;
};
```

- データベースの `books` テーブルの**1行分のデータの形**を `Book` という型で定義しています。
- `publisher: string | null` のように `| null` が付いているものは「**値が無いこともある（任意）**」項目です。出版社やメモは未入力でもよいので `null` を許しています。
- `id: string`（UUID）や `created_at`（作成日時）はデータベースが自動で付ける値です。
- `export` を付けているので、`BookList` や `page.tsx` などほかのファイルからこの `Book` 型を `import` して使い回せます。

> **用語:** UUID（ユーユーアイディー）とは「世界中で重複しないように作られた長いID文字列」のこと（例: `123e4567-e89b-...`）。連番より推測されにくく、データの一意な目印（主キー）に向いている。

---

**解説3: Propsの型とコンポーネント本体**

```
type BookCardProps = {
  book: Book;
};

export default function BookCard({ book }: BookCardProps) {
```

- `BookCardProps` は「親（`BookList`）から `book` を1つ受け取るだけ」というシンプルな Props の型です。
- `export default function BookCard({ book }: BookCardProps)` で本体を定義し、引数の分割代入で `book`（1冊分のデータ）を取り出しています。
- 以降の JSX では、この `book.title` や `book.author` のように各項目を取り出して表示していきます。

> **用語:** 分割代入とは、オブジェクトや配列から必要な要素だけを取り出して変数にする書き方。`{ book }` は「渡された Props オブジェクトの `book` プロパティを `book` という変数にする」という意味。

---

**解説4: カード全体をクリック可能なリンクにする**

```
    <Link
      href={`/books/${book.id}`}
      className="
        block
        bg-white
        rounded-lg
        shadow-md
        hover:shadow-lg
        transition-shadow
        duration-200
        p-6
        border
        border-gray-200
        hover:border-blue-300
      "
    >
```

- `<Link>` でカード全体を囲むことで、**カードのどこをクリックしても詳細ページへ移動**できるようにしています。

- `href={`/books/${book.id}`}` はテンプレートリテラルで、`book.id` を埋め込んで `/books/123...` のような遷移先URLを動的に組み立てています。
- `className` のクラスは見た目の指定です。`bg-white`（白背景）、`rounded-lg`（角丸）、`shadow-md`（影）、`hover:shadow-lg`（ホバーで影を濃く）、`hover:border-blue-300`（ホバーで枠線を青く）などにより、「クリックできるカード」だと視覚的に伝わります。

> **用語:** `hover:` プレフィックス（Tailwind）は「**マウスカーソルを乗せたときだけ適用するスタイル**」を表す。`hover:shadow-lg` は「ホバー時に影を大きくする」という意味になる。

---

**解説5: 子コンポーネントを組み合わせて中身を作る（コンポジション）**

```
      <div className="flex items-start justify-between gap-2 mb-3">
        <h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
          {book.title}
        </h2>
        <StatusBadge status={book.status} />
      </div>

      {book.publisher && (
        <p className="text-sm text-gray-600 mb-1">
          <span className="font-medium text-gray-500">出版社:</span>{" "}
          {book.publisher}
        </p>
      )}

      <div className="mt-3 pt-3 border-t border-gray-100">
        <RatingStars rating={book.rating} />
      </div>
```

- 1行目では `flex justify-between` で「左にタイトル、右にステータスバッジ」を両端に配置しています。`<StatusBadge status={book.status} />` のように、**さきほど作った部品を子として呼び出して**いるのがポイントです。
- `{book.publisher && (...)}` は短絡評価で、「`publisher` **に値があるときだけ**出版社の行を表示」します。`null` や空文字のときは何も表示されません（出版日・メモも同じパターン）。
- 区切り線の下では `<RatingStars rating={book.rating} />` を呼び、星評価を表示しています。
- このように「小さな部品（`StatusBadge`・`RatingStars`）を寄せ集めて大きな部品（`BookCard`）を作る」考え方を「**コンポジション（合成）**」と呼びます。

> **用語:** コンポジション（composition・合成）とは、小さなコンポーネントを組み合わせて大きなコンポーネントを作る設計手法。1つの巨大な部品を作るより、再利用しやすく見通しもよくなる。

---

## 1d. BookList コンポーネント

**ファイル:** `components/BookList.tsx`

BookCard を並べて表示するコンポーネントです。書籍が0冊の場合のメッセージも含みます。

> **▼ このコードがやること（先に日本語で）：書籍の配列を受け取り、`BookCard` を冊数分だけ並べて一覧表示する**入れ物（コンテナ）を作ります。カギは2つ——① `map` で「配列の各書籍を1枚のカードに変換」して並べること、②書籍が**0冊のときは空状態（empty state）として案内メッセージと登録ボタンを出す**分岐を持つこと。並べ方は CSS Grid で、画面幅に応じて1〜3列に自動で変わります。初心者は「**カードを並べる係**」と「**0冊のときの案内係**」の2役だと押さえればOKです（詳細はコード内コメントにあります）。

```tsx
// components/BookList.tsx  ← ファイルパス

// 同フォルダの BookCard を読み込む。Book 型も同時に取り込む（コンポーネントと型を同一ファイルから export しているのは効率のため）。
// 「type」キーワードを付けると「型だけインポート」になり、ビルド時に消えるので JS バンドルサイズが小さくなる。
import BookCard, { type Book } from "./BookCard";

/**
 * BookList - 書籍一覧をグリッドレイアウトで表示するコンポーネント
 *
 * - 書籍が1冊以上ある場合: カードをグリッド状に並べる
 * - 書籍が0冊の場合: 「書籍が登録されていません」メッセージを表示
 *
 * レスポンシブ対応（responsive：画面幅で見た目を変える）:
 * - スマホ（デフォルト）: 1列
 * - タブレット（md = medium breakpoint：768px〜）: 2列
 * - デスクトップ（lg = large breakpoint：1024px〜）: 3列
 */

// Props 型：親（page.tsx など）から books 配列を受け取るだけのシンプルな型。
// 「Book[]」は「Book型を要素とする配列」という意味。
type BookListProps = {
  books: Book[];
};

export default function BookList({ books }: BookListProps) {
  // 書籍が0冊の場合の「空状態（empty state）」表示。
  // books.length は配列の要素数。0なら登録なし。
  if (books.length === 0) {
    return (
      // text-center で中央寄せ、py-16 で上下に大きめの余白を取って「主役感」を出す。
      <div className="text-center py-16">
        {/* 大きなアイコン的テキスト。絵文字を巨大表示することで視覚的なアクセントにする。 */}
        <p className="text-6xl mb-4">📚</p>
        <h2 className="text-xl font-bold text-gray-700 mb-2">
          書籍が登録されていません
        </h2>
        <p className="text-gray-500 mb-6">
          「新規登録」ボタンから最初の書籍を登録しましょう。
        </p>
        {/* <a> は通常のリンク。<Link> でも書けるが、ここでは Next.js のインポートを増やさないため普通の a タグにしている。
            ※本格運用では <Link> を使った方がページ間遷移が高速になる（プリフェッチ機能のため）。
*/}
        <a
          href="/books/new"
          className="
            /* aタグ（普段インライン）をブロック寄りにして余白を効かせる */
```

```
            inline-block
            /* 背景: 濃い青 */
            bg-blue-600
            /* 文字色: 白 */
            text-white
            /* 内側余白: 横6 / 縦3（Tailwindの単位は4px基準） */
            px-6 py-3
            /* 角丸 */
            rounded-lg
            /* 文字の太さ: 中 */
            font-medium
            /* マウスホバー時にもう一段濃い青へ */
            hover:bg-blue-700
            /* 色の変化を滑らかに */
            transition-colors
            /* アニメーション時間 200ms */
            duration-200
          "
        >
          最初の書籍を登録する
        </a>
      </div>
    );
  }

  // 書籍が1冊以上ある場合の表示（早期 return しなかったので、ここに到達）。
  return (
    <div
      // grid と grid-cols-X で CSS Grid Layout を使う。
      // grid-cols-1            : デフォルト（スマホ）は1列
      // md:grid-cols-2         : 中サイズ画面以上では2列に切り替え
      // lg:grid-cols-3         : 大画面では3列に切り替え
      // gap-6                  : セル間の隙間（24px）
      className="
        grid
        grid-cols-1
        md:grid-cols-2
        lg:grid-cols-3
        gap-6
      "
    >
      {/* books 配列を map で1冊ずつ BookCard に変換して並べる。
          key={book.id} は React の必須要件：「どの要素がどれか」を識別するための一意キー。
          book={book} は Props として book オブジェクトを子に渡している（"book" という名前で受け取る）。 */}
      {books.map((book) => (
        <BookCard key={book.id} book={book} />
      ))}
    </div>
```

```
  );
}
```

**表示の説明:**

- 書籍がある場合: CSS Grid を使って、画面幅に応じて1〜3列にカードが並びます。各カードの間は `gap-6`（24px）の余白があります。
- 書籍がない場合: 画面中央に本の絵文字、「書籍が登録されていません」というメッセージ、そして新規登録ページへのリンクボタンが表示されます。

## ▼ コードを1つずつ分解して解説

`BookList` には「2つの役割（カードを並べる係・0冊のときの案内係）」が詰まっています。塊ごとに見ていきましょう。

---

#### 解説1: BookCard と Book 型を import する

```
import BookCard, { type Book } from "./BookCard";
```

- 同じフォルダの `BookCard.tsx` から、部品 `BookCard` と型 `Book` を**まとめて読み込んで**います。
- `BookCard` は値（部品）、`Book` は型です。`{ type Book }` のように `type` を付けると「型だけのインポート」になり、ビルド後のJavaScriptには残らないため、ファイルサイズが小さくなります。
- このように「コンポーネントと、それに対応する型」を同じファイルから取り込めるのは、`BookCard.tsx` 側で両方を `export` しているからです。

> **用語:** 型だけのインポート（ `import { type X }` ）とは、「実行時には不要な型情報だけを読み込む」書き方。TypeScript はビルド時に型を消すため、この記法で書くと最終的なJSバンドルに余計なコードが含まれない。

---

#### 解説2: Props の型（書籍の配列を受け取る）

```
type BookListProps = {
  books: Book[];
};

export default function BookList({ books }: BookListProps) {
```

- 親（ `page.tsx` など）から `books` という**書籍の配列**を1つ受け取ります。
- `Book[]` の `[]` は「**配列**」を表します。`Book[]` で「 `Book` 型のオブジェクトがいくつも入った配列」という意味になります。
- 本体では分割代入で `books` を取り出し、これから「0冊かどうか」で表示を切り替えていきます。

> **用語:** 配列型（ 型[] ）とは「その型の要素が並んだリスト」を表す記法。`Book[]` は「Book がいくつも入った配列」、`string[]` は「文字列の配列」となる。

---

**解説3: 0冊のときの「空状態（empty state）」表示**

```
if (books.length === 0) {
  return (
    <div className="text-center py-16">
      <p className="text-6xl mb-4">📚</p>
      <h2 className="text-xl font-bold text-gray-700 mb-2">
        書籍が登録されていません
      </h2>
      <p className="text-gray-500 mb-6">
        「新規登録」ボタンから最初の書籍を登録しましょう。
      </p>
      <a href="/books/new" className="...">
        最初の書籍を登録する
      </a>
    </div>
  );
}
```

- `books.length === 0` は「**配列の要素数が0、つまり書籍が1冊も無い**」状態を判定しています。`.length` は配列の個数です。
- 0冊なら、空っぽの画面に「📚 書籍が登録されていません」というメッセージと、登録ページへのボタンを表示して、その場で `return` します（早期 return）。
- このように「**データが無いときに専用の案内を出す**」UIを「**空状態（empty state）**」と呼びます。何も出さずに真っ白だとユーザーが不安になるため、次に何をすればよいかを示すのが親切な設計です。

> **用語:** 空状態（empty state）とは、「表示するデータがゼロ件のときに見せる専用の画面」のこと。案内文や次の行動ボタンを置くことで、ユーザーが迷わないようにする。

---

**解説4: 1冊以上あるときのグリッド表示と map**

```
  return (
    <div
      className="
        grid
        grid-cols-1
        md:grid-cols-2
        lg:grid-cols-3
        gap-6
      "
    >
      {books.map((book) => (
        <BookCard key={book.id} book={book} />
      ))}
    </div>
  );
}
```

- 解説3で早期 return しなかった場合（1冊以上ある場合）に、ここへ到達します。
- `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3` は CSS Grid のクラスで、「**スマホでは1列、タブレットでは2列、デスクトップでは3列**」と画面幅に応じてカードの列数を自動で変えます。これを「レスポンシブ対応」と呼びます。
- `books.map((book) => <BookCard ... />)` で、配列の**各書籍を1枚の `BookCard` に変換**して並べています。
- `key={book.id}` は React がリスト要素を識別するための必須属性で、各書籍の一意なID（`book.id`）を使っています。`book={book}` で、その1冊分のデータを子に渡しています。

> **用語:** レスポンシブ対応（responsive）とは、画面幅に応じてレイアウトを自動的に変える設計のこと。Tailwind では `md:`（中サイズ以上）・`lg:`（大サイズ以上）などのプレフィックスで「画面幅ごとの指定」を書ける。

**▼ 画面イメージ: 書籍が0冊のとき（空状態）**

アプリを作ったばかりで本が1冊も無いときは、解説3の早期 return によって次の画面が表示されます。何も無い真っ白な画面ではなく、「次に何をすればよいか」が分かる画面になっているのがポイントです。




図7-5: 書籍が0冊のときの画面イメージ（空状態 / empty state）

### 動かして確かめる: map の中で `{ }` を使うと何も返らない

`books.map(...)` は、この章で何度も出てくる大事な書き方です。そして初心者が**一番多く「画面に何も出ない」原因を作る場所**でもあります。JSX を外した小さな例で確かめてみましょう。

> **▼ このコードがやること（先に日本語で）**：本の配列を `map` で「カード: タイトル」という文字列の配列に変換します。そのとき、矢印関数（ `=>` ）の右側を `{ }` で囲んで `return` を書き忘れた場合（NG）と、正しく書いた場合（OK を2通り）を比べます。

```
const books = [
  { id: "a1", title: "リーダブルコード" },
  { id: "b2", title: "JavaScript Primer" },
];

// ❌ NG: { } で囲んだのに return を書いていない
const ng = books.map((book) => {
  `カード: ${book.title}`;
});

// ✅ OK その1: ( ) で囲む（または何も囲まない）と、式の値がそのまま返る
const ok1 = books.map((book) => `カード: ${book.title}`);

// ✅ OK その2: { } を使うなら return を書く
const ok2 = books.map((book) => {
  return `カード: ${book.title}`;
});

console.log("NG :", ng);
console.log("OK1:", ok1);
console.log("OK2:", ok2);
```

▼ 実行結果（`node map-demo.ts`）

```
NG : [ undefined, undefined ]
OK1: [ 'カード: リーダブルコード', 'カード: JavaScript Primer' ]
OK2: [ 'カード: リーダブルコード', 'カード: JavaScript Primer' ]
```

## ▼ コードを1つずつ分解して解説

**解説1: `=> { }` は「処理のかたまり」なので return が要る**

```
const ng = books.map((book) => {
  `カード: ${book.title}`;
});
```

- 矢印関数の右側を `{ }` で囲むと、そこは「何行でも処理を書ける**ブロック**」になります。ブロックの中では、`return` と書いた値しか外に返りません。
- この NG 例では文字列を作っただけで返していないので、2冊分とも `undefined`（値が無い）になりました。**エラーは1つも出ない**ので、気づきにくいのが怖いところです。

**解説2: `=> ( )` は「値そのもの」なので自動で返る**

```
const ok1 = books.map((book) => `カード: ${book.title}`);
```

- 矢印関数の右側に**式（値になるもの）**を直接書くと、その値が自動的に返ります（暗黙の return）。
- JSX は複数行になることが多いので、`BookList` では `(book) => ( <BookCard ... /> )` のように `( )` で囲んで「ここからここまでが1つの値だよ」と示しています。

> **つまずきポイント: map の中で `{ }` を使って、一覧が真っ白になる**
>
> JSX でも同じことが起きます。`undefined` は画面に何も描かないので、「データは取れているのにカードが1枚も出ない」という症状になります。

```
{/* ❌ NG: { } の中に JSX を書いただけ。return が無いので何も表示されない */}
{books.map((book) => {
  <BookCard key={book.id} book={book} />;
})}
```

```
{/* ✅ OK: ( ) で囲んで JSX をそのまま返す（BookList の書き方） */}
{books.map((book) => (
  <BookCard key={book.id} book={book} />
))}
```

> 見分け方は「`=>` のすぐ右が `{` か `(` か」です。`{` なら、中に `return` があるか必ず確認しましょう。

> **つまずきポイント: key を付け忘れる・並び順（index）を key にする**
>
> `key` は React がカード1枚1枚を見分けるための「名札」です。名札が無いと、ブラウザの開発者ツールに `Each child in a list should have a unique "key" prop.` という警告が出ます。

```
{/* ❌ NG: key が無い → 警告が出る */}
{books.map((book) => (
  <BookCard book={book} />
))}
```

```
{/* △ 避けたい: 並び順（index）を名札にする → 本が増えると名札がずれる */}
{books.map((book, index) => (
  <BookCard key={index} book={book} />
))}
```

```
{/* ✅ OK: データベースの id（変わらない・重複しない）を名札にする */}
{books.map((book) => (
  <BookCard key={book.id} book={book} />
))}
```

> このアプリの一覧は「新しい順」なので、本を1冊登録するたびに**先頭に**新しいカードが入ります。index を名札にすると、既存の本の名札が全部1つずつずれてしまい、React が「どれが同じカードか」を取り違える原因になります。`book.id` なら並び順が変わっても名札は変わりません。

## 1e. トップページ

**ファイル:** `app/page.tsx`

Supabase からデータを取得して BookList に渡すトップページです。Next.js の **Server Component** として実装します。

> **▼ このコードがやること（先に日本語で）**：アプリのトップページとして、**Supabase から書籍一覧を取ってきて、`BookList` に渡して画面に表示する**部分を作ります。カギは、これが **Server Component（サーバー側で動く部品）**であること――だから関数に `async` を付け、`await` でデータ取得を待ってから、完成したHTMLをブラウザに送れます。データベースの秘密情報がブラウザに漏れず、初回表示も速い、というメリットがあります。初心者は「**サーバーでデータを取り、それを一覧部品に渡しているだけ**」と押さえれば十分です（取得処理やエラー処理の詳細はコード内コメントにあります）。

```tsx
// app/page.tsx  ← Next.js App Router の規約で「ルート（/）に対応するページ」になる

// Next.js が提供する Link コンポーネント。<a> よりも遷移が速い（プリフェッチ機能あり）。
import Link from "next/link";
// サーバー用 Supabase クライアント。cookies などの「サーバーしか知らない情報」を扱えるバージョン。
import { createClient } from "@/lib/supabase/server";
// 自作の一覧表示コンポーネント。「@/」は tsconfig.json で設定した「プロジェクトルート」のエイリアス。
import BookList from "@/components/BookList";
// Book 型だけを取り込む。値ではなく型なので「type」キーワード付き。
import { type Book } from "@/components/BookCard";

/**
 * トップページ（Server Component）
 *
 * このコンポーネントは Server Component として動作する。
 * つまり、サーバー側で Supabase からデータを取得し、
 * HTML をレンダリングしてからクライアントに送信する。
 *
 * "use client" を書いていない = Server Component（Next.js App Router のデフォルト）
 *
 * 重要な特徴:
 *  - 関数定義に async が付いている → 内部で await が使える
 *  - サーバー上で動くので、Supabase の秘密鍵（service role 等）も安全に使える
 *  - JavaScript としてブラウザに送信されないので、バンドルサイズが小さい
 *  - useState や onClick などの「インタラクティブな機能」は使えない（必要なら子要素に Client Component を置く）
 */
export default async function HomePage() {
  // Supabase クライアントを作成（サーバー用）。
  // この createClient は内部で cookies() を呼ぶので async（await が必要）。
  const supabase = await createClient();

  // books テーブルから全件取得（作成日の降順＝新しい順）。
  //   .from("books")                          : 操作対象テーブルを指定
  //   .select("*")                            : 全カラムを取得（SELECT * 相当）
  //   .order("created_at", { ascending: false }) : created_at で並び替え、false なので降順（新しい順）
  // await で結果を待つと、{ data, error } という形のオブジェクトが返る。
  // 「data: books」は「data プロパティを books という別名で取り出す」分割代入のリネーム記法。
  const { data: books, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  // エラーが発生した場合の処理。
  // Supabase は通常エラーを throw せず、戻り値の error プロパティに格納する設計。
  if (error) {
    // console.error はサーバーのターミナルにログを出す（ブラウザではなくサーバー側に表示される）。
```

```
    console.error("書籍の取得に失敗しました:", error.message);
    // エラー時でも画面は表示する（空の一覧として）。
    // 本番アプリでは throw error して error.tsx で受け止める設計にすることもある。
  }

  // data が null の場合は空配列にする。
  // 「??」（Nullish Coalescing 演算子）は左辺が null か undefined の時だけ右辺を使う。
  // これにより books が null でも以降のコードで安心して .length などを呼べる。
  const bookList: Book[] = books ?? [];

  return (
    // min-h-screen = 最低でも画面の高さ分（h = height、screen = ビューポート1画面）
    // bg-gray-50  = 非常に薄いグレー背景（コンテンツの白を引き立てる）
    <div className="min-h-screen bg-gray-50">
      {/* ===== ヘッダー ===== */}
      {/* <header> は意味的なHTML要素（セマンティクス）。ページの上部「ナビ・タイトル」帯に使う。
          shadow-sm = 薄い影、border-b = 下方向の枠線 */}
      <header className="bg-white shadow-sm border-b border-gray-200">
        {/* max-w-7xl = 最大幅 1280px、mx-auto = 左右の余白を自動（中央寄せ）。
            px-4 sm:px-6 lg:px-8 = 横の内側余白を画面幅で段階的に変更（レスポンシブ）。 */}
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          {/* flex で横並び、items-center で縦中央、justify-between で「左右両端に寄せる」。
*/}
          <div className="flex items-center justify-between">
            {/* アプリタイトル（左側） */}
            <div>
              {/* sm:text-3xl は「sm（640px）以上の画面では text-3xl」というレスポンシブ指定。
                  狭い画面では text-2xl で控えめに、広い画面では大きく表示する。 */}
              <h1 className="text-2xl sm:text-3xl font-bold text-gray-900">
                書籍管理アプリ
              </h1>
              <p className="mt-1 text-sm text-gray-500">
                あなたの読書記録を管理しましょう
              </p>
            </div>

            {/* 新規登録ボタン（右側）。
                <Link> は内部的に <a> をレンダリングするが、JavaScript で画面遷移を高速化する。
                href="/books/new" は app/books/new/page.tsx に対応。 */}
            <Link
              href="/books/new"
              className="
                /* インライン・フレックス: SVGとテキストを横並び・中央揃え */
                inline-flex items-center gap-2
                bg-blue-600
                text-white
                px-4 py-2.5
                rounded-lg
                font-medium
                text-sm
```

```
                hover:bg-blue-700
                transition-colors
                duration-200
                shadow-sm
              "
            >
              {/* + アイコン。SVG（Scalable Vector Graphics）で「拡大しても綺麗な画像」を描く。
                  w-5 h-5 = 幅・高さともに 20px。
                  fill="none" stroke="currentColor" は「中身は塗らず、線の色は親のテキスト色を引き継ぐ」。
                  viewBox="0 0 24 24" は内部座標系（左上(0,0)〜右下(24,24)）。
                  <path d="M12 4v16m8-8H4" />
                    M12 4 → x=12,y=4 に移動  v16 → 縦に16進む（縦線）
                    m8 -8 → 相対移動  H4 → 水平に x=4 まで進む（横線）
                  結果として「+」マークが描かれる。 */}
              <svg
                className="w-5 h-5"
                fill="none"
                stroke="currentColor"
                viewBox="0 0 24 24"
              >
                <path
                  // 線の端を丸くする
                  strokeLinecap="round"
                  // 線の交点も丸くする
                  strokeLinejoin="round"
                  // 線の太さ
                  strokeWidth={2}
                  d="M12 4v16m8-8H4"
                />
              </svg>
              新規登録
            </Link>
          </div>
        </div>
      </header>

      {/* ===== メインコンテンツ ===== */}
      {/* <main> はページの主要コンテンツを表すセマンティック要素。 */}
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        {/* 書籍数の表示 */}
        <div className="mb-6">
          <p className="text-sm text-gray-600">
            {/* {bookList.length} で配列の要素数（書籍冊数）を JSX に埋め込む。
                JSX では中括弧 {} の中に JavaScript 式を書ける。 */}
            全 <span className="font-bold text-gray-900">{bookList.length}</span> 冊
          </p>
        </div>
```

```
        {/* 書籍一覧 - 子コンポーネント BookList に books 配列を Props として渡す。
            BookList は中で 0冊なら空状態、1冊以上ならカードグリッドを表示する。 */}
        <BookList books={bookList} />
      </main>
    </div>
  );
}
```

**画面全体はこのように表示されます:**

画面は大きく **ヘッダー** と **メインコンテンツ** の2つの領域に分かれています。背景は薄いグレー（`bg-gray-50`）です。

**ヘッダー部分:**

- 白い背景（`bg-white`）に薄い影（`shadow-sm`）と下ボーダーがついた帯状のヘッダーです。
- 左側に「書籍管理アプリ」というアプリタイトルが大きな太字で表示され、その下に「あなたの読書記録を管理しましょう」という小さな説明文があります。
- 右側に青いボタン「＋ 新規登録」が配置されています。ボタンの左にはプラスアイコン（SVG）が表示されます。
- ヘッダーの内容は `max-w-7xl`（1280px）で中央揃えされ、画面幅が広くても広がりすぎません。

**メインコンテンツ部分:**

- ヘッダーの下に「全 N 冊」という書籍数のテキストがあります。数字部分は太字です。
- その下に BookList コンポーネントが配置されます。
- **書籍が登録されている場合**: 白いカードがグリッド状に並びます。スマホでは1列、タブレットでは2列、デスクトップでは3列です。各カードの間には適度な余白があります。
- **書籍が0冊の場合**: 画面中央に本の絵文字と「書籍が登録されていません」というメッセージ、そして「最初の書籍を登録する」ボタンが表示されます。

例として、3冊の書籍が登録されている場合のデスクトップ表示をイメージすると:




## ▼ コードを1つずつ分解して解説

このトップページ（`page.tsx`）は **Server Component**（サーバー側で動く部品）として、Supabase からデータを取って `BookList` に渡します。塊ごとに見ていきましょう。

---

**解説1: import 群**

```
import Link from "next/link";
import { createClient } from "@/lib/supabase/server";
import BookList from "@/components/BookList";
import { type Book } from "@/components/BookCard";
```

- `Link` は Next.js のリンク部品（高速な画面遷移用）です。
- `createClient` は「**サーバー用** の Supabase クライアント」を作る関数です。クライアント用（ブラウザ用）とは別物で、cookie などサーバーしか知らない情報を扱えます。
- `BookList` は自作の一覧表示コンポーネント、`{ type Book }` は `Book` 型だけのインポートです。
- `@/` は「プロジェクトのルートフォルダ」を表すエイリアス（近道の書き方）で、`tsconfig.json` で設定されています。`../../components` のような長い相対パスを書かずに済みます。

> **用語:** パスエイリアス（ `@/` ）とは、長い相対パス（ `../../` ）の代わりに使える「プロジェクト基準の短い書き方」。設定ファイルで `@` がどのフォルダを指すかを決めておくと、どのファイルからでも同じ書き方で import できる。

---

**解説2: async なコンポーネント本体と Supabase クライアント作成**

```
export default async function HomePage() {
  const supabase = await createClient();
```

- 関数に `async`（エイシンク）が付いている点が Server Component ならではです。`async` を付けると関数内で `await` が使えるようになり、**データ取得が終わるのを待ってから**HTMLを組み立てられます。
- 通常のクライアント側コンポーネントの関数には `async` を付けられませんが、Server Component なら付けられます。
- `await createClient()` で、サーバー用の Supabase 接続オブジェクトを用意しています。内部で cookie を読むため `await` が必要です。

> **用語:** Server Component（サーバーコンポーネント）とは、サーバー側でHTMLを組み立ててからブラウザへ送る部品。`async` を付けてデータ取得を待てる反面、`useState` や `onClick` などのブラウザ専用機能は使えない。

---

**解説3: Supabase からデータを取得する**

```
  const { data: books, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });
```

- `await supabase.from("books").select("*").order(...)` で、`books` テーブルから全件取得しています。
- `.from("books")` … 操作対象のテーブル
- `.select("*")` … 全カラムを取得（SQL の `SELECT *` に相当）
- `.order("created_at", { ascending: false })` … 作成日時で並び替え、`ascending: false` なので**降順（新しい順）**
- 結果は `{ data, error }` という形で返ります。`data: books` は「`data` プロパティを `books` という別名で取り出す」分割代入のリネームです。
- `error` には、取得に失敗したときのエラー情報が入ります（成功なら `null`）。

> **用語:** 分割代入のリネーム（`{ data: books }`）とは、オブジェクトからプロパティを取り出すときに別名を付ける書き方。`data` という汎用的な名前を、用途が分かる `books` に付け替えている。

---

**解説4: エラー処理と null ガード**

```
  if (error) {
    console.error("書籍の取得に失敗しました:", error.message);
  }

  const bookList: Book[] = books ?? [];
```

- `if (error)` で、取得が失敗したときだけ `console.error` でサーバーのログにエラーを出します。Supabase は失敗しても例外を投げず `error` プロパティに入れる設計なので、自分でチェックします。
- `console.error` の出力は**サーバーのターミナル**に表示されます（ブラウザのコンソールではありません。Server Component だからです）。
- `const bookList: Book[] = books ?? [];` の `??` は「`books` が `null` や `undefined` なら空配列 `[]` を使う」という意味です。こうしておけば、以降で安心して `.length` や `.map` を呼べます。

> **用語:** null ガードとは、「値が無い（ `null` / `undefined` ）かもしれない変数を、安全な初期値に置き換えておく」処理。`?? []` で「無ければ空配列」にしておくと、後続のコードでエラーになりにくい。

---

**解説5: JSX — ヘッダーとメインを組み立てる**

```
  return (
    <div className="min-h-screen bg-gray-50">
      <header className="bg-white shadow-sm border-b border-gray-200">
        {/* ...タイトルと「+ 新規登録」ボタン... */}
      </header>

      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <div className="mb-6">
          <p className="text-sm text-gray-600">
            全 <span className="font-bold text-gray-900">{bookList.length}</span> 冊
          </p>
        </div>

        <BookList books={bookList} />
      </main>
    </div>
  );
}
```

- 画面を `<header>`（上部の帯）と `<main>`（主要コンテンツ）に分けています。これらは「意味のあるHTML要素（セマンティックタグ）」で、構造を分かりやすくします。
- `{bookList.length}` で書籍の冊数を「全 N 冊」と表示しています。JSX では `{ }` の中に JavaScript の値を埋め込めます。

- `<BookList books={bookList} />` で、取得した書籍配列を子の `BookList` に Props として渡しています。あとは `BookList` が「0冊なら空状態、1冊以上ならカードグリッド」を判断して描画します。

> **用語:** セマンティックタグとは、`<header>`・`<main>`・`<nav>` のように「その部分が何を表すか」を意味するHTMLタグのこと。`<div>` の代わりに使うと、検索エンジンや支援技術にも構造が伝わりやすくなる。

> **つまずきポイント: 取得に失敗したのに「書籍が登録されていません」と表示される**
>
> 上の `HomePage` は、取得に失敗しても `books ?? []` で空配列にして表示を続けます。すると利用者には「全0冊」「書籍が登録されていません」と見えてしまい、「**登録した本が全部消えた！**」と**誤解させる**おそれがあります。失敗したときは、失敗したとはっきり伝えるのが親切です。




図7-6: 一覧の取得に失敗したときの画面イメージ（左: NG / 右: OK）

```
// ❌ NG（HomePage の抜粋）: エラーをログに出すだけで、画面は「0冊」として表示を続ける
if (error) {
  console.error("書籍の取得に失敗しました:", error.message);
}
const bookList: Book[] = books ?? [];
```

```
// ✅ OK（HomePage の抜粋）：エラーなら「読み込めませんでした」という専用の表示を返す
if (error) {
  // サーバーのターミナルには、原因調べのために error を丸ごと出しておく
  console.error("書籍の取得に失敗しました:", error);
  return (
    <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
      {/* role="alert" は、読み上げソフトに「重要なお知らせ」だと伝える属性 */}
      <div role="alert" className="p-4 bg-red-50 border border-red-200 rounded-lg">
        <p className="font-bold text-red-700">書籍を読み込めませんでした</p>
        <p className="mt-1 text-sm text-red-600">
          通信状況を確認して、ページを再読み込みしてください。
        </p>
      </div>
    </main>
  );
}
const bookList: Book[] = books ?? [];
```

- OK 版は、`if (error)` の中で**早期 return** して、一覧の代わりにエラー用の表示を返しています。Next.js の公式ドキュメントも、Server Component で「予想できる失敗」が起きたときは、このように条件分岐でメッセージを表示する方法を紹介しています。
- もう1つの方法は、`if (error)` の中で `throw new Error("書籍の取得に失敗しました")` と例外を投げることです。こうすると、3節で作る `error.tsx`（「もう一度試す」ボタン付きのエラー画面）にページ全体が置き換わります。「画面の一部で知らせたい」なら return、「ページごと再試行させたい」なら throw、と使い分けます（3節の図7-9も参照）。
- `console.error` には `error.message` だけでなく `error` を丸ごと渡すと、`code`・`hint`・`details` も一緒にターミナルに出るので、原因を調べやすくなります。

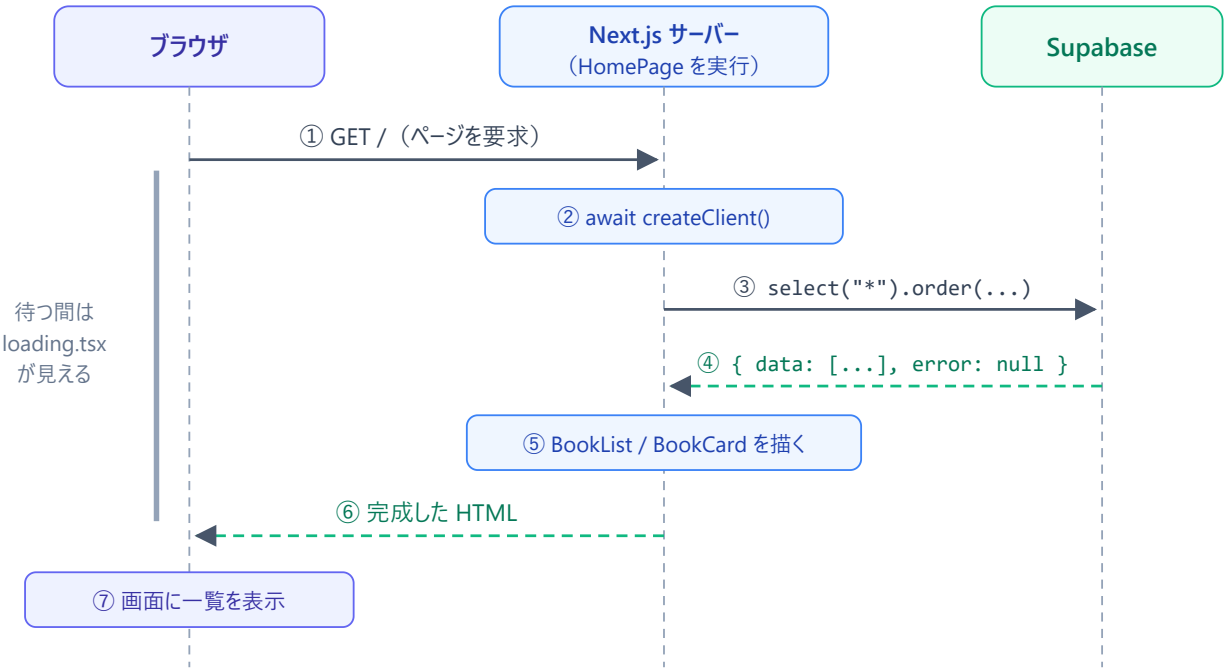
---

## 1f. データ取得フローの図解

以下は、トップページが表示されるまでのデータの流れを、起きる順番に並べた図です。

1. **ブラウザ → Next.js サーバー**
   GET / (トップページをリクエスト)
2. **Next.js サーバー → HomePage (Server Component)**
   Server Component を実行
3. **HomePage → Supabase**
   supabase.from("books").select("*")
4. **Supabase → HomePage**
   書籍データ（JSON配列）を返す
5. **HomePage 内部処理**
   BookList → BookCard をレンダリング（HTML生成）
6. **HomePage → Next.js サーバー**
   完成した HTML
7. **Next.js サーバー → ブラウザ**
   HTML レスポンスを返す
8. **ブラウザ**
   画面にHTMLを表示

同じ流れを、「**誰と誰が**やり取りしているか」が分かる**シーケンス図**（登場人物を横に並べ、時間を上から下へ進めて描く図）にすると、次のようになります。縦の点線が登場人物、横の矢印が「お願い」と「返事」です。




③④ の Supabase とのやり取りはサーバーの中だけで完結し、ブラウザからは見えない

図7-7: トップページが表示されるまでのシーケンス図（上から下へ時間が進む。実線＝お願い、点線＝返事）

**ポイント:**

- Server Component はサーバー側で実行されるため、Supabase へのリクエストもサーバーから行われます。クライアント（ブラウザ）から直接データベースにアクセスすることはありません。
- ブラウザは完成した HTML を受け取って表示するだけなので、初回表示が高速です。
- Supabase のシークレットキーがブラウザに漏れることもありません（サーバー側でのみ使用されるため）。

---

## 2. ローディング状態

Next.js App Router では、`loading.tsx` ファイルを作成すると、ページの読み込み中に自動的にローディング表示を行えます。

### LoadingSpinner コンポーネント

**ファイル:** `components/LoadingSpinner.tsx`

> **▼ このコードがやること（先に日本語で）**：データ読み込み中に表示する、**くるくる回るスピナー（読み込み中マーク）**の部品を作ります。仕組みはシンプルで、円のうち1辺だけ色を変え、それを CSS アニメーション（ `animate-spin` ）で回し続けて「処理中」を表現します。サイズ（小・中・大）やスピナー下のメッセージは Props で切り替えられるようにします。初心者は「**回転する円と説明文を出すだけの部品**」と捉えればOKです（サイズの辞書パターンなどはコード内コメントにあります）。

```tsx
// components/LoadingSpinner.tsx
// このファイル内のコンポーネントは Client Component として扱う宣言。必ず1行目に書く。
"use client";

/**
 * LoadingSpinner - ローディング中に表示するスピナーコンポーネント
 *
 * CSS アニメーション（animate-spin）で回転する円を表示する。
 * "use client" を付けているのは、アニメーションがクライアント側で
 * 実行されるため（厳密には Server Component でも動作するが、
 * 再利用性のために Client Component にしている）。
 */

type LoadingSpinnerProps = {
  /** スピナーのサイズ（デフォルト: "md"）。?: は「省略可能」の意味。 */
  // ユニオン型 = この3つの文字列リテラルのうちどれか
  size?: "sm" | "md" | "lg";
  /** スピナーの下に表示するテキスト。省略可。 */
  message?: string;
};

// 引数で size と message にデフォルト値を設定（呼び出し側で省略された時に使う値）。
export default function LoadingSpinner({
  size = "md",
  message = "読み込み中...",
}: LoadingSpinnerProps) {
  // サイズに応じたクラスを定義（StatusBadge と同じ「辞書パターン」）。
  // キー：サイズ識別子、値：Tailwind の幅・高さ・枠線太さのクラス文字列。
  const sizeClasses = {
    // 24px × 24px、枠線2px
    sm: "w-6 h-6 border-2",
    // 40px × 40px、枠線3px
    md: "w-10 h-10 border-3",
    // 64px × 64px、枠線4px
    lg: "w-16 h-16 border-4",
  };

  return (
    // flex-col = 縦並び（column）、items-center / justify-center = 縦横とも中央揃え。
    <div className="flex flex-col items-center justify-center py-16">
      {/* 回転するスピナー：円形のうち1辺だけ色を変えて、それを回転させることで「読み込み中」を表現する。 */}
      <div
        className={`
          /* 上で選んだサイズクラスを展開（テンプレートリテラル） */
          ${sizeClasses[size]}
          /* 枠線の基本色：薄いグレー */
          border-gray-300
          /* 上辺だけ青に：これが回ると進捗感が出る */
```

```
          border-t-blue-600
          /* 完全な円形に */
          rounded-full
          /* Tailwind のクラスで「回り続ける」アニメーションを適用 */
          animate-spin
        `}
        // スクリーンリーダーに「ステータス表示」と伝える
        role="status"
        // 読み上げ文言
        aria-label="読み込み中"
      />
      {/* メッセージが渡されていれば下に表示。
          「&&」は短絡評価：左辺が真なら右辺の JSX を描画、偽（空文字や undefined）なら何も描画しない。 */}
      {message && (
        <p className="mt-4 text-sm text-gray-500">
          {message}
        </p>
      )}
    </div>
  );
}
```

### ▼ コードを1つずつ分解して解説

`LoadingSpinner` は「サイズの辞書パターン」や「デフォルト値付きの Props」など、再利用しやすくする工夫が入っています。塊ごとに見ていきましょう。

---

#### 解説1: "use client" 宣言

```
"use client";
```

- ファイルの**先頭**に書く特別な1行で、「このファイルの部品は **Client Component**（ブラウザ側で動く部品）として扱う」という宣言です。
- スピナーの回転アニメーション自体はサーバーでも動きますが、いろいろな画面から再利用しやすいよう、あえて Client Component にしています。
- `"use client"` は必ずファイルの一番上（import より前）に書く決まりです。

> **用語:** `"use client"` ディレクティブとは、Next.js App Router で「このファイルはクライアント側で動く」と宣言する目印。これを書かないファイルはデフォルトで Server Component になる。

---

**解説2: Props の型（省略可能なサイズとメッセージ）**

```
type LoadingSpinnerProps = {
  size?: "sm" | "md" | "lg";
  message?: string;
};
```

- `size?` の `?`（クエスチョンマーク）は「**この Props は省略してもよい（オプショナル）**」という意味です。
- `size` の型 `"sm" | "md" | "lg"` はユニオン型で、「小・中・大の3択のどれか」に限定されます。これ以外の文字列を渡すと TypeScript が警告します。
- `message?` も省略可能で、スピナーの下に出す説明文（文字列）です。

> **用語:** オプショナルプロパティ（ `?:` ）とは、「あってもなくてもよいプロパティ」のこと。`size?` のように `?` を付けると、呼び出し側で省略できる。省略時の値は次の解説3で設定する。

---

**解説3: デフォルト値付きの分割代入**

```
export default function LoadingSpinner({
  size = "md",
  message = "読み込み中...",
}: LoadingSpinnerProps) {
```

- 引数の分割代入で `size` と `message` を取り出すと同時に、`= "md"` や `= "読み込み中..."` で**デフォルト値**を設定しています。
- 呼び出し側が `size` を省略したら自動的に `"md"`（中サイズ）が、`message` を省略したら `"読み込み中..."` が使われます。
- 例えば `<LoadingSpinner />` とだけ書けば「中サイズ・"読み込み中..."」のスピナーになります。

> **用語:** デフォルト引数とは、引数が渡されなかったときに使う「初期値」のこと。`size = "md"` のように `=` で書くと、省略時にその値が自動的に入る。

---

**解説4: サイズの辞書とスピナー本体**

```
const sizeClasses = {
  sm: "w-6 h-6 border-2",
  md: "w-10 h-10 border-3",
  lg: "w-16 h-16 border-4",
};

return (
  <div className="flex flex-col items-center justify-center py-16">
    <div
      className={`
        ${sizeClasses[size]}
        border-gray-300
        border-t-blue-600
        rounded-full
        animate-spin
      `}
      role="status"
      aria-label="読み込み中"
    />
```

- `sizeClasses` は「サイズ名 → Tailwind のクラス文字列」という**辞書（オブジェクト）**です。`StatusBadge` と同じ「辞書パターン」で、`if` 文を使わずにサイズを切り替えています。
- `sizeClasses[size]` で、選ばれたサイズに対応するクラス（幅・高さ・枠線の太さ）を取り出し、テンプレートリテラルで埋め込んでいます。
- スピナーの正体は「**円のうち上辺だけ色を変えた円**」です。`rounded-full` で円形にし、`border-t-blue-600`（上辺だけ青）と `animate-spin`（回転アニメ）を組み合わせることで、くるくる回って「処理中」に見えます。
- `role="status"` と `aria-label="読み込み中"` で、画面読み上げソフトにも「読み込み中である」ことを伝えています。

> **用語:** `animate-spin`（Tailwind）とは、要素を「ぐるぐる回転させ続ける」アニメーションを適用するクラス。`border-t-...`（上辺だけ色付き）と組み合わせると、進捗が回っているように見える。

---

解説5: メッセージの条件付き表示

```
      {message && (
        <p className="mt-4 text-sm text-gray-500">
          {message}
        </p>
      )}
    </div>
  );
}
```

- `{message && (...)}` は短絡評価で、「**`message` に中身があるときだけ**スピナーの下に説明文を表示する」という意味です。
- `message` が空文字や `undefined` のときは、`&&` の右側が評価されず、何も表示されません。
- `mt-4`（上の余白）でスピナーから少し離し、`text-sm text-gray-500` で控えめな小さいグレー文字にしています。

> **用語:** 短絡評価（`A && B`）とは、「`A` が真のときだけ `B` を実行・表示する」JavaScript の仕組み。JSX では `false`・`null`・`undefined` は何も描画されないため、「条件 && JSX」が「条件付き表示」として使える。

---

## loading.tsx

ファイル: `app/loading.tsx`

> **▼ このコードがやること（先に日本語で）: ページの読み込み中に自動で表示される「待ち画面」**を作ります。Next.js には「`loading.tsx` という名前のファイルを置くと、同じ場所の `page.tsx` がデータを取得している間、自動でこの画面を出してくれる」という決まりがあり、それを使います。中身は、ヘッダーの形だけ先に出すグレーの矩形（スケルトンUI）と、さきほど作った `LoadingSpinner` です。初心者は「**ファイル名を `loading.tsx` にするだけで、読み込み中に勝手に出る画面になる**」という1点を押さえれば十分です（詳細はコード内コメントにあります）。

```tsx
// app/loading.tsx  ← Next.js のルールで「loading.tsx」というファイル名にすると、
//                    同じ階層の page.tsx の読み込み中に自動でこの画面が表示される。

import LoadingSpinner from "@/components/LoadingSpinner";

/**
 * ルートレイアウトのローディング UI
 *
 * Next.js App Router は、ページコンポーネント（page.tsx）の
 * データ取得中にこの loading.tsx を自動的に表示する。
 *
 * 仕組み:
 * 1. ユーザーがページに遷移する
 * 2. page.tsx 内の async 処理（Supabase からのデータ取得等）が実行される
 * 3. その間、この loading.tsx が表示される
 * 4. データ取得が完了すると、page.tsx の内容に自動的に切り替わる
 *
 * これは React の Suspense（サスペンス：読み込み中のフォールバック表示機能）を内部的に利用している。
 */
// async は不要：データ取得しない静的な画面
export default function Loading() {
  return (
    <div className="min-h-screen bg-gray-50">
      {/* ヘッダーのスケルトン（ローディング中もヘッダーのような見た目を維持）。
          スケルトン UI（skeleton UI）とは「コンテンツの形だけ先に出す」UI手法。
          完成形と同じ位置にグレーの矩形を置くことで、レイアウトの「ガタつき」を防げる。 */}
      <header className="bg-white shadow-sm border-b border-gray-200">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center justify-between">
            <div>
              {/* h-8 = 高さ32px、w-48 = 幅192px。タイトル位置にグレーの矩形を置く。
                  animate-pulse = ゆっくり点滅するアニメーション（読み込み感を演出）。 */}
              <div className="h-8 w-48 bg-gray-200 rounded animate-pulse" />
              {/* mt-2 = 上方向のマージン8px。説明文の位置に小さい矩形。 */}
              <div className="mt-2 h-4 w-64 bg-gray-100 rounded animate-pulse" />
            </div>
            {/* ボタン位置の矩形。実際のボタンとほぼ同じ大きさ。 */}
            <div className="h-10 w-28 bg-gray-200 rounded-lg animate-pulse" />
          </div>
        </div>
      </header>

      {/* メインコンテンツのローディング表示。
          LoadingSpinner に size="lg"（大きいサイズ）と message を渡す。 */}
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <LoadingSpinner size="lg" message="書籍データを読み込んでいます..." />
      </main>
    </div>
```

```
  );
}
```

**表示の説明:**

ローディング中は、ヘッダー部分がスケルトン UI（灰色の長方形がパルスアニメーションする）として表示され、メインコンテンツ領域の中央に大きなスピナー（回転する円）と「書籍データを読み込んでいます...」というメッセージが表示されます。ユーザーは「何かが読み込まれている」ことをすぐに理解できます。

## ▼ コードを1つずつ分解して解説

`loading.tsx` は「ファイル名で機能が決まる」Next.js の規約ファイルです。塊ごとに見ていきましょう。

---

#### 解説1: import と Loading 関数

```tsx
import LoadingSpinner from "@/components/LoadingSpinner";

export default function Loading() {
```

- ファイル名を `loading.tsx` にすると、Next.js が「**同じ階層の `page.tsx` がデータ取得している間、自動でこの画面を出す**」ようになります。コードで呼び出さなくても勝手に表示されるのが特徴です。
- さきほど作った `LoadingSpinner` を import して使います。
- 関数に `async` が付いていない点に注目してください。この画面は**自分ではデータを取得しない静的な表示**なので、`await` は不要です。

> **用語:** ファイルベースルーティングの特殊ファイルとは、`page.tsx`・`loading.tsx`・`error.tsx` のように「決まった名前を置くと決まった役割を持つ」Next.js のファイルのこと。`loading.tsx` は読み込み中の表示を自動で担う。

---

#### 解説2: ヘッダーのスケルトン UI

```tsx
      <header className="bg-white shadow-sm border-b border-gray-200">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center justify-between">
            <div>
              <div className="h-8 w-48 bg-gray-200 rounded animate-pulse" />
              <div className="mt-2 h-4 w-64 bg-gray-100 rounded animate-pulse" />
            </div>
            <div className="h-10 w-28 bg-gray-200 rounded-lg animate-pulse" />
          </div>
        </div>
      </header>
```

- ここは「**コンテンツの形だけ先に出す**」スケルトン UI です。本物のタイトルやボタンの代わりに、同じ位置・同じ大きさのグレーの矩形（`bg-gray-200` など）を置いています。
- `h-8 w-48`（高さ32px・幅192px）はタイトルの位置、`h-10 w-28` はボタンの位置に対応します。完成形とほぼ同じ大きさにすることで、読み込み完了時に**レイアウトがガタつかない**ようにしています。
- `animate-pulse` は「ゆっくり点滅する」アニメーションで、「いま読み込み中ですよ」という雰囲気を出します。

> **用語:** スケルトン UI（skeleton screen）とは、本来のコンテンツが表示されるまでの間、その「骨組み（灰色の枠）」だけを先に見せるUI手法。空白やガタつきを減らし、待ち時間の体感を和らげる。

---

**解説3: メイン領域にスピナーを置く**

```
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <LoadingSpinner size="lg" message="書籍データを読み込んでいます..." />
      </main>
```

- メインコンテンツ領域には、解説2のスケルトンとは別に、さきほど作った `LoadingSpinner` を置いています。
- `size="lg"`（大サイズ）と `message="書籍データを読み込んでいます..."` を Props として渡し、大きな回転スピナーと説明文を表示します。
- スケルトン（ヘッダーの形）＋スピナー（中央の回転）の組み合わせで、「ヘッダーはこういう見た目になる予定」「中身は今読み込み中」という2つの情報を同時に伝えています。

> **用語:** Props を渡してコンポーネントの見た目を変える、というのは React の基本。同じ `LoadingSpinner` でも `size` や `message` を変えるだけで、画面ごとに最適な表示にできる。

### ▼ loading.tsx は「いつ」表示されるのか

`loading.tsx` は、自分で呼び出すコードを1行も書いていないのに表示されます。その「タイミング」を時間の流れで描くと次のとおりです。レストランで料理を待つ間に「ただいま調理中です」の札が出て、料理が来たら札が片付けられる、というイメージです。




図7-8: loading.tsx が表示されるタイミング（React の Suspense の仕組みで自動的に差し替わる）

- ① ユーザーがリンクをクリック（または URL を開く）すると、② Next.js は**すぐに** `loading.tsx` の画面を届けます。
- その裏で `page.tsx` は Supabase の返事を `await` で待っています。③ 返事が来て一覧の HTML ができあがった瞬間に、待ち画面が本物の一覧に**自動で差し替わります**。
- 自分で「表示する／消す」を切り替えるコードを書かなくてよいのは、Next.js がファイル名（`loading.tsx`）を見て、React の **Suspense**（サスペンス：中身の準備ができるまで代わりの表示を出す仕組み）を自動で組み立ててくれるからです。

---

## 3. エラーハンドリング

**ファイル:** `app/error.tsx`

ページのレンダリング中にエラーが発生した場合に表示される画面です。Next.js App Router の Error Boundary 機能を利用しています。

> **▼ このコードがやること（先に日本語で）：ページの処理中にエラーが起きたとき、自動で表示される「エラー画面」**を作ります。Next.js では「`error.tsx` という名前のファイルを置くと、そのページで想定外のエラーが起きたときに自動で出してくれる」決まりがあり、それを使います。画面には警告アイコンとお詫びメッセージ、そして**もう一度試すボタン（`reset` 関数を呼ぶ）**を置きます。注意点は、この `error.tsx` は必ず先頭に `"use client"` を書く必要があること（Next.js の仕様）です。初心者は「**エラー時の専用画面で、再試行ボタンを出すもの**」と押さえれば十分です（詳細はコード内コメントにあります）。

```tsx
// app/error.tsx  ← Next.js の規約で、ページレンダリング中に未処理エラーが起きた時に自動表示される。
// 必須：Next.js の仕様で error.tsx は Client Component でなければならない
"use client";

/**
 * エラーページ
 *
 * 重要: error.tsx は必ず "use client" でなければならない。
 * これは Next.js の仕様で、Error Boundary（エラー境界：内部のエラーを捕まえる仕組み）は
 * Client Component である必要がある。
 *
 * このコンポーネントは以下の場合に自動的に表示される:
 * - Server Component でエラーが throw された場合
 * - Client Component で未処理のエラーが発生した場合
 * - Supabase からのデータ取得が失敗した場合（throw した場合）
 */

type ErrorPageProps = {
  /** 発生したエラーオブジェクト。Error 型に digest（Next.js が付与するエラーID）を追加した型。
*/
  // 「&」は交差型：両方の性質を併せ持つ
  error: Error & { digest?: string };
  /** エラーからの復帰を試みる関数（ページの再レンダリングを試行する）。
      「() => void」は「引数なし、戻り値なし」の関数型。 */
  reset: () => void;
};

export default function ErrorPage({ error, reset }: ErrorPageProps) {
  return (
    // 画面全体を埋め、その中央に1枚のカードを配置するレイアウト。
    // flex items-center justify-center で「子要素を縦横中央寄せ」を実現。
    <div className="min-h-screen bg-gray-50 flex items-center justify-center px-4">
      {/* max-w-md = 最大幅 28rem（約448px）、w-full = 親いっぱい。
          結果として「最大448pxまで広がり、それ以上は広がらない」カードになる。 */}
      <div className="max-w-md w-full bg-white rounded-lg shadow-lg p-8 text-center">
        {/* エラーアイコン。赤い円の中に三角形の警告マーク。
            mx-auto = 左右マージン自動（中央寄せ）。 */}
        <div className="mx-auto w-16 h-16 bg-red-100 rounded-full flex items-center
justify-center mb-6">
          <svg
            className="w-8 h-8 text-red-600"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            {/* この path 文字列は警告三角形（中に「!」のような縦線と点）を描画している。 */}
            <path
              strokeLinecap="round"
```

```
              strokeLinejoin="round"
              strokeWidth={2}
              d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-2.5L13.732
4c-.77-.833-1.964-.833-2.732 0L4.082 16.5c-.77.833.192 2.5 1.732 2.5z"
            />
          </svg>
        </div>

        {/* エラーメッセージ */}
        <h2 className="text-xl font-bold text-gray-900 mb-2">
          エラーが発生しました
        </h2>
        <p className="text-gray-600 mb-6">
          申し訳ありません。データの取得中にエラーが発生しました。
          しばらく時間をおいてから再度お試しください。
        </p>

        {/* エラー詳細（開発時のデバッグ用）。
            process.env.NODE_ENV は Node.js が提供する環境変数。
              "development" → 開発中（npm run dev）
              "production"  → 本番ビルド（npm run build → npm start）
            開発時だけ詳細表示することで、本番ユーザーに技術的な情報を見せないようにする。 */}
        {process.env.NODE_ENV === "development" && (
          // <details> / <summary> は HTML 標準の「折りたたみ」要素。
          // クリックで <summary> 以外の中身が開く。JS 不要。
          <details className="mb-6 text-left">
            <summary className="text-sm text-gray-500 cursor-pointer hover:text-gray-
700">
              エラー詳細を表示
            </summary>
            {/* <pre> はそのままの体裁（改行・空白）を保持して表示する HTML 要素。
                overflow-auto max-h-40 で「内容が多ければスクロール、最大160pxまで」。 */}
            <pre className="mt-2 p-3 bg-gray-100 rounded text-xs text-red-600 overflow-
auto max-h-40">
              {/* Error オブジェクトの message プロパティ（人間可読なエラー説明文）。 */}
              {error.message}
            </pre>
          </details>
        )}

        {/* アクションボタン群。
            flex-col sm:flex-row で「スマホでは縦並び、smサイズ以上では横並び」を切り替え。 */}
        <div className="flex flex-col sm:flex-row gap-3 justify-center">
          {/* reset 関数を onClick に渡す。Next.js が用意するこの関数を呼ぶと、
              ページの再レンダリングが試みられ、エラーから復帰できる可能性がある。 */}
          <button
            onClick={reset}
            className="
              px-6 py-2.5
              bg-blue-600
```

```
              text-white
              rounded-lg
              font-medium
              hover:bg-blue-700
              transition-colors
              duration-200
            "
          >
            もう一度試す
          </button>
          {/* トップページへ戻るリンク。エラー時は <Link> ではなく普通の <a> を使うことで
              「アプリ全体を完全に初期化」できる（クライアントサイドキャッシュも含めてリセット）。
*/}
          <a
            href="/"
            className="
              px-6 py-2.5
              bg-gray-100
              text-gray-700
              rounded-lg
              font-medium
              hover:bg-gray-200
              transition-colors
              duration-200
            "
          >
            トップに戻る
          </a>
        </div>
      </div>
    </div>
  );
}
```

**表示の説明:**

画面中央に白いカードが表示されます。赤い三角形の警告アイコン、「エラーが発生しました」というメッセージ、そして「もう一度試す」ボタンと「トップに戻る」ボタンが配置されます。開発環境（`NODE_ENV === "development"`）では、エラーの詳細メッセージも折りたたみで確認できます。

### ▼ コードを1つずつ分解して解説

`error.tsx` も Next.js の規約ファイルで、`"use client"` 必須・特別な Props（`error`・`reset`）を受け取る、という特徴があります。塊ごとに見ていきましょう。

---

**解説1: "use client" が必須**

```
"use client";
```

- `error.tsx` は **必ず先頭に `"use client"` を書く**決まりです（Next.js の仕様）。
- 理由は、エラーを捕まえる仕組み（Error Boundary）が Client Component でしか動かないためです。書き忘れるとビルド時にエラーになります。
- `loading.tsx` には不要でしたが、`error.tsx` には必須、という違いを押さえておきましょう。

> **用語:** Error Boundary（エラー境界）とは、配下のコンポーネントで起きたエラーを捕まえ、アプリ全体がクラッシュする代わりに代替UI（エラー画面）を出す仕組み。Next.js では `error.tsx` がこれを担う。

---

**解説2: 受け取る Props の型（error と reset）**

```
type ErrorPageProps = {
  error: Error & { digest?: string };
  reset: () => void;
};

export default function ErrorPage({ error, reset }: ErrorPageProps) {
```

- `error.tsx` は、Next.js から自動的に2つの Props を受け取ります。
- `error: Error & { digest?: string }` … 発生したエラー情報です。`&`（アンド）は「**交差型**」で、「`Error` 型の性質と `{ digest? }` の性質を**両方**併せ持つ」という意味です。`digest` は Next.js が付けるエラーIDです。
- `reset: () => void` …「**エラーからの復帰を試みる関数**」です。`() => void` は「引数なし・戻り値なしの関数」という型です。このボタンを押すとページの再描画が試みられます。

> **用語:** 交差型（intersection type・`A & B`）とは、「`A` と `B` の両方の性質を同時に持つ型」のこと。ユニオン型（`A | B` ＝どちらか）の反対で、`&` は「両方を兼ね備える」を表す。

---

**解説3: 開発時だけエラー詳細を表示する**

```
        {process.env.NODE_ENV === "development" && (
          <details className="mb-6 text-left">
            <summary className="text-sm text-gray-500 cursor-pointer hover:text-gray-
700">
              エラー詳細を表示
            </summary>
            <pre className="mt-2 p-3 bg-gray-100 rounded text-xs text-red-600 overflow-
auto max-h-40">
              {error.message}
            </pre>
          </details>
        )}
```

- `process.env.NODE_ENV` は「**今が開発中か本番か**」を表す環境変数です。`"development"`（開発中）と一致するときだけ、`&&` の右側が表示されます。
- つまり、エラーの技術的な詳細（`error.message`）は**開発者が見るとき**（`npm run dev`）**だけ**表示され、本番のユーザーには見せません。技術情報を一般ユーザーに晒さない配慮です。
- `<details>` / `<summary>` は HTML 標準の「折りたたみ」要素で、JavaScript なしでクリック開閉できます。`<pre>` は改行や空白をそのまま保持して表示する要素です。

> **用語:** 環境変数 `NODE_ENV` とは、アプリが「開発モード」か「本番モード」かを示す値。開発時だけログや詳細を出し、本番では隠す、といった切り替えに使う。

---

**解説4: 再試行ボタンで reset を呼ぶ**

```
          <button
            onClick={reset}
            className="..."
          >
            もう一度試す
          </button>
          <a href="/" className="...">
            トップに戻る
          </a>
```

- 「もう一度試す」ボタンの `onClick={reset}` で、Props で受け取った `reset` 関数を呼びます。これによりページの再レンダリングが試みられ、一時的なエラー（通信の不調など）なら復帰できる可能性があります。
- `onClick={reset}` のように**関数名だけ**を渡している点に注意してください。`reset()` と `()` を付けると、レンダリング時に即実行されてしまいバグの原因になります。

- 「トップに戻る」は `<Link>` ではなく通常の `<a href="/">` を使っています。`<a>` だとページ全体が完全に再読み込みされ、クライアント側のキャッシュも含めてアプリを初期化できるため、エラー時の確実なリセット手段になります。

> **用語:** `onClick={関数}` と `onClick={関数()}` の違い。前者は「クリック時にこの関数を呼んで」という意味で正しい。後者は「今この場で関数を実行した戻り値を渡す」になってしまい、意図しない即時実行が起きる。

> **注意（Next.js 16.3 以降を使っている場合）**: Next.js 16.3 から、`error.tsx` には `reset` のほかに **`retry`** という関数も渡されるようになり、公式ドキュメントでは `retry` の利用が推奨されています。違いは次のとおりです。
>
> - `reset()` … エラー状態を消して描き直すだけ。**サーバーのデータは取り直さない**ので、Supabase の取得失敗のようなエラーだと、同じエラー画面がもう一度出ることがあります。
> - `retry()` … サーバーのデータを**取り直してから**描き直す。一時的な通信の不調なら、これで復帰できます。
>
> 使っている Next.js の版は `package.json` の `"next"` の行で確かめられます。16.3 以降なら `retry` を使うと「もう一度試す」がより確実に働きます。次は `retry` を使った最小限の `error.tsx` です。

```tsx
// app/error.tsx（Next.js 16.3 以降・retry を使う最小限の例）
"use client";

type ErrorPageProps = {
  error: Error & { digest?: string };
  // データを取り直してから描き直す関数（Next.js 16.3 以降）
  retry: () => void;
};

export default function ErrorPage({ retry }: ErrorPageProps) {
  return (
    <div className="min-h-screen flex flex-col items-center justify-center gap-4">
      <h2 className="text-xl font-bold text-gray-900">エラーが発生しました</h2>
      {/* reset ではなく retry を呼ぶ。サーバーのデータから取り直してくれる */}
      <button
        onClick={retry}
        className="px-6 py-2.5 bg-blue-600 text-white rounded-lg font-medium hover:bg-blue-700"
      >
        もう一度試す
      </button>
    </div>
  );
}
```

この節の `ErrorPage` に組み込む場合は、Props の型の `reset: () => void;` を `retry: () => void;` に、引数の `{ error, reset }` を `{ error, retry }` に、ボタンの `onClick={reset}` を `onClick={retry}`

に変えるだけです。Next.js 15 など 16.3 より前の版では `retry` は渡されないので、この節のコード（`reset`）のままにしてください。

**▼ エラーには「2つの道」がある**

ここまでで、エラーへの備えが2種類出てきました。1つは 1e のつまずきポイントで見た「`if (error)` で分岐して、画面の中で知らせる」方法、もう1つはこの節の `error.tsx` です。どちらを使うかは、**そのエラーが予想できるものかどうか**で決めます。

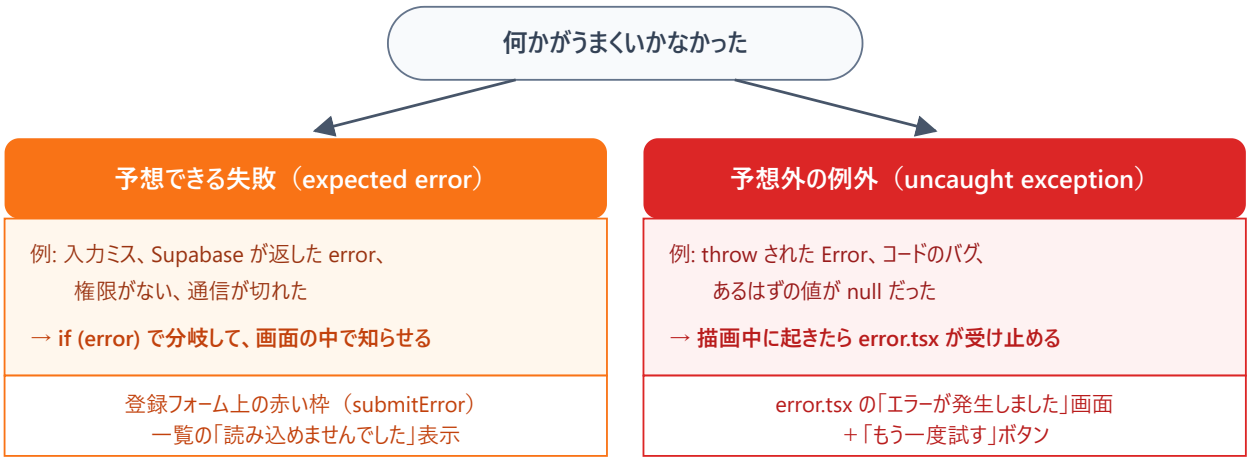



図7-9: エラーの2つの道。予想できる失敗は自分で画面に出し、予想外の例外は error.tsx に任せる

- 図の下の※が大事です。`error.tsx` が受け止めるのは「**画面を描いている最中**」に起きた例外だけです。登録ボタンを押したあとの処理（4b の `handleSubmit`）の中で起きたエラーは `error.tsx` には届きません。だから 4b では `try / catch` と `submitError` という state を使い、自分でエラー枠を表示しているのです。

---

## 4. 登録機能の実装

ここからは書籍の新規登録機能を作ります。ユーザーがフォームに情報を入力し、Supabase のデータベースに新しいレコードを INSERT する流れです。

### 4-0. 登録機能を作る前に知っておきたい用語

実装に入る前に、いくつかの言葉の意味を押さえておきましょう。

| 用語 | 読み・英語 | 1行説明 |
| --- | --- | --- |
| **フォーム** | form | ユーザーが入力する箱（input、textarea、selectなど）を集めた HTML 要素 `<form>` |
| **FormData** | フォームデータ | フォームの入力内容をまとめて持つブラウザ標準のオブジェクト |
| **Server Action** | サーバーアクション | Next.js の機能。Client側から呼べるけど実行はサーバーで行われる関数 |
| **revalidatePath** | リバリデートパス | 指定したパスのキャッシュを破棄して次回アクセス時に作り直す関数 |
| **redirect** | リダイレクト | 別のURLにブラウザを移動させる関数 |
| **INSERT** | インサート | SQL で「新しい行を追加する」操作 |

> **本書での登録機能のアプローチ:** Next.js には「Server Action」というサーバー側で実行される関数を使う登録方式と、「ブラウザ側の Supabase クライアントから直接 INSERT」する方式の2通りがあります。**本書ではこの章では後者（クライアント方式）** を採用します。理由は、Reactの useState や onSubmit といった基本機能だけで完結し、初学者にとって流れが追いやすいためです。
>
> Server Action 方式は仕組みがやや高度（ `'use server'` ディレクティブ、`<form action={fn}>` 、 `FormData` の扱い、`revalidatePath` 、`redirect` などの理解が必要）なので、参考として下の「Server Action 方式の概要」コラムにまとめておきます。実装は読み飛ばしても構いません。

### コラム: Server Action 方式の概要（参考）

参考までに、Server Action を使うとどんなコードになるかだけ簡単に紹介します。**この章では実装しません**（クライアント方式で進めます）。

```ts
// app/actions.ts - Server Action を定義するファイル例（参考）
// ファイル先頭でこのディレクティブを書くと、中の関数はすべてサーバー実行になる
"use server";

// サーバー用 Supabase クライアント
import { createClient } from "@/lib/supabase/server";
// 指定パスのキャッシュを破棄する関数
import { revalidatePath } from "next/cache";
// 別のページに遷移させる関数
import { redirect } from "next/navigation";

// フォーム送信時に呼ばれる Server Action。引数 formData はブラウザ標準の FormData オブジェクト。
// FormData とは <form> 内の入力値（name 属性をキーに）をまとめた箱。
export async function createBook(formData: FormData) {
  // formData.get("title") の戻り値の型は FormDataEntryValue | null。
  // FormDataEntryValue は「string か File のどちらか」。
  // 通常のテキスト入力なら string になるが、TypeScript 上はそうとは限らないので、
  // String(...) で文字列にキャストするのが安全（null なら "null" になるので注意）。
  const title = String(formData.get("title") ?? "");
  const author = String(formData.get("author") ?? "");

  const supabase = await createClient();
  await supabase.from("books").insert({ title, author });

  // 関連パスのキャッシュを破棄 → 次の表示で最新データが取られる
  revalidatePath("/");

  // 登録完了後にトップへ遷移
  redirect("/");
}
```

```tsx
// app/books/new/page.tsx - Server Action を使う側（参考）
import { createBook } from "@/app/actions";

export default function NewBookPage() {
  return (
    // <form action={createBook}> と書くと、送信時に Next.js が自動で
    // FormData を組み立てて createBook(formData) を呼んでくれる。
    <form action={createBook}>
      {/* name="title" が formData.get("title") に対応 */}
      <input name="title" />
      <input name="author" />
      <button type="submit">登録</button>
    </form>
  );
}
```

### ▼ 2つの方式を並べて比較する

どちらの方式でも「入力 → 保存 → 一覧へ戻って最新を表示」というゴールは同じです。違うのは、**保存の処理がブラウザで動くか、サーバーで動くか**と、**一覧を最新にする命令の名前**です。

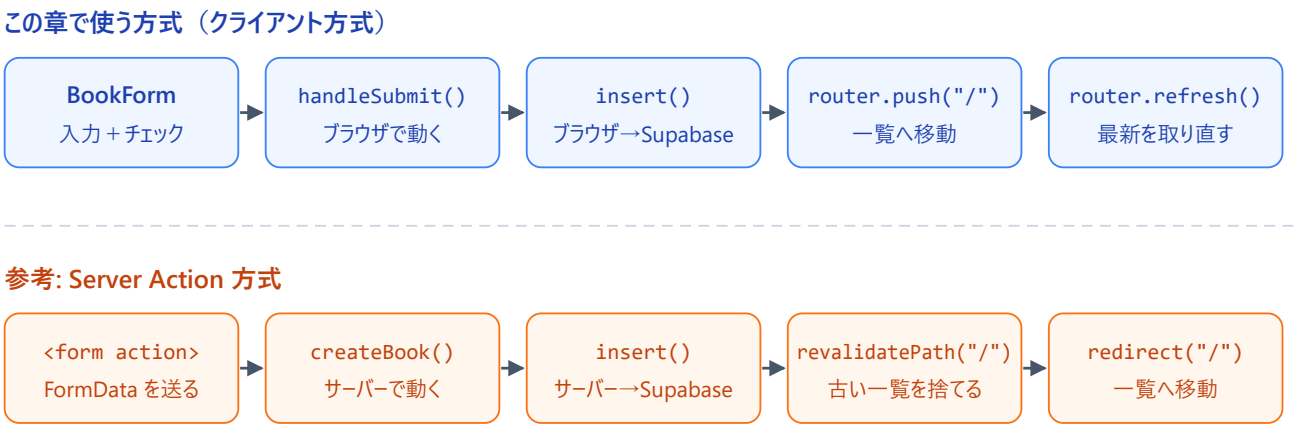



図7-10: 登録処理の2つの方式（上: この章のクライアント方式 / 下: 参考の Server Action 方式）

- Server Action 方式では `revalidatePath("/")` を `redirect("/")` **より前**に書きます。`redirect` は呼んだ瞬間にその関数の処理を打ち切って移動してしまうため、後ろに書いた行は実行されないからです。
- `revalidatePath` は Server Action やサーバー側の処理の中でしか呼べません。Client Component（この章の登録ページ）から呼ぶことはできないので、この章では代わりに `router.refresh()` を使っています。

**Server Action のメリット（参考）:**

- ブラウザ側 JS の量が減る（送信ロジックがサーバーに置かれる）。
- Supabase の秘密鍵などをサーバー側だけで使える。
- フォーム送信時に JavaScript が無効でも動作する（プログレッシブエンハンスメント）。

**Server Action のセキュリティ:**

- Server Action はサーバー上で実行されるため、フォームに「価格を勝手に書き換える」「他人の ID を送る」といった改ざんが届いても、Server Action の中で再検証すれば防げます。**ただし「クライアントから送られた値は信用しない」前提でバリデーションを書くことが必須**です。

**Server Action と組み合わせるフック（参考）:**

- `useActionState`（React 19 以降。`react` から import する。React 18 時代は `react-dom` の `useFormState` という名前でした）: フォーム送信の結果（成功/エラー）と、送信中かどうか（pending）を state として保持する。
- `useFormStatus`（`react-dom` から import）: フォーム送信中かどうか（pending状態）を、フォームの内側に置いた部品から取得する。

これらは第8章以降のオプションとして紹介します。

---

## 4a. BookForm コンポーネント

**ファイル:** `components/BookForm.tsx`

フォーム全体を管理するコンポーネントです。新規登録と編集の両方で使えるように設計します（この章では新規登録のみ使用、編集は次章で使用）。

> **▼ このコードがやること（先に日本語で）**：書籍情報を入力する**フォーム本体**を作ります。カギは3つ——①入力中の値を `useState` という「変化する箱」に1つのオブジェクトとしてまとめて持つこと、②入力が変わるたびにその箱を更新する（**制御コンポーネント**＝画面の値とプログラムの値を常に一致させる方式）こと、③送信前に「タイトルは必須」などの**入力チェック（バリデーション）**を行い、問題があればエラーを表示して送信を止めること。実際のデータベース保存は、このフォームではなく**親から渡された `onSubmit` 関数に任せる**設計です（登録と編集で同じフォームを使い回すため）。初心者は「**入力を集めてチェックし、OKなら親に渡す係**」と押さえれば十分です（詳細はコード内コメントにあります）。

```tsx
// components/BookForm.tsx
// useState などブラウザ専用機能を使うので Client Component 化
"use client";

/**
 * BookForm - 書籍の登録・編集フォームコンポーネント
 *
 * "use client" が必要な理由:
 * - useState でフォームの入力状態を管理する
 * - onChange イベントハンドラを使う
 * - フォーム送信時の処理（onSubmit）をハンドリングする
 * これらはすべてブラウザ側で動作する機能のため、Client Component にする。
 *
 * このコンポーネントは「登録」と「編集」の両方で使い回す。
 * - 新規登録時: initialData を渡さない → 空のフォーム
 * - 編集時: initialData に既存データを渡す → 値が入ったフォーム
 */

// useState は「状態」を管理するための React フック。
// type FormEvent は「フォーム送信イベントオブジェクト」の型。"type" を付けると型だけインポート。
import { useState, type FormEvent } from "react";
// 同フォルダの StatusBadge から BookStatus 型だけインポート（値は使わない）
import { type BookStatus } from "./StatusBadge";

// フォームのデータ型（データベースの books テーブルに対応するが、
// id, created_at, updated_at はフォームでは扱わない）。
// export しているので、親ページがこの型を使って handleSubmit の引数を型付けできる。
export type BookFormData = {
  // タイトル（必須）
  title: string;
  // 著者（必須）
  author: string;
  // 出版社（任意）。文字列のまま扱い、null は親側で変換する
  publisher: string;
  // 出版日（任意）。"YYYY-MM-DD" 形式の文字列
  published_date: string;
  // 評価（任意）。1〜5の数値 または null
  rating: number | null;
  // ステータス（必須）。"reading" | "completed" | "want_to_read"
  status: BookStatus;
  // メモ（任意）
  memo: string;
};

// Props の型定義（親から受け取る値の形）
type BookFormProps = {
  /** 編集時に渡す初期データ（新規登録時は undefined）。?: で省略可能。 */
  initialData?: BookFormData;
  /** フォーム送信時に呼ばれるコールバック関数。
```

```
   *  - 引数: フォームのデータ
   *  - 戻り値: Promise<void>（非同期処理を行う想定。中で await できる）
   */
  onSubmit: (data: BookFormData) => Promise<void>;
  /** 送信ボタンのテキスト（"登録する" / "更新する" など）。省略時はデフォルト値を使う。 */
  submitLabel?: string;
  /** 送信中かどうか（ボタンの無効化・ローディング表示に使う） */
  isSubmitting?: boolean;
};

// フォームの初期値（新規登録時に使う）。すべての項目を「空」または「初期選択」にしておく。
const defaultFormData: BookFormData = {
  title: "",
  author: "",
  publisher: "",
  published_date: "",
  // 「未選択」を表す null
  rating: null,
  // 新規登録時は「読みたい」を初期値に
  status: "want_to_read",
  memo: "",
};

export default function BookForm({
  initialData,
  onSubmit,
  // デフォルト値：呼び出し側が省略したらこれを使う
  submitLabel = "登録する",
  isSubmitting = false,
}: BookFormProps) {
  // ----- State -----
  // フォームの入力値を管理する state。
  // useState<型>(初期値) で「型と初期値」を指定する。
  // initialData が渡されていればそれを使い、なければデフォルト値を使う（?? は nullish
coalescing）。
  const [formData, setFormData] = useState<BookFormData>(
    initialData ?? defaultFormData
  );

  // バリデーションエラーを管理する state。
  // Partial<T>          = T の全プロパティを「省略可能」にした型
  // Record<K, V>        = キーが K、値が V のオブジェクト型
  // keyof BookFormData = BookFormData のプロパティ名のユニオン型（"title" | "author" |
...）
  // つまり {title?: string; author?: string; ...} という型。
  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>
({});

  // ----- バリデーション -----
  /**
```

```
 * フォーム全体のバリデーションを行う
 * @returns バリデーションを通過したら true、エラーがあれば false
 */
// 戻り値型を明示
const validate = (): boolean => {
  // 新しいエラー集を作る
  const newErrors: Partial<Record<keyof BookFormData, string>> = {};

  // タイトル: 必須、100文字以内
  // .trim() は前後の空白文字を削った文字列を返す。
  // 空文字列 "" は falsy なので、! を付けると「空または空白だけ」を検出できる。
  if (!formData.title.trim()) {
    newErrors.title = "タイトルは必須です";
  } else if (formData.title.trim().length > 100) {
    newErrors.title = "タイトルは100文字以内で入力してください";
  }

  // 著者: 必須、50文字以内
  if (!formData.author.trim()) {
    newErrors.author = "著者は必須です";
  } else if (formData.author.trim().length > 50) {
    newErrors.author = "著者は50文字以内で入力してください";
  }

  // 出版社: 任意、50文字以内（空ならエラーにしない）
  if (formData.publisher.trim().length > 50) {
    newErrors.publisher = "出版社は50文字以内で入力してください";
  }

  // メモ: 任意、1000文字以内
  if (formData.memo.trim().length > 1000) {
    newErrors.memo = "メモは1000文字以内で入力してください";
  }

  // 集めたエラーを state に反映 → 画面に表示される
  setErrors(newErrors);

  // Object.keys(obj) は obj のキー名を配列で返す。
  // エラーが1つもなければ length が 0 になり、true を返す。
  return Object.keys(newErrors).length === 0;
};

// ----- イベントハンドラ -----

/**
 * テキスト入力フィールドの変更ハンドラ
 * input, textarea, select の onChange に共通で使える
 *
 * ──────────────────────────────────────────────────
 * (a) 引数 e は「変更イベントオブジェクト」。ユーザーが文字を打つと
```

```
 *     React がこのオブジェクトをハンドラに渡してくる。
 * (b) e.target は「変更が起きた DOM 要素」。ここから name と value を取り出す。
 *     - name  : <input name="title"> のように JSX で指定した名前
 *     - value : 現在の入力値（文字列）
 * ──────────────────────────────────────────────────────────
 */
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>
) => {
  // (1) e.target から name, value を分割代入で取り出す
  //     例: <input name="title" value="..." /> なら
  //         name = "title", value = ユーザーが打った文字列
  const { name, value } = e.target;

  // (2) 現在の formData を「コピーして1つの項目だけ書き換える」更新パターン
  //     setFormData((prev) => ...) の prev は「直前の formData の値」。
  //     ...prev は「prev の中身を全部コピー」（スプレッド構文）。
  //     [name]: value は「name 変数の値をキーにして value を入れる」記法
  //      （Computed Property Names）。
  //
  //     例: name="title", value="React入門" のとき
  //       { ...prev, title: "React入門" } という新しいオブジェクトを作る。
  //     React は「formData が新しいオブジェクトに入れ替わった」と検知して
  //     再描画する。直接 prev.title = value のように書き換えてはいけない。
  setFormData((prev) => ({
    ...prev,
    [name]: value,
  }));

  // (3) ユーザーが入力をやり直し始めたら、そのフィールドの古いエラーを消す。
  //     [name]: undefined を入れることでエラー表示が消える。
  //     `as keyof BookFormData` は「name は BookFormData のキーですよ」と
  //     TS に教えるアサーション（型注釈の補助）。
  if (errors[name as keyof BookFormData]) {
    setErrors((prev) => ({
      ...prev,
      [name]: undefined,
    }));
  }
};

/**
 * 評価（rating）の変更ハンドラ
 * select の値は文字列なので、数値に変換する必要がある。
 * 「選択してください」を選んだ時は value が空文字列 "" になるので、それを null に変換する。
 */
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  // HTMLSelectElement の value は常に string。<option value="3"> なら "3"。
  const value = e.target.value;
  setFormData((prev) => ({
```

```
      ...prev,
      // 三項演算子: value === "" なら null、それ以外なら Number(value) で数値化。
      // Number("3") は 3 を返す。Number("abc") は NaN を返すので、数値以外が来る場合は注意が必要。
      // ここでは <option value="..."> の値は固定なので問題ない。
      rating: value === "" ? null : Number(value),
    }));
  };

  /**
   * フォーム送信ハンドラ
   * <form onSubmit={handleSubmit}> から呼ばれる。
   * - 引数 e はフォーム送信イベントオブジェクト。
   * - 関数全体に async が付くので、内部で await が使える。
   */
  const handleSubmit = async (e: FormEvent) => {
    // (1) e.preventDefault() を呼ばないと、ブラウザが既定の動作として
    //     ページ全体をリロードしてしまう（HTML本来の挙動）。
    //     SPA では絶対に必要な「お決まりの一行」。
    e.preventDefault();

    // (2) バリデーション関数を呼び、エラーがあれば中断
    //     validate() は内部で setErrors を呼ぶので、画面にエラー文も表示される。
    if (!validate()) {
      // 関数を終了（送信しない）
      return;
    }

    // (3) 親コンポーネントから渡された onSubmit コールバックを呼ぶ
    //     onSubmit は Promise を返す async 関数なので await で待つ。
    //     ここで実際のDB書き込みやページ遷移が行われる（実装は親側にある）。
    await onSubmit(formData);
  };

  // ----- 表示 -----
  return (
    // <form> は HTML 標準のフォーム要素。
    // onSubmit={handleSubmit} で送信時に handleSubmit を呼ぶ。
    // space-y-6 は Tailwind の「直下の子要素の間隔を縦方向に 6（24px）空ける」ユーティリティ。
    <form onSubmit={handleSubmit} className="space-y-6">
      {/* ===== タイトル ===== */}
      <div>
        {/* <label htmlFor="title"> は「title という id を持つ入力要素のラベル」という関連付け。
            ラベルをクリックすると、関連付けられた入力にフォーカスが移る。
            ※React では「for」が JS の予約語と被るので、HTML の for 属性は「htmlFor」と書く。
        */}
        <label
          htmlFor="title"
          className="block text-sm font-medium text-gray-700 mb-1"
```

```
        >
          {/* 赤い「*」で必須を表す */}
          タイトル <span className="text-red-500">*</span>
        </label>
        <input
          // 1行のテキスト入力
          type="text"
          // label の htmlFor と一致させる
          id="title"
          // FormData で使うキー名（Server Action方式の場合に重要）
          name="title"
          // 制御コンポーネント：state の値を入力欄の値として使う
          value={formData.title}
          // 入力が変わったら handleChange を呼ぶ
          onChange={handleChange}
          placeholder="例: リーダブルコード"
          // テンプレートリテラル内で「エラーがあれば赤い枠線、なければ通常の枠線」を切り替える。
          className={`
            w-full px-4 py-2.5
            border rounded-lg
            text-gray-900
            placeholder-gray-400
            /* フォーカス時に青い縁取り */
            focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
            transition-colors duration-200
            ${errors.title ? "border-red-500 bg-red-50" : "border-gray-300"}
          `}
        />
        {/* エラーがあれば、フィールドのすぐ下に赤文字でエラーメッセージを表示。
            「errors.title &&」は短絡評価：左辺が真（文字列がある）なら右辺の JSX を描画。 */}
        {errors.title && (
          <p className="mt-1 text-sm text-red-600">{errors.title}</p>
        )}
      </div>

      {/* ===== 著者 ===== */}
      <div>
        <label
          htmlFor="author"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          著者 <span className="text-red-500">*</span>
        </label>
        <input
          type="text"
          id="author"
          name="author"
          value={formData.author}
          onChange={handleChange}
```

```
          placeholder="例: Dustin Boswell"
          className={`
            w-full px-4 py-2.5
            border rounded-lg
            text-gray-900
            placeholder-gray-400
            focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
            transition-colors duration-200
            ${errors.author ? "border-red-500 bg-red-50" : "border-gray-300"}
          `}
        />
        {errors.author && (
          <p className="mt-1 text-sm text-red-600">{errors.author}</p>
        )}
      </div>

      {/* ===== 出版社 ===== */}
      <div>
        <label
          htmlFor="publisher"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          出版社
        </label>
        <input
          type="text"
          id="publisher"
          name="publisher"
          value={formData.publisher}
          onChange={handleChange}
          placeholder="例: オライリージャパン"
          className={`
            w-full px-4 py-2.5
            border rounded-lg
            text-gray-900
            placeholder-gray-400
            focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
            transition-colors duration-200
            ${errors.publisher ? "border-red-500 bg-red-50" : "border-gray-300"}
          `}
        />
        {errors.publisher && (
          <p className="mt-1 text-sm text-red-600">{errors.publisher}</p>
        )}
      </div>

      {/* ===== 出版日 ===== */}
      <div>
```

```
          <label
            htmlFor="published_date"
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            出版日
          </label>
          {/* type="date" にすると、ブラウザがカレンダー UI を出してくれる。
              値は "YYYY-MM-DD" 形式の文字列で取得できる（ISO 8601 形式の一部）。 */}
          <input
            type="date"
            id="published_date"
            name="published_date"
            value={formData.published_date}
            onChange={handleChange}
            className="
              w-full px-4 py-2.5
              border border-gray-300 rounded-lg
              text-gray-900
              focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
              transition-colors duration-200
            "
          />
        </div>

        {/* ===== 評価 と ステータス を横に並べる =====
            grid-cols-1 sm:grid-cols-2 で「スマホでは1列、smサイズ以上では2列」のグリッドに。
*/}
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-6">
          {/* 評価 */}
          <div>
            <label
              htmlFor="rating"
              className="block text-sm font-medium text-gray-700 mb-1"
            >
              評価
            </label>
            {/* <select> はドロップダウン。
                value={formData.rating ?? ""} で「null なら空文字列」を value にする。
                これは <option value=""> と一致するので「選択してください」が選ばれた状態になる。
*/}
            <select
              id="rating"
              name="rating"
              value={formData.rating ?? ""}
              // rating は数値変換が必要なので別ハンドラ
              onChange={handleRatingChange}
              className="
                w-full px-4 py-2.5
                border border-gray-300 rounded-lg
```

```
              text-gray-900
              focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
              transition-colors duration-200
              bg-white
            "
          >
            {/* <option> の value 属性が、選択時に select の value になる値。 */}
            <option value="">選択してください</option>
            <option value="1">★☆☆☆☆ (1)</option>
            <option value="2">★★☆☆☆ (2)</option>
            <option value="3">★★★☆☆ (3)</option>
            <option value="4">★★★★☆ (4)</option>
            <option value="5">★★★★★ (5)</option>
          </select>
        </div>

        {/* ステータス */}
        <div>
          <label
            htmlFor="status"
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            ステータス <span className="text-red-500">*</span>
          </label>
          <select
            id="status"
            name="status"
            value={formData.status}
            onChange={handleChange}
            className="
              w-full px-4 py-2.5
              border border-gray-300 rounded-lg
              text-gray-900
              focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
              transition-colors duration-200
              bg-white
            "
          >
            <option value="want_to_read">読みたい</option>
            <option value="reading">読書中</option>
            <option value="completed">読了</option>
          </select>
        </div>
      </div>

      {/* ===== メモ ===== */}
      <div>
        <label
```

```
          htmlFor="memo"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          メモ
        </label>
        {/* <textarea> は複数行入力。input と違って終了タグが必要。
            rows={4} で「最初の表示高さ」を4行分に設定。実際には縦方向に自由に広げられる（resize-
y）。 */}
        <textarea
          id="memo"
          name="memo"
          value={formData.memo}
          onChange={handleChange}
          rows={4}
          placeholder="この書籍に関するメモを自由に入力してください"
          className={`
            w-full px-4 py-2.5
            border rounded-lg
            text-gray-900
            placeholder-gray-400
            focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
            transition-colors duration-200
            /* 縦方向だけリサイズ可能 */
            resize-y
            ${errors.memo ? "border-red-500 bg-red-50" : "border-gray-300"}
          `}
        />
        {errors.memo && (
          <p className="mt-1 text-sm text-red-600">{errors.memo}</p>
        )}
        {/* 文字数カウンター。入力するたびに再レンダリングされて数値が更新される。 */}
        <p className="mt-1 text-xs text-gray-400">
          {formData.memo.length} / 1000 文字
        </p>
      </div>

      {/* ===== 送信ボタン ===== */}
      <div className="flex items-center gap-4 pt-4">
        <button
          // type="submit" で「これがフォームの送信ボタン」と明示
          type="submit"
          // 送信中はボタンを無効化（連打防止）
          disabled={isSubmitting}
          // 送信中は薄い青＆クリック不可マウスカーソル、通常時は濃い青＆ホバーで色変化
          className={`
            px-8 py-3
            rounded-lg
            font-medium
            text-white
```

```
            transition-colors duration-200
            ${
              isSubmitting
                ? "bg-blue-400 cursor-not-allowed"
                : "bg-blue-600 hover:bg-blue-700"
            }
          `}
        >
          {/* 送信中ならスピナー＋「送信中...」、それ以外なら submitLabel（"登録する"等）を表示
*/}
          {isSubmitting ? (
            <span className="flex items-center gap-2">
              {/* 送信中のスピナー（SVG）。animate-spin で回転アニメーション。 */}
              <svg
                className="animate-spin h-5 w-5"
                viewBox="0 0 24 24"
                fill="none"
              >
                {/* 背景の薄い円（不透明度25%）。 */}
                <circle
                  className="opacity-25"
                  // 中心 x
                  cx="12"
                  // 中心 y
                  cy="12"
                  // 半径
                  r="10"
                  stroke="currentColor"
                  strokeWidth="4"
                />
                {/* 上に重ねる「動く一部の弧」（不透明度75%）。これが回って見える。 */}
                <path
                  className="opacity-75"
                  fill="currentColor"
                  d="M4 12a8 8 0 018-8v4a4 4 0 00-4 4H4z"
                />
              </svg>
              送信中...
            </span>
          ) : (
            submitLabel
          )}
        </button>

        {/* キャンセルボタン（トップページに戻る）。<a> なのでフォーム送信はしない。 */}
        <a
          href="/"
          className="
            px-6 py-3
            rounded-lg
```

```
            font-medium
            text-gray-700
            bg-gray-100
            hover:bg-gray-200
            transition-colors duration-200
          "
        >
          キャンセル
        </a>
      </div>
    </form>
  );
}
```

**フォームはこのように表示されます:**

白い背景のページ上に、縦に並んだフォームフィールドが表示されます。各フィールドの構成は以下の通りです。




- 各フィールドのラベルの横にある `*` マーク（赤色）は必須フィールドを示します。
- 入力フィールドにフォーカスすると、青い枠線（`focus:ring-2 focus:ring-blue-500`）が表示されます。
- バリデーションエラーがある場合、そのフィールドの枠線が赤くなり（`border-red-500`）、背景が薄い赤色（`bg-red-50`）になり、フィールドの下にエラーメッセージが赤文字で表示されます。
- 「評価」と「ステータス」はデスクトップでは横に並び（`sm:grid-cols-2`）、スマホでは縦に並びます。
- メモ欄の下には入力文字数カウンター（`0 / 1000 文字`）が表示されます。
- 送信ボタンは青色で、送信中は薄い青色になりスピナーアイコンが回転します。

### ▼ 画面イメージ: 何も入力せずに「登録する」を押したとき

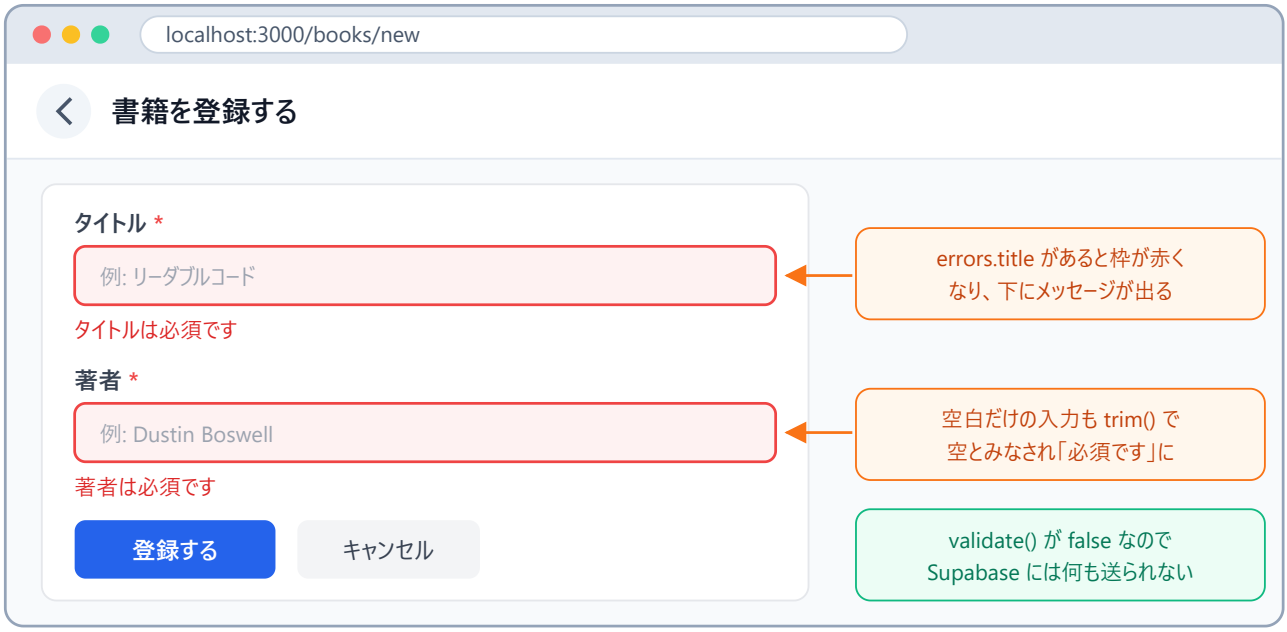



図7-11: バリデーションエラーの画面イメージ（出版社より下の欄は省略）

## ▼ コードを1つずつ分解して解説

`BookForm` はこの章で一番大きな部品です。「制御コンポーネント」「汎用ハンドラ」「バリデーション」「親への委譲」など、フォームの定番パターンが詰まっています。塊ごとにていねいに見ていきましょう。

---

**解説1: "use client" と import**

```
"use client";

import { useState, type FormEvent } from "react";
import { type BookStatus } from "./StatusBadge";
```

- フォームは `useState` や `onChange` などブラウザ専用の機能を使うため、先頭に `"use client"` を書いて Client Component にします。
- `useState` は「**変化する値を持つための道具**」、`type FormEvent` は「フォーム送信イベントの型」です。`type` を付けたものは型だけのインポートです。
- `BookStatus` 型は `StatusBadge.tsx` から取り込み、ステータス欄の型として使います。

> **用語:** フック（hook）とは、`useState` のように `use` で始まる React の特別な関数。コンポーネントに「状態」や「副作用」などの機能を追加するために使う。Client Component の中でのみ使える。

---

解説2: フォームデータの型と Props の型

```ts
export type BookFormData = {
  title: string;
  author: string;
  publisher: string;
  published_date: string;
  rating: number | null;
  status: BookStatus;
  memo: string;
};

type BookFormProps = {
  initialData?: BookFormData;
  onSubmit: (data: BookFormData) => Promise<void>;
  submitLabel?: string;
  isSubmitting?: boolean;
};
```

- `BookFormData` は「フォームが扱う入力値の形」です。データベースの `books` テーブルに似ていますが、`id`・`created_at`・`updated_at` は含みません（フォームで入力する項目ではないため）。
- `BookFormProps` は親から受け取る値の形です。
- `initialData?` … 編集時の初期値（新規登録時は省略）。`?` で省略可能。
- `onSubmit: (data) => Promise<void>` … **送信時に呼ぶ親の関数**。実際のDB保存は親に任せる設計です。`Promise<void>` は「非同期で、最終的な戻り値は無し」という意味です。
- `submitLabel?`・`isSubmitting?` … ボタンの文字と送信中フラグ（いずれも省略可能）。

> **用語:** コールバック関数とは、「あとで呼んでもらうために他人に渡す関数」のこと。`onSubmit` は親が用意した関数で、フォームは送信時にそれを呼ぶだけ。こうすると同じフォームを登録用・編集用で使い回せる。

---

解説3: state の初期化（formData と errors）

```
const defaultFormData: BookFormData = {
  title: "", author: "", publisher: "", published_date: "",
  rating: null, status: "want_to_read", memo: "",
};

export default function BookForm({
  initialData, onSubmit,
  submitLabel = "登録する", isSubmitting = false,
}: BookFormProps) {
  const [formData, setFormData] = useState<BookFormData>(
    initialData ?? defaultFormData
  );
  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>
({});
```

- `defaultFormData` は新規登録時の初期値です。すべて空（または初期選択）にしてあり、ステータスだけ `"want_to_read"`（読みたい）を初期値にしています。
- `useState<BookFormData>(initialData ?? defaultFormData)` で、**`initialData` があればそれを（編集時）、なければ `defaultFormData` を（新規時）** 初期値にします。`??` は「左が無ければ右を使う」演算子です。
- `errors` の型 `Partial<Record<keyof BookFormData, string>>` は「**エラーが出た項目だけ、項目名→メッセージ で入る箱**」です。`keyof` で項目名一覧を作り、`Record` で「項目名→文字列」、`Partial` で「全部省略可」にしています。初期値は空 `{}`。

> **用語:** `Partial<Record<keyof T, string>>` とは「`T` の各項目名をキーに、エラーメッセージ（文字列）を入れられる、全項目が省略可能なオブジェクト」の型。「エラーが出た項目だけ覚えておく箱」を表す定番の型。

**解説4: バリデーション関数**

```
const validate = (): boolean => {
  const newErrors: Partial<Record<keyof BookFormData, string>> = {};

  if (!formData.title.trim()) {
    newErrors.title = "タイトルは必須です";
  } else if (formData.title.trim().length > 100) {
    newErrors.title = "タイトルは100文字以内で入力してください";
  }
  // ...author, publisher, memo も同様にチェック...

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};
```

- `validate` は「送信してよい内容か」をチェックし、`true`（全部OK）か `false`（エラーあり）を返す関数です。
- `!formData.title.trim()` … `.trim()` は前後の空白を削った文字列を返し、空文字 `""` は「偽（falsy）」なので、`!` を付けると「**空または空白だけ**」を検出できます。タイトル・著者は必須なのでこれで未入力を弾きます。
- `else if (... .length > 100)` … 文字数の上限もチェックしています。
- 最後に `setErrors(newErrors)` で集めたエラーを画面に反映し、`Object.keys(newErrors).length === 0` で「エラーが0個なら true」を返します。

> **用語:** バリデーション（validation・検証）とは、ユーザーの入力が正しいか（必須項目が埋まっているか、文字数は適切かなど）を送信前に確認する処理。問題があればエラーを表示し、送信を止める。

---

解説5: 汎用入力ハンドラ handleChange

```
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>
) => {
  const { name, value } = e.target;

  setFormData((prev) => ({
    ...prev,
    [name]: value,
  }));

  if (errors[name as keyof BookFormData]) {
    setErrors((prev) => ({
      ...prev,
      [name]: undefined,
    }));
  }
};
```

- これ1つで `input`・`textarea`・`select` の**すべての入力欄の変更**を処理する汎用ハンドラです。引数 `e` の型がその3種類のユニオンになっているのはそのためです。
- `const { name, value } = e.target;` で「変化した入力欄の `name` と現在値 `value`」を取り出します。`name` は `<input name="title">` のように指定した名前です。
- `setFormData((prev) => ({ ...prev, [name]: value }))` がキモです。`...prev` で**今の全項目をコピー**し、`[name]: value` で**変化した欄だけを上書き**します。`[name]` は「変数 `name` の中身をキーとして使う」動的キー記法で、これにより1つの関数でどの欄でも正しく更新できます。
- 後半の `if (errors[...])` は「入力し直したら、その欄の古いエラーを消す」親切処理です。

> **用語:** 動的なキー（computed property name・`[name]: value`）とは、オブジェクトのキー名を「変数の中身」で決める記法。`name` が `"title"` なら `{ title: value }`、`"author"` なら `{ author: value }` になり、1つのハンドラで全項目に対応できる。

---

解説6: 評価専用ハンドラ handleRatingChange

```
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,
    rating: value === "" ? null : Number(value),
  }));
};
```

- 評価（rating）だけは専用ハンドラにしています。理由は、`<select>` の値は常に**文字列**ですが、`rating` は**数値 または null** で持ちたいからです。
- `value === "" ? null : Number(value)` … 三項演算子で、「『選択してください』（空文字）なら `null`、そうでなければ `Number(value)` で**数値に変換**」しています。`Number("3")` は `3` になります。
- このひと手間で、画面上は文字列の選択肢でも、内部のデータは正しく「数値 or null」で保てます。

> **用語:** 型変換（キャスト）とは、ある型の値を別の型に変えること。`Number("3")` は「文字列の "3" を数値の 3 に変える」型変換。`<input>` や `<select>` の値は常に文字列なので、数値が必要なときは変換が要る。

---

**解説7: 送信ハンドラ handleSubmit（親に委譲）**

```
const handleSubmit = async (e: FormEvent) => {
  e.preventDefault();

  if (!validate()) {
    return;
  }

  await onSubmit(formData);
};
```

- `e.preventDefault()` は「フォーム送信時にブラウザが**ページ全体をリロード**してしまう既定動作を止める」お決まりの一行です。これを書かないと React の処理が走る前に画面がリセットされます。
- `if (!validate()) { return; }` で、バリデーションに失敗したら（`false`）ここで関数を終え、送信しません。
- `await onSubmit(formData)` が重要です。**実際のDB保存はこのフォームではせず、親から渡された `onSubmit` 関数に丸ごと任せ**ています。`async` / `await` なので、親の非同期処理（DB書き込み）の完了を待てます。
- この「自分では保存せず親に渡す」設計のおかげで、同じフォームを登録ページでも編集ページでも使い回せます。

> **用語:** `e.preventDefault()`（プリベント・デフォルト）とは、イベントの「ブラウザ標準の動作」をキャンセルする命令。フォームの場合は「送信でページがリロードされる」動作を止めるために、`onSubmit` の冒頭でほぼ必ず呼ぶ。

---

**解説8: 制御コンポーネントとしての input**

```
<input
  type="text"
  id="title"
  name="title"
  value={formData.title}
  onChange={handleChange}
  placeholder="例: リーダブルコード"
  className={`
    ...
    ${errors.title ? "border-red-500 bg-red-50" : "border-gray-300"}
  `}
/>
{errors.title && (
  <p className="mt-1 text-sm text-red-600">{errors.title}</p>
)}
```

- `value={formData.title}` と `onChange={handleChange}` の組み合わせが「**制御コンポーネント**」です。入力欄の表示値を常に state（`formData`）から取り、変化したら `handleChange` で state を更新します。これにより「**画面の値とプログラムの値が常に一致**」します。
- `name="title"` は解説5の動的キー（`[name]`）と対応します。この名前のおかげで `handleChange` が「タイトル欄が変わった」と分かります。
- `className` の中の `${errors.title ? "border-red-500 bg-red-50" : "border-gray-300"}` で、エラーがある欄だけ**枠線を赤・背景を薄赤**に切り替えています。
- 下の `{errors.title && (<p ...>)}` は短絡評価で、エラーがあるときだけ赤文字のメッセージを表示します（他のフィールドも同じ作りです）。

> **用語:** 制御コンポーネント（controlled component）とは、入力欄の値を React の state で管理する方式（`value` + `onChange` のペア）。常に state が「唯一の正しい値」になるので、バリデーションや初期値設定がしやすい。

キーを1つ押すたびに、次の1周がぐるっと回っています。**画面の入力欄は state を映す鏡**で、入力欄が自分で文字を覚えているわけではない、というのがポイントです。

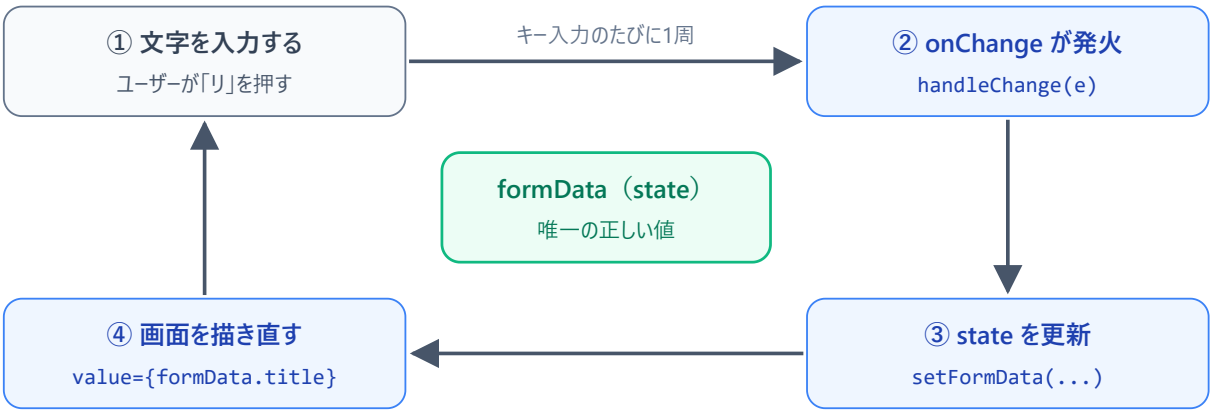



図7-12: 制御コンポーネントの1周（入力 → state 更新 → 描き直し。画面は state を映す鏡）

---

**解説9: 送信ボタン（送信中の無効化と表示切替）**

```
      <button
        type="submit"
        disabled={isSubmitting}
        className={`
          ...
          ${
            isSubmitting
              ? "bg-blue-400 cursor-not-allowed"
              : "bg-blue-600 hover:bg-blue-700"
          }
        `}
      >
        {isSubmitting ? (
          <span className="flex items-center gap-2">
            <svg className="animate-spin h-5 w-5" ...>...</svg>
            送信中...
          </span>
        ) : (
          submitLabel
        )}
      </button>
```

- `type="submit"` で「これがフォームの送信ボタン」と明示します。これを押すと `<form>` の `onSubmit`（＝`handleSubmit`）が走ります。
- `disabled={isSubmitting}` で、送信中はボタンを無効化します。これにより**二重送信（連打）を防止**できます。
- `className` の三項演算子で、送信中は薄い青＋禁止カーソル、通常時は濃い青＋ホバーで色変化、と見た目も切り替えています。

- ボタンの中身も `isSubmitting ? (スピナー + "送信中...") : submitLabel` の三項演算子で切り替え、送信中は回転スピナーを表示します。

> **用語:** 二重送信（double submit）とは、ユーザーがボタンを連打して同じデータが2回送られてしまう問題。送信中はボタンを `disabled` にしておくことで防げる。

---

## 動かして確かめる: BookForm の中のロジック

`BookForm` には、画面（JSX）と、ロジック（入力チェック・型の変換・state の更新）が1つのファイルに同居しています。ロジックの部分だけを取り出せば、ブラウザも Supabase も使わずに、Node.js だけで動きを確かめられます。ここでは3つの小さな実験をします。

### サンプル: バリデーション関数を単体で動かす

> **▼ このコードがやること（先に日本語で）:** `BookForm` の `validate` と同じルール（タイトル・著者は必須、文字数に上限あり）を、画面から切り離した「ただの関数」`validateBookForm` にします。そこへ「何も入力しない」「空白だけ」「タイトルが長すぎる」「正しい入力」の4パターンを渡して、どんなエラーが返るかを確かめます。

```ts
// BookForm.tsx と同じ型（Node で単体実行するためにここへ写している）
type BookStatus = "reading" | "completed" | "want_to_read";

type BookFormData = {
  title: string;
  author: string;
  publisher: string;
  published_date: string;
  rating: number | null;
  status: BookStatus;
  memo: string;
};

// 「エラーが出た項目だけ、項目名 → メッセージ」で入る箱の型
type FormErrors = Partial<Record<keyof BookFormData, string>>;

// BookForm の validate と同じルールを、画面と切り離した「ただの関数」にしたもの
function validateBookForm(data: BookFormData): FormErrors {
  const errors: FormErrors = {};

  // タイトル: 必須、100文字以内
  if (!data.title.trim()) {
    errors.title = "タイトルは必須です";
  } else if (data.title.trim().length > 100) {
    errors.title = "タイトルは100文字以内で入力してください";
  }

  // 著者: 必須、50文字以内
  if (!data.author.trim()) {
    errors.author = "著者は必須です";
  } else if (data.author.trim().length > 50) {
    errors.author = "著者は50文字以内で入力してください";
  }

  // 出版社: 任意、50文字以内
  if (data.publisher.trim().length > 50) {
    errors.publisher = "出版社は50文字以内で入力してください";
  }

  // メモ: 任意、1000文字以内
  if (data.memo.trim().length > 1000) {
    errors.memo = "メモは1000文字以内で入力してください";
  }

  return errors;
}

// 新規登録フォームの初期値（BookForm の defaultFormData と同じ）
const empty: BookFormData = {
```

```
  title: "",
  author: "",
  publisher: "",
  published_date: "",
  rating: null,
  status: "want_to_read",
  memo: "",
};

// 試したい入力のパターンを並べる
const cases: { label: string; data: BookFormData }[] = [
  { label: "1. 何も入力しない", data: empty },
  { label: "2. 空白だけ入力", data: { ...empty, title: "   ", author: "  " } },
  {
    label: "3. タイトルが101文字",
    data: { ...empty, title: "あ".repeat(101), author: "山田" },
  },
  {
    label: "4. 正しい入力",
    data: { ...empty, title: "リーダブルコード", author: "Dustin Boswell" },
  },
];

for (const { label, data } of cases) {
  const errors = validateBookForm(data);
  const canSubmit = Object.keys(errors).length === 0;
  console.log(label, "→", canSubmit ? "送信OK" : "送信しない", errors);
}
```

▼ 実行結果（`node validate-demo.ts`）

```
1. 何も入力しない → 送信しない { title: 'タイトルは必須です', author: '著者は必須です' }
2. 空白だけ入力 → 送信しない { title: 'タイトルは必須です', author: '著者は必須です' }
3. タイトルが101文字 → 送信しない { title: 'タイトルは100文字以内で入力してください' }
4. 正しい入力 → 送信OK {}
```

## ▼ コードを1つずつ分解して解説

**解説1: state ではなく「引数で受け取り、戻り値で返す」**

```
function validateBookForm(data: BookFormData): FormErrors {
```

- `BookForm` の `validate` は、state の `formData` を読み、結果を `setErrors` で state に書き込んでいました。この関数は、入力を引数 `data` で受け取り、エラーを戻り値で返すだけです。

- このように「同じ入力なら必ず同じ結果を返し、外側の何も書き換えない関数」を**純粋関数**と呼びます。画面が無くても試せるのはこのおかげです。
- 実際の `BookForm` でも、`validate` の中身を `const newErrors = validateBookForm(formData);` と `setErrors(newErrors);` の2行に置き換えられます。

> **用語:** 純粋関数（じゅんすいかんすう／pure function）とは、引数だけを材料に戻り値を作り、画面や変数など外側のものを一切変えない関数のこと。結果が予想しやすく、テストもしやすい。

**解説2: 空白だけの入力も trim() で弾ける**

```
if (!data.title.trim()) {
```

- ケース2では、タイトルに半角スペース3つ、著者に**全角スペース**1つを入れました。`trim()` は全角スペースも取り除くので、どちらも空文字になり「必須です」になりました。
- `!` は「空文字（falsy）なら true」に反転させる記号です。`trim()` を付け忘れると、スペースだけのタイトルが登録できてしまいます。

**解説3: スプレッド構文でテスト用のデータを作る**

```
{ label: "2. 空白だけ入力", data: { ...empty, title: "   ", author: "  " } },
```

- `{ ...empty, title: " " }` は「`empty` の中身を全部コピーし、`title` だけ上書きした**新しいオブジェクト**」です。`handleChange` の `{ ...prev, [name]: value }` とまったく同じ書き方です。
- `"あ".repeat(101)` は「"あ" を101回つなげた文字列」を作るメソッドで、上限ちょうどを1文字だけ超える値を手軽に作れます。

### サンプル: select の値を数値に変える（Number の落とし穴）

> **▼ このコードがやること（先に日本語で）:** `<select>` から届く値（見た目が数字でも、中身はいつも文字列）を数値に変えるとき、`Number()` だけで変換した場合と、`BookForm` と同じく「空文字なら null」を先に判定した場合を比べます。「選択してください」（空文字 `""`）を選んだときに何が起きるかに注目してください。

```ts
// <select> や <input> から来る値は、いつも「文字列」
// "" は「選択してください」を選んだときの値
const inputs = ["3", "5", "", "abc"];

for (const value of inputs) {
  // ❌ 素朴に Number() だけで変換した場合
  const naive = Number(value);
  // ✅ BookForm と同じく「空文字なら null」を先に判定した場合
  const safe = value === "" ? null : Number(value);
  console.log(
    `入力 ${JSON.stringify(value).padEnd(5)}`,
    `| Number()だけ → ${String(naive).padEnd(3)}`,
    `| 安全版 → ${safe}`,
  );
}

// 型も確かめる（typeof は値の種類を文字列で返す）
console.log('typeof "3"         =', typeof "3");
console.log('typeof Number("3") =', typeof Number("3"));
```

### ▼ 実行結果（`node number-demo.ts`）

```
入力 "3"   | Number()だけ → 3   | 安全版 → 3
入力 "5"   | Number()だけ → 5   | 安全版 → 5
入力 ""    | Number()だけ → 0   | 安全版 → null
入力 "abc" | Number()だけ → NaN | 安全版 → NaN
typeof "3"         = string
typeof Number("3") = number
```

### ▼ コードを1つずつ分解して解説

**解説1: Number("") は 0 になる**

```ts
  const naive = Number(value);
```

- `Number("3")` は `3` ですが、`Number("")`（空文字）は `NaN` **ではなく** `0` になります。JavaScript の有名な落とし穴です。
- もし `rating: Number(value)` とだけ書いていたら、「選択してください」を選んだ本が**評価 0** として保存されようとします。books テーブルには `CHECK (rating >= 1 AND rating <= 5)` という制約（第5章）があるので、Supabase からエラーが返って登録に失敗します。

**解説2: 空文字を先に null にする**

```
  const safe = value === "" ? null : Number(value);
```

- `BookForm` の `handleRatingChange` と同じ書き方です。空文字かどうかを先に調べ、空なら `null`（評価なし）、そうでなければ数値に変換します。
- `"abc"` のような数字でない文字は `NaN`（ナン／Not a Number：数ではない）になります。このアプリでは `<option>` の値が `"1"` 〜 `"5"` に固定されているので起きませんが、自由に入力できる欄では `Number.isNaN(値)` でのチェックが必要です。

> **用語:** `typeof`（タイプオブ）とは、値の種類を `"string"`（文字列）・`"number"`（数値）などの文字列で教えてくれる演算子。「見た目は 3 でも中身は文字列」という状態を見破るのに使える。

> **つまずきポイント: 評価（rating）に 0 や文字列を送ってしまう**
>
> `<select>` の値は、見た目が数字でも中身はいつも**文字列**です。変換を省いたり `Number()` だけで済ませたりすると、Supabase への登録で失敗します。

```
// ❌ NG: Number() だけで変換している
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  setFormData((prev) => ({
    ...prev,
    // 「選択してください」を選ぶと Number("") で 0 になってしまう
    rating: Number(e.target.value),
  }));
};
```

```
// ✅ OK: 空文字なら null、それ以外は数値に（BookForm の書き方）
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,
    rating: value === "" ? null : Number(value),
  }));
};
```

> NG のまま「選択してください」で登録すると、Supabase から `new row for relation "books" violates check constraint "books_rating_check"`（code: `23514`、CHECK 制約違反）というエラーが返ります。また、変換せずに空文字 `""` のまま送ると `invalid input syntax for type integer: ""`（code: `22P02`、整数として読めない）になります。エラーコードの読み方は 7節の「問題2」で詳しく扱います。

### サンプル: state を直接書き換えると画面が変わらない理由

> **▼ このコードがやること（先に日本語で）**: React は state が「変わったかどうか」を、中身ではなく**箱（オブジェクト）が入れ替わったかどうか**で判断します。その判断（ `Object.is` ）をまねた関数を作り、①今の箱を直接書き換えた場合と、②スプレッド構文で新しい箱を作った場合を比べます。

```
// React は「前の state」と「次の state」を Object.is で比べて、
// 同じ箱（同じオブジェクト）なら「変化なし」とみなして再描画しない。
// その判定をまねる関数
function willRerender(prev: object, next: object): boolean {
  return !Object.is(prev, next);
}

// ❌ NG: 今の state を直接書き換えて、同じ箱をそのまま返す
const prev1 = { title: "", author: "" };
const next1 = prev1;
next1.title = "リーダブルコード";
console.log("NG: 再描画される？", willRerender(prev1, next1), "/ 中身:", next1);

// ✅ OK: スプレッド構文で「新しい箱」を作り、変えたい項目だけ上書きする
const prev2 = { title: "", author: "" };
const next2 = { ...prev2, title: "リーダブルコード" };
console.log("OK: 再描画される？", willRerender(prev2, next2), "/ 中身:", next2);
console.log("OK: 元の箱はそのまま:", prev2);
```

**▼ 実行結果（ `node state-demo.ts` ）**

```
NG: 再描画される？ false / 中身: { title: 'リーダブルコード', author: '' }
OK: 再描画される？ true / 中身: { title: 'リーダブルコード', author: '' }
OK: 元の箱はそのまま: { title: '', author: '' }
```

### ▼ コードを1つずつ分解して解説

**解説1: Object.is は「同じ箱かどうか」を見る**

```
function willRerender(prev: object, next: object): boolean {
  return !Object.is(prev, next);
}
```

- `Object.is(a, b)` は、`a` と `b` が「まったく同じもの」なら `true` を返します。オブジェクトの場合は「中身が同じか」ではなく「**同じ1つの箱か**」を比べます。
- React も `setFormData` が呼ばれるたびに、前の値と次の値をこの方法で比べ、同じ箱なら描き直しを省略します。

**解説2: 直接書き換えると「同じ箱」のまま**

```
const next1 = prev1;
next1.title = "リーダブルコード";
```

- `const next1 = prev1;` はコピーではありません。**同じ箱に名前を2つ付けた**だけです。
- 中身（`title`）は確かに書き換わっていますが、箱は同じなので「再描画されない（false）」と判定されました。

**解説3: スプレッド構文なら「新しい箱」**

```
const next2 = { ...prev2, title: "リーダブルコード" };
```

- `{ ...prev2 }` で新しい箱を作ってから `title` を上書きしたので、「再描画される（true）」になりました。
- 元の `prev2` は空のまま残っています。古い値を壊さないので、React が「前と今」を正しく比べられます。

> **つまずきポイント: state を直接書き換えて、入力しても文字が増えない**

```
// ❌ NG: prev（今の state）を直接書き換えて、同じ箱を返している
setFormData((prev) => {
  prev.title = value;
  return prev;
});
```

```
// ✅ OK: 新しい箱を作り、変えたい項目だけ上書きする（BookForm の書き方）
setFormData((prev) => ({
  ...prev,
  [name]: value,
}));
```

> NG 版では React が「変化なし」と判断して描き直しを省くため、キーを押しても入力欄の文字が増えません（図7-12 の③から④へ進まない状態）。state のオブジェクトや配列は「直接いじらず、新しく作り直して渡す」と覚えておきましょう。

> **つまずきポイント: input の name と state のキー名がずれている**
>
> `handleChange` は `[name]: value` で「入力欄の `name` と同じ名前のキー」を更新します。1文字でもずれると、正しい項目が更新されません。

```
{/* ❌ NG: name が "Title"（大文字始まり）。state のキーは "title" なので一致しない */}
<input id="title" name="Title" value={formData.title} onChange={handleChange} />
```

```
{/* ✅ OK: name を BookFormData のキー名とまったく同じにする */}
<input id="title" name="title" value={formData.title} onChange={handleChange} />
```

> NG 版では、`formData` に `Title` という**別のキーが増えるだけ**で、`title` は空のままです。その結果「入力しても欄に文字が出ない」「何度送信しても『タイトルは必須です』が消えない」という不思議な症状になります。`name` はただの文字列なので TypeScript も気づいてくれません。大文字・小文字や `published_date` の `_`（アンダースコア）まで、型のキー名と一字一句そろえましょう。

---

## 4b. 登録ページ

**ファイル:** `app/books/new/page.tsx`

BookForm を使って新規登録画面を構成するページです。

> **▼ このコードがやること（先に日本語で）**：さきほどの `BookForm` を画面に置き、**フォームが送信されたらSupabase に新しい書籍を INSERT（新規追加）し、成功したらトップページへ戻す**処理を書きます。カギは送信処理の流れ——①送信中フラグを立ててボタンを無効化（二重送信防止）、② `supabase.from("books").insert({...})` でデータベースに登録、③成功なら `router.push("/")` で一覧へ戻り `router.refresh()` で最新データを取り直す、④失敗ならエラーメッセージを表示。`try / catch / finally` で「成功・失敗どちらでも最後にフラグを戻す」点も押さえどころです。初心者は「**フォームの送信を受け取ってDBに保存し、画面を切り替える係**」と捉えれば十分です（各行の意味はコード内コメントにあります）。

```tsx
// app/books/new/page.tsx  ← URL "/books/new" に対応するページ（App Router のファイルベースルーティング）
// useState・useRouter などブラウザ専用機能を使うので Client Component
"use client";

/**
 * 書籍登録ページ
 *
 * "use client" を付けている理由:
 * - BookForm（Client Component）の onSubmit コールバックで
 *   Supabase への INSERT 処理を行う
 * - useRouter で登録後のリダイレクトを行う
 * - useState で送信中の状態を管理する
 *
 * ※ Server Action を使う方法もあるが、この章では
 *   初心者にとって分かりやすい Client Component 方式で実装する。
 */

// React の状態管理フック
import { useState } from "react";
// Next.js App Router のページ遷移フック
import { useRouter } from "next/navigation";
// ブラウザ用 Supabase クライアント（サーバー用と別物）
import { createClient } from "@/lib/supabase/client";
// フォーム本体と型
import BookForm, { type BookFormData } from "@/components/BookForm";

export default function NewBookPage() {
  // useRouter() でルーター操作オブジェクトを取得。
  // .push("/path") で遷移、.refresh() で Server Component の再実行など。
  const router = useRouter();

  // 送信中かどうかの状態。
  // ボタンの「送信中...」表示や、二重送信防止に使う。
  const [isSubmitting, setIsSubmitting] = useState(false);

  // エラーメッセージの状態。
  // 「string | null」で「エラー文字列があるか、null（エラーなし）」を表現。
  const [submitError, setSubmitError] = useState<string | null>(null);

  /**
   * フォーム送信時の処理
   * BookForm の onSubmit に渡すコールバック関数
   */
  const handleSubmit = async (data: BookFormData) => {
    // (1) 「送信中フラグ」を ON にする → ボタンが「送信中...」表示になり無効化される
    //     setSubmitError(null) で前回のエラーメッセージをクリアする
    setIsSubmitting(true);
    setSubmitError(null);
```

```
// (2) try / catch / finally の3ブロック構成
//     - try     : 通常実行する処理。途中で例外（Error）が起きると catch へ飛ぶ
//     - catch   : 想定外のエラーをキャッチ
//     - finally : 成功・失敗どちらでも最後に必ず実行される
try {
  // (3) Supabase クライアントを作る
  //     ここではブラウザ用クライアント（ブラウザのJSから直接DBにつなぐ）。
  //     RLSポリシーがあるので、許可された操作だけが通る。
  const supabase = createClient();

  // (4) books テーブルに INSERT する
  //     .from("books")  : 操作対象テーブルを指定
  //     .insert({...})  : 書き込みたいレコードのオブジェクト
  //     await           : 非同期処理を待ち、結果を { error } に展開
  //
  //     入力ガード:
  //       data.title.trim()                  ← 前後の空白を削る
  //       data.publisher.trim() || null      ← 空文字列なら null（DBではnullで保存）
  //       data.published_date || null        ← 同上
  //       data.memo.trim() || null           ← 同上
  //     こうすることで「空欄」と「null」を区別せずに DB に綺麗な値が入る。
  const { error } = await supabase.from("books").insert({
    title: data.title.trim(),
    author: data.author.trim(),
    publisher: data.publisher.trim() || null,
    published_date: data.published_date || null,
    rating: data.rating,
    status: data.status,
    memo: data.memo.trim() || null,
  });

  // (5) Supabase が返す error は「成功なら null、失敗ならオブジェクト」。
  //     エラーがあれば、ユーザー向けメッセージをセットして関数を終了。
  if (error) {
    console.error("書籍の登録に失敗しました:", error.message);
    setSubmitError(
      "書籍の登録に失敗しました。入力内容を確認して再度お試しください。"
    );
    return;
  }

  // (6) 登録成功 → トップページに戻す
  //     router.push("/")    : URLを "/" に変える（クライアント側遷移）
  //     ページ全体のリロードは起きないので体感が速い
  router.push("/");

  // (7) router.refresh() で Server Component を再実行させ、
  //     登録したばかりのデータも含む最新の一覧を取得・表示する。
  router.refresh();
```

```
    } catch (err) {
      // (8) 想定外のエラー（環境変数の設定漏れでクライアント作成に失敗した、など）の最後の砦
      //     ※ 通信切断は supabase-js が throw せず (5) の error に入れて返すので、ここには来ない
      console.error("予期しないエラー:", err);
      setSubmitError("予期しないエラーが発生しました。");
    } finally {
      // (9) 成功・失敗いずれでもボタンの「送信中」状態は解除する
      setIsSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen bg-gray-50">
      {/* ヘッダー */}
      <header className="bg-white shadow-sm border-b border-gray-200">
        {/* max-w-3xl = 最大幅 48rem (約768px)。フォームは狭いほうが読みやすいので一覧画面より狭くしている。 */}
        <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center gap-4">
            {/* 戻るボタン。丸い灰色ボタンに矢印アイコンを入れた「戻る」UI。
                aria-label でスクリーンリーダーに「トップに戻る」と読ませる。 */}
            <a
              href="/"
              className="
                inline-flex items-center justify-center
                w-10 h-10
                /* 完全に丸い形 */
                rounded-full
                bg-gray-100
                text-gray-600
                hover:bg-gray-200
                transition-colors duration-200
              "
              aria-label="トップに戻る"
            >
              {/* 左向き矢印を描く SVG。
                  d="M15 19l-7-7 7-7" は「(15,19)→(8,12)→(15,5)」と進む線（左向き「く」の字）。 */}
              <svg
                className="w-5 h-5"
                fill="none"
                stroke="currentColor"
                viewBox="0 0 24 24"
              >
                <path
                  strokeLinecap="round"
                  strokeLinejoin="round"
                  strokeWidth={2}
                  d="M15 19l-7-7 7-7"
```

```
                />
              </svg>
            </a>

            {/* ページタイトル */}
            <div>
              <h1 className="text-2xl font-bold text-gray-900">
                書籍を登録する
              </h1>
              <p className="mt-1 text-sm text-gray-500">
                新しい書籍の情報を入力してください
              </p>
            </div>
          </div>
        </div>
      </header>

      {/* メインコンテンツ */}
      <main className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        {/* エラーメッセージ（送信失敗時に表示）。
            submitError が null なら短絡評価で何も表示しない。
            文字列が入っているときだけ赤い警告ボックスが現れる。 */}
        {submitError && (
          <div className="mb-6 p-4 bg-red-50 border border-red-200 rounded-lg">
            <div className="flex items-center gap-2">
              {/* 円形の警告アイコン（中に「!」風の縦線と点）。
                  shrink-0 は「テキストが長くてもアイコンを縮めない」指定（Tailwind v3 では
flex-shrink-0 とも書けた）。 */}
              <svg
                className="w-5 h-5 text-red-600 shrink-0"
                fill="none"
                stroke="currentColor"
                viewBox="0 0 24 24"
              >
                <path
                  strokeLinecap="round"
                  strokeLinejoin="round"
                  strokeWidth={2}
                  d="M12 8v4m0 4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z"
                />
              </svg>
              <p className="text-sm text-red-700">{submitError}</p>
            </div>
          </div>
        )}

        {/* フォーム本体。BookForm に必要な Props を渡す。
            - onSubmit     : 上で定義したフォーム送信処理
            - submitLabel  : ボタンに表示する文字
            - isSubmitting : 送信中フラグ（ボタンを薄くしてスピナーを出すため） */}
```

```
        <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6 sm:p-
8">
          <BookForm
            onSubmit={handleSubmit}
            submitLabel="登録する"
            isSubmitting={isSubmitting}
          />
        </div>
      </main>
    </div>
  );
}
```

**「登録ボタンを押すとこうなる」の説明:**

1. ユーザーがフォームの各フィールドに情報を入力し、「登録する」ボタンをクリックします。
2. まず **クライアント側でバリデーション** が行われます。- タイトルと著者が空の場合、赤いエラーメッセージ（「タイトルは必須です」等）が該当フィールドの下に表示され、送信は行われません。- 文字数制限を超えている場合も同様です。
3. バリデーションに通ると、「登録する」ボタンが **薄い青色に変わり**、スピナー（回転アイコン）と「送信中...」というテキストに変化します。ボタンは無効化（`disabled`）されるため、二重送信を防止します。
4. Supabase への INSERT リクエストが **ブラウザから直接** Supabase API に送信されます。
5. **成功した場合**: `router.push("/")` によりトップページにリダイレクトされます。一覧が再読み込みされ、今登録した書籍がリストの先頭に表示されます。画面遷移はスムーズで、ページ全体がリロードされることはありません。
6. **失敗した場合**: フォームの上部に赤い背景のエラーメッセージ（「書籍の登録に失敗しました。入力内容を確認して再度お試しください。」）が表示されます。フォームの入力内容は保持されるので、修正して再送信できます。

## ▼ コードを1つずつ分解して解説

この登録ページ（`NewBookPage`）は、`BookForm` から送信を受け取り、Supabase に INSERT して画面遷移する「送信処理の本体」を持ちます。塊ごとに見ていきましょう。

---

**解説1: "use client" と import**

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";
import BookForm, { type BookFormData } from "@/components/BookForm";
```

- このページは `useState`・`useRouter` などブラウザ専用機能を使うため、`"use client"` を付けて Client Component にします。
- `useRouter` は Next.js のページ遷移用フックです。登録後にトップへ戻すために使います。

- `createClient` は「**ブラウザ用** の Supabase クライアント」です。トップページ（`page.tsx`）で使った**サーバー用とは別物**である点に注意してください。
- `BookForm` とその型 `BookFormData` を取り込み、フォーム本体として使います。

> **用語:** サーバー用クライアントとブラウザ用クライアント。Supabase には用途別に2種類のクライアント作成関数があり、Server Component ではサーバー用、Client Component ではブラウザ用を使う。混同すると正しく動かない。

---

**解説2: state とルーターの準備**

```
export default function NewBookPage() {
  const router = useRouter();
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [submitError, setSubmitError] = useState<string | null>(null);
```

- `const router = useRouter();` でページ遷移を操作するオブジェクトを取得します。`router.push("/")` で移動、`router.refresh()` で再取得などができます。
- `isSubmitting`（送信中フラグ）は、ボタンの「送信中...」表示や二重送信防止に使う `true` / `false` の state です。
- `submitError` は送信失敗時のエラーメッセージです。型 `string | null` で「メッセージがあるか、`null`（エラーなし）か」を表します。

> **用語:** `useRouter`（ユーズ・ルーター）とは、コードからページ遷移や再読み込みを行うための Next.js のフック。`router.push("/path")` でリンクを踏まずにプログラム側で画面を移動できる。

---

**解説3: 送信開始の準備とフラグ操作**

```
  const handleSubmit = async (data: BookFormData) => {
    setIsSubmitting(true);
    setSubmitError(null);
```

- `handleSubmit` は `BookForm` の `onSubmit` に渡す関数です。フォームがバリデーションを通したあと、入力データ `data` を引数として呼んでくれます。
- 最初に `setIsSubmitting(true)` で送信中フラグを立てます。これがフォームの送信ボタンに伝わり、ボタンが無効化＆スピナー表示になります。
- `setSubmitError(null)` で、前回の送信で出たエラーメッセージをクリアしておきます。

> **用語:** ローディング状態の管理とは、「処理中かどうか」を state で持ち、それに応じてUI（ボタンの無効化やスピナー）を切り替えること。ユーザーに「今処理中です」と伝え、操作の重複を防ぐ。

---

**解説4: try で Supabase に INSERT する**

```
    try {
      const supabase = createClient();

      const { error } = await supabase.from("books").insert({
        title: data.title.trim(),
        author: data.author.trim(),
        publisher: data.publisher.trim() || null,
        published_date: data.published_date || null,
        rating: data.rating,
        status: data.status,
        memo: data.memo.trim() || null,
      });

      if (error) {
        console.error("書籍の登録に失敗しました:", error.message);
        setSubmitError("書籍の登録に失敗しました。入力内容を確認して再度お試しください。");
        return;
      }
```

- `createClient()` でブラウザ用 Supabase クライアントを作り、`supabase.from("books").insert({...})` で**新しい行を追加**（INSERT）します。`await` で完了を待ちます。
- `insert` に渡すオブジェクトでは「入力ガード」をしています。`data.publisher.trim() || null` は「空文字なら `null`」という意味で、`||`（OR）は左が空文字（falsy）のとき右の `null` を返します。これにより「空欄」をデータベースに `null` できれいに保存できます。
- `if (error)` で失敗を判定し、エラーがあればユーザー向けメッセージを `setSubmitError` でセットして `return`（中断）します。
- `console.error` には開発者向けの技術的詳細、`setSubmitError` には一般ユーザー向けの分かりやすい文言、と**出し分け**ているのもポイントです。

> **用語:** INSERT（インサート）とは、SQL で「テーブルに新しい行を追加する」操作。`supabase.from("テーブル").insert({...})` が、その Supabase 版の書き方になる。

---

**解説5: 成功時の画面遷移**

```
      router.push("/");
      router.refresh();
```

- INSERT が成功したら（`if (error)` を通り抜けたら）、`router.push("/")` でトップページに移動します。ページ全体のリロードは起きないので体感が速いです。
- 続く `router.refresh()` が重要です。トップページは Server Component なので、これを呼んで**サーバー側を再実行**させ、「今登録したばかりの書籍も含む最新の一覧」を取り直して表示します。
- `push` だけだとキャッシュされた古い一覧が出ることがあるため、`refresh` をセットで呼んで最新化しています。

> **用語:** `router.refresh()` とは、現在のページの Server Component を再実行し、最新データで作り直す Next.js の機能。クライアント側の入力状態は保ったまま、サーバー由来の表示だけを更新できる。

---

**解説6: catch と finally（エラーと後始末）**

```
    } catch (err) {
      console.error("予期しないエラー:", err);
      setSubmitError("予期しないエラーが発生しました。");
    } finally {
      setIsSubmitting(false);
    }
  };
```

- `catch (err)` は、`try` の中で**想定外の例外**（環境変数の設定漏れで `createClient()` が失敗した、自分のコードのミスで例外が起きた、など）が起きたときに実行されます。`if (error)` で拾えない種類のトラブルの最後の砦です。なお、通信切断のような失敗は supabase-js が throw せずに `error` へ入れて返すので、`if (error)` の側で拾われます。
- `finally` は「**成功・失敗どちらでも、最後に必ず実行される**」ブロックです。ここで `setIsSubmitting(false)` を呼び、送信中フラグを必ず解除します。
- `finally` に置くことで、「成功しても失敗しても、ボタンの『送信中』状態が解除されずに固まる」事故を確実に防げます。

> **用語:** `try / catch / finally` とは、エラー処理の構文。`try` で通常処理を試み、`catch` で例外を捕まえ、`finally` で「結果に関わらず必ず行う後始末」を書く。送信中フラグの解除のような後始末は `finally` が最適。

---

解説7: JSX — エラー表示とフォームの配置

```
        {submitError && (
          <div className="mb-6 p-4 bg-red-50 border border-red-200 rounded-lg">
            {/* ...警告アイコン... */}
            <p className="text-sm text-red-700">{submitError}</p>
          </div>
        )}

        <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6 sm:p-
8">
          <BookForm
            onSubmit={handleSubmit}
            submitLabel="登録する"
            isSubmitting={isSubmitting}
          />
        </div>
```

- `{submitError && (...)}` は短絡評価で、`submitError` に文字列があるときだけ、フォーム上部に赤い警告ボックスを表示します。`null` のときは何も出ません。
- `<BookForm ... />` に3つの Props を渡しています。
- `onSubmit={handleSubmit}` … 上で定義した送信処理（INSERT〜遷移）
- `submitLabel="登録する"` … ボタンの文字
- `isSubmitting={isSubmitting}` … 送信中フラグ（ボタンの無効化・スピナー表示に使う）
- このように「**フォームの見た目・入力管理は `BookForm`、実際の保存処理はこのページ**」と役割分担しているのが、この設計の要点です。

> **用語:** 関心の分離（separation of concerns）とは、「見た目・入力管理」と「データ保存」のように役割ごとに担当を分ける設計の考え方。`BookForm`（入力係）と `NewBookPage`（保存係）に分けることで、それぞれが小さく分かりやすくなる。

> **つまずきポイント: 送信中もボタンが押せて、同じ本が2冊登録される**
>
> 送信中フラグは「見た目のため」だけのものではありません。`NewBookPage` の `isSubmitting`（state）→ `BookForm` の `isSubmitting`（props）→ ボタンの `disabled`、とつながって、**データを守る**役目を果たしています。

```
{/* ❌ NG: 送信中もボタンが押せる。通信が遅いと、連打で2回 INSERT される */}
<button type="submit">登録する</button>
```

```
{/* ✅ OK: 送信中フラグで無効化し、表示も切り替える（BookForm の書き方） */}
<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? "送信中..." : "登録する"}
</button>
```

> NG 版で回線が遅いと、1回目の INSERT が終わる前に2回目のクリックが受け付けられ、同じ内容の行が2つできてしまいます。books テーブルの `id` は INSERT のたびに新しく作られるので、データベースは「同じ本だ」と気づいて止めてはくれません。開発者ツールの Network タブで通信速度を「Slow 4G」などに落とすと、連打の危険性を自分の目で確かめられます。

---

## 4c. 登録フローの図解

1 **ユーザー → BookForm**
フォームに情報を入力し、「登録する」ボタンをクリック

2 **BookForm 内部**
バリデーション実行

! **バリデーションエラーの場合**
エラーメッセージをユーザーに表示（処理中断）

3 **BookForm → NewBookPage**
onSubmit(formData) を呼び出し

4 **NewBookPage**
isSubmitting = true（ボタン無効化・スピナー表示）

5 **NewBookPage → Supabase**
supabase.from("books").insert(data)

6 **INSERT 成功の場合**
router.push("/") → router.refresh() → トップページにリダイレクト（一覧に新しい書籍が表示される）

! **INSERT 失敗の場合**
isSubmitting = false → エラーメッセージを表示（フォーム内容は保持）

上の流れを、登場人物ごとのシーケンス図にすると次のようになります。注目してほしいのは後半の ⑦〜⑩ です。

`router.refresh()` によって**一覧ページ（Server Component）がもう一度動き、Supabase から取り直す**ので、今登録した本が一覧に現れます。

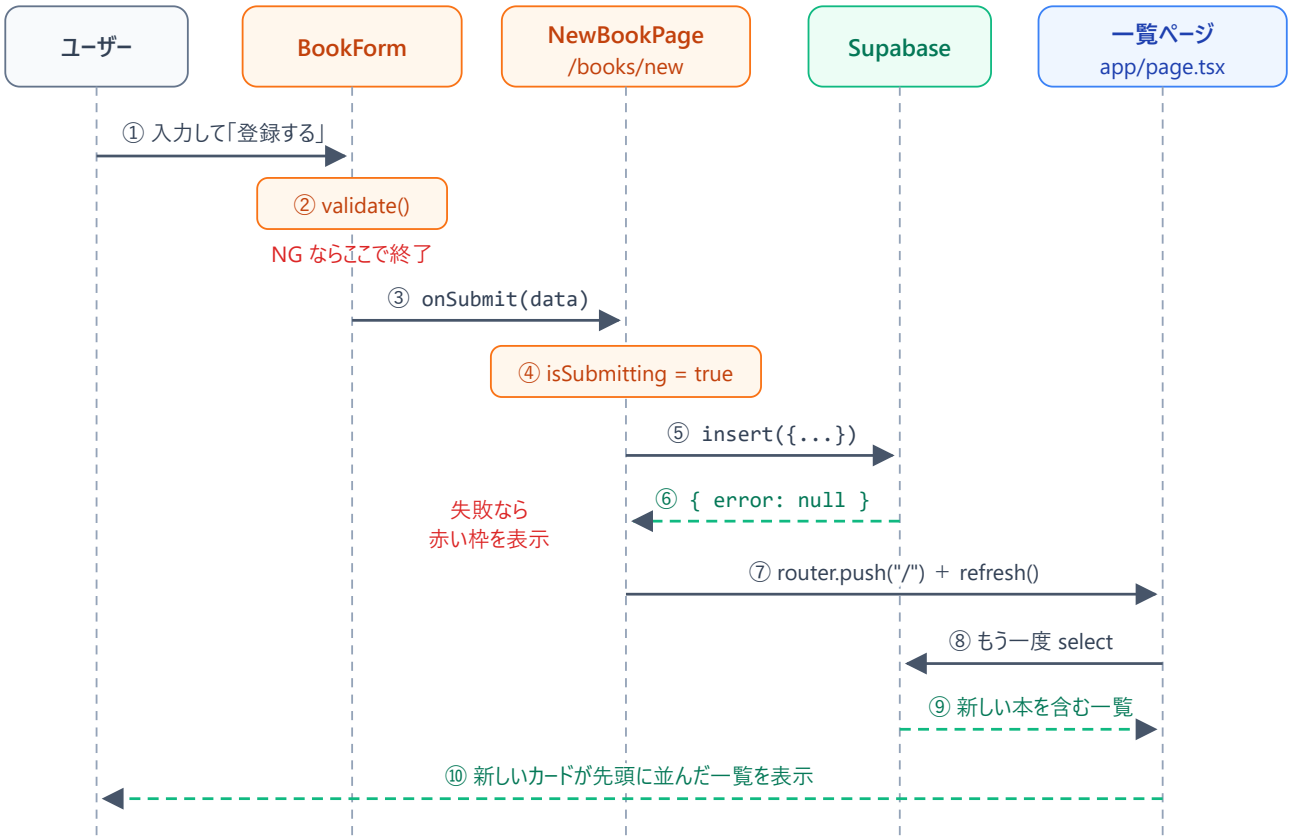



図7-13: 新規登録から一覧が更新されるまでのシーケンス図（実線＝お願い、点線＝返事）

## 5. コード解説

ここまでに実装したコードの重要なポイントを、初心者向けに詳しく解説します。

### 5-1. Server Component と Client Component の使い分け

この章で作ったコンポーネントは、2種類に分かれています。

| コンポーネント | 種類 | 理由 |
| --- | --- | --- |
| `app/page.tsx` | Server Component | Supabase からのデータ取得をサーバー側で行うため |
| `StatusBadge` | Server Component | 状態を持たない純粋な表示コンポーネントのため |
| `RatingStars` | Server Component | 同上 |
| `BookCard` | Server Component | 同上 |
| `BookList` | Server Component | 同上 |
| `LoadingSpinner` | Client Component | アニメーションの再利用性のため（`"use client"`） |
| `app/error.tsx` | Client Component | **Next.js の仕様上必須**（`"use client"`） |
| `BookForm` | Client Component | useState, onChange 等を使うため（`"use client"`） |
| `app/books/new/page.tsx` | Client Component | useRouter, useState を使うため（`"use client"`） |

**なぜこの使い分けが重要なのか:**

- **Server Component のメリット**: JavaScript バンドルに含まれないため、ブラウザに送信されるデータが少なく、ページの読み込みが速くなります。また、サーバー上で直接データベースにアクセスできるため、API エンドポイントを別途作る必要がありません。
- **Client Component のメリット**: `useState`、`useEffect`、イベントハンドラなどの React のインタラクティブ機能を使えます。ユーザーの操作に応じて画面を動的に更新できます。

**初心者がつまづきやすいポイント:**

> 「**"use client" をどこに書くか迷う**」— 原則として、`useState` や `useEffect`、`onClick` などのイベントハンドラを使うコンポーネントにだけ `"use client"` を付けます。書かなければ Server Component になります（App Router のデフォルト）。「迷ったら Server Component のままにして、エラーが出たら `"use client"` を追加する」というアプローチでも構いません。

## 5-2. statusConfig のパターン（StatusBadge）

```
const statusConfig: Record<BookStatus, { label: string; bgColor: string; textColor:
string }> = {
  reading: { label: "読書中", bgColor: "bg-blue-100", textColor: "text-blue-800" },
  // ...
};
```

**なぜ if 文ではなくオブジェクトを使うのか:**

`if (status === "reading") { ... } else if (status === "completed") { ... }` と書くこともできますが、オブジェクトで管理する方が:

1. **コードが短くなる**: 条件分岐を書かなくてよい。
2. **拡張しやすい**: 新しいステータスを追加するときにオブジェクトに1行追加するだけ。
3. **型安全**: `Record<BookStatus, ...>` を使うことで、全てのステータスに対する設定を定義し忘れるとTypeScriptがエラーを出す。

## 5-3. `line-clamp-2` について（BookCard）

```
<h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
```

`line-clamp-2` は Tailwind CSS のユーティリティクラスで、テキストを2行までに制限し、それを超える部分を `...` で省略します。書籍のタイトルが長い場合にカードのレイアウトが崩れるのを防ぎます。

## 5-4. Book 型と Supabase のテーブル構造の対応

```
export type Book = {
  id: string;
  title: string;
  author: string;
  publisher: string | null;
  // ...
};
```

**なぜ `publisher` が `string | null` なのか:**

データベースの `publisher` カラムは NULL を許容しています（任意入力項目だから）。TypeScript では `null` を明示的に型に含める必要があるため、`string | null` としています。

**初心者がつまづきやすいポイント:**

> **`null` と `undefined` の違い** — データベースから取得した値が「存在しない」場合は `null` が返ります。一方、JavaScript のオブジェクトで「プロパティ自体が存在しない」場合は `undefined` になります。Supabase から返ってくるデータは常に `null`（`undefined` ではない）なので、型定義は `string | null` とします。

## 5-5. フォームの状態管理（BookForm）

```
const [formData, setFormData] = useState<BookFormData>(initialData ?? defaultFormData);
```

**なぜ1つの state にまとめるのか:**

各フィールドごとに別々の `useState` を使う方法もあります:

```
// NG ではないが、フィールド数が多いと管理が大変
const [title, setTitle] = useState("");
const [author, setAuthor] = useState("");
const [publisher, setPublisher] = useState("");
// ...7つの useState...
```

これだと、フィールドが増えるたびに `useState` が増え、`handleChange` も個別に書く必要があります。1つのオブジェクトにまとめることで:

1. **コードが簡潔**: `handleChange` を全フィールドで共有できる。
2. **データの受け渡しが楽**: `onSubmit(formData)` で全フィールドを一度に渡せる。
3. **初期値の設定が楽**: `initialData ?? defaultFormData` で一括設定できる（編集時に便利）。

## 5-6. `router.push` と `router.refresh` の組み合わせ（NewBookPage）

```
// URLを変えて遷移
router.push("/");
// Server Component のデータを取り直す
router.refresh();
```

**なぜ両方必要なのか:**

- `router.push("/")` はトップページに遷移しますが、Next.js はパフォーマンスのためにページの内容を**キャッシュ**（一時保存）しています。キャッシュが残っていると、登録前の古いデータが表示されることがあります。
- `router.refresh()` はキャッシュを無効化し、Server Component を再実行させます。これにより、Supabase から最新のデータが取得され、新しく登録した書籍が一覧に反映されます。これが冒頭で触れた **revalidation（リバリデーション／再検証）** の一つの形です。

> **補足: Server Action 方式なら `revalidatePath`**: Server Action を使う場合は、`router.refresh()` の代わりにサーバー側で `revalidatePath("/")` を呼びます。意味は同じ「キャッシュを破棄して次の表示で取り直す」ですが、サーバー側からの指示なのでより確実です。

## 5-7. 空文字列を null に変換する理由

```
publisher: data.publisher.trim() || null,
```

**なぜ空文字列のまま保存しないのか:**

- データベースでは「値がない」ことを `NULL` で表現するのが一般的です。

- 空文字列 `""` と `NULL` は意味が異なります。`""` は「入力したが空だった」、`NULL` は「入力されていない」を意味します。
- 将来的に「出版社が未入力の書籍」を検索する場合、`WHERE publisher IS NULL` で統一的に取得できます。`""` が混在すると `WHERE publisher IS NULL OR publisher = ''` と書く必要があり面倒です。
- `data.publisher.trim() || null` は「空白を除去した後に空文字列なら null にする」という意味です。`||` 演算子は左辺が falsy（空文字列は falsy）なら右辺を返します。

## 5-8. e.preventDefault() の役割

```
const handleSubmit = async (e: FormEvent) => {
  // ブラウザの「フォーム送信→画面遷移」を止める
  e.preventDefault();
  // ...
};
```

HTML の `<form>` は、送信ボタンが押されるとデフォルトでページ全体をリロードしてデータを送信しようとします（昔ながらの「サーバーへPOSTしてHTMLを返してもらう」挙動）。`e.preventDefault()` はこのデフォルト動作を止め、JavaScript で非同期にデータを送信できるようにします。これを忘れるとページがリロードされ、`useState` の値がすべてリセットされてしまいます。

## 5-9. FormData と FormDataEntryValue について（参考）

この章ではブラウザ用 Supabase クライアントで `useState` の値を直接 INSERT していますが、Server Action 方式（参考コラム参照）では `formData.get("title")` のようにフォームから値を取り出します。その時の戻り値の型に注意が必要です。

```
// formData.get の戻り値の型
const value: FormDataEntryValue | null = formData.get("title");
//           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//           FormDataEntryValue = string | File
//           フィールドが存在しないと null
```

- **FormDataEntryValue**: 「文字列か File（ファイル）か」のどちらか。普通の `<input type="text">` なら string、`<input type="file">` なら File。
- **null**: そもそもそのフィールドが送信されなかった場合（HTML 上に無い、disabled だった、など）。

そのため Server Action では `String(formData.get("title") ?? "")` のように、まず `??` で null を空文字に変換し、その後 `String()` で文字列にキャストするのが安全です。

```
const title = String(formData.get("title") ?? "");
//                                     ^^^^^   null だったら "" を使う
//            ^^^^^^                           string | File を string に変換
```

### 動かして確かめる: FormData から値を取り出して検証する

Server Action 方式に進んだときに必ず書くことになるのが、「`FormData` から値を取り出し、型をそろえ、検証する」処理です。Node.js にもブラウザと同じ `FormData` が用意されているので、画面無しで試せます。

> **▼ このコードがやること（先に日本語で）：** `FormData` から書籍の各項目を取り出し、①前後の空白を削る、②評価を文字列から数値に変える、③ステータスが決められた3種類か調べる、④空の任意項目を `null` にそろえる、までを行う関数 `parseBookForm` を作ります。普通の入力と、開発者ツールなどで書き換えられた「ありえない入力」の2つを渡して、結果を比べます。

```ts
type BookStatus = "reading" | "completed" | "want_to_read";

// データベースに INSERT する直前の形（books テーブルの列名に合わせる）
type BookInsert = {
  title: string;
  author: string;
  publisher: string | null;
  published_date: string | null;
  rating: number | null;
  status: BookStatus;
  memo: string | null;
};

// 成功なら value（保存してよい値）、失敗なら errors（項目名 → メッセージ）を返す
type ParseResult =
  | { ok: true; value: BookInsert }
  | { ok: false; errors: Record<string, string> };

// 許可するステータスの一覧
const STATUSES: BookStatus[] = ["reading", "completed", "want_to_read"];

// FormData から1項目を「必ず文字列」として取り出す小さな道具
function getText(formData: FormData, name: string): string {
  // 戻り値は string か File か null（FormDataEntryValue | null）
  const raw = formData.get(name);
  // 文字列以外（File や null）が来たら空文字として扱う
  if (typeof raw !== "string") {
    return "";
  }
  return raw.trim();
}

function parseBookForm(formData: FormData): ParseResult {
  const errors: Record<string, string> = {};

  // (1) 必須項目
  const title = getText(formData, "title");
  if (title === "") {
    errors.title = "タイトルは必須です";
  }
  const author = getText(formData, "author");
  if (author === "") {
    errors.author = "著者は必須です";
  }

  // (2) 評価: 空なら null、それ以外は「1〜5 の整数」かを確かめてから数値にする
  const ratingText = getText(formData, "rating");
  let rating: number | null = null;
  if (ratingText !== "") {
```

```
    const n = Number(ratingText);
    if (Number.isInteger(n) && n >= 1 && n <= 5) {
      rating = n;
    } else {
      errors.rating = "評価は1〜5から選んでください";
    }
  }

  // (3) ステータス: 決められた3つのどれかと一致するものを探す
  const statusText = getText(formData, "status");
  const status = STATUSES.find((s) => s === statusText);
  if (status === undefined) {
    errors.status = "ステータスの値が正しくありません";
  }

  // (4) エラーが1つでもあれば失敗として返す
  if (Object.keys(errors).length > 0 || status === undefined) {
    return { ok: false, errors };
  }

  // (5) 任意項目は「空なら null」にそろえて返す
  return {
    ok: true,
    value: {
      title,
      author,
      publisher: getText(formData, "publisher") || null,
      published_date: getText(formData, "published_date") || null,
      rating,
      status,
      memo: getText(formData, "memo") || null,
    },
  };
}

// 動作確認1: 画面から普通に送られてきた入力
const good = new FormData();
good.append("title", "  リーダブルコード  ");
good.append("author", "Dustin Boswell");
good.append("publisher", "");
good.append("rating", "4");
good.append("status", "completed");
console.log("確認1:", parseBookForm(good));

// 動作確認2: 開発者ツールなどで書き換えられた入力（著者欄なし・評価99・存在しないステータス）
const bad = new FormData();
bad.append("title", "テスト");
bad.append("rating", "99");
```

```
bad.append("status", "deleted");
console.log("確認2:", parseBookForm(bad));
```

### ▼ 実行結果（`node formdata-demo.ts`）

```
確認1: {
  ok: true,
  value: {
    title: 'リーダブルコード',
    author: 'Dustin Boswell',
    publisher: null,
    published_date: null,
    rating: 4,
    status: 'completed',
    memo: null
  }
}
確認2: {
  ok: false,
  errors: {
    author: '著者は必須です',
    rating: '評価は1〜5から選んでください',
    status: 'ステータスの値が正しくありません'
  }
}
```

### ▼ コードを1つずつ分解して解説

**解説1: getText で「string | File | null」を string にそろえる**

```
function getText(formData: FormData, name: string): string {
  const raw = formData.get(name);
  if (typeof raw !== "string") {
    return "";
  }
  return raw.trim();
}
```

- `formData.get(name)` の戻り値は、上で説明したとおり `string`・`File`・`null` のどれかです。`typeof raw !== "string"` で「文字列でなければ空文字」にしてしまえば、以降は文字列だけを考えればよくなります。
- 確認2では `author` を `append` していないので `get("author")` は `null` でしたが、ここで空文字になり「著者は必須です」として扱われました。
- `String(formData.get("title") ?? "")` と比べて、`File` が来たときに `"[object File]"` という変な文字列にならない点が安全です。

**解説2: 文字列の評価を「確かめてから」数値にする**

```
if (ratingText !== "") {
  const n = Number(ratingText);
  if (Number.isInteger(n) && n >= 1 && n <= 5) {
    rating = n;
  } else {
    errors.rating = "評価は1〜5から選んでください";
  }
}
```

- `FormData` の値はすべて文字列なので、評価 `"4"` を `Number()` で数値の `4` に変えます。空文字は先に除外しているので、`Number("")` が `0` になる落とし穴（4a の「動かして確かめる」参照）も踏みません。
- `Number.isInteger(n)` は「整数かどうか」を調べます。`"abc"`（NaN）や `"2.5"` はここで弾かれ、範囲チェックで `"99"` も弾かれました。

**解説3: 決められた選択肢かどうかを find で調べる**

```
const status = STATUSES.find((s) => s === statusText);
if (status === undefined) {
  errors.status = "ステータスの値が正しくありません";
}
```

- `find` は配列の中から条件に合う最初の要素を返し、見つからなければ `undefined` を返します。`"deleted"` は3つのどれとも一致しないので `undefined` になり、エラーになりました。
- 見つかった場合、`status` の型は自動的に `BookStatus`（3択のどれか）になります。`statusText as BookStatus` のような無理やりの型指定（アサーション）を書かずに済むのが、この書き方の良いところです。

> **ヒント:** 画面側（`BookForm` の `validate`）でチェックしていても、サーバー側でもう一度チェックするのが Web 開発の鉄則です。確認2のように、ブラウザの開発者ツールを使えば `<select>` の選択肢に無い値でも送れてしまうからです。最後の砦として、データベースの CHECK 制約（第5章）も働きます。

## 5-10. Server Action のセキュリティ（参考）

Server Action は「サーバー上で実行される関数」です。クライアント（ブラウザ）から呼び出せますが、コードはサーバーで走るため:

- **シークレット情報を扱える**: 環境変数の SUPABASE_SERVICE_ROLE_KEY など、ブラウザに漏らしたくない情報をそのまま使える。
- **データベースに直接アクセスできる**: クライアントを経由しないので、改ざんされにくい。
- **ただしクライアントからの入力は信用しない**: フォームから送られた `formData` の中身は誰でも書き換えられる可能性があるので、サーバー側でも改めてバリデーションする必要がある。

> **覚えておくべきこと:** 「クライアントから来た値はすべて疑え」は Web 開発全般の鉄則です。Client Component 方式（本書）でも、Supabase の Row Level Security（RLS）ポリシーが「最後の砦」として働きます。

---

## 6. 動作確認手順

以下の手順で、実装した機能が正しく動作するか確認しましょう。

### 前提

- 開発サーバーが起動していること（`npm run dev` を実行済み）
- Supabase のプロジェクトとデータベースが前章で構築済みであること
- `.env.local` に Supabase の接続情報が設定されていること

### ステップ1: トップページの表示確認（書籍0冊の状態）

1. ブラウザで `http://localhost:3000` を開きます。
2. 以下を確認してください: - ヘッダーに「書籍管理アプリ」と表示されている - 「＋ 新規登録」ボタンが右上に表示されている - 「全 0 冊」と表示されている - 「書籍が登録されていません」というメッセージと「最初の書籍を登録する」ボタンが中央に表示されている
3. ブラウザの開発者ツール（F12）の Console タブにエラーが出ていないことを確認します。

### ステップ2: 登録ページへの遷移確認

1. ヘッダーの「＋ 新規登録」ボタンをクリックします。
2. `http://localhost:3000/books/new` に遷移し、登録フォームが表示されることを確認します。
3. 以下の要素が表示されていることを確認: - 左上に戻るボタン（矢印アイコン） - 「書籍を登録する」というタイトル - タイトル、著者、出版社、出版日、評価、ステータス、メモの各フィールド - 「登録する」ボタンと「キャンセル」ボタン

### ステップ3: バリデーションの確認

1. 何も入力せずに「登録する」ボタンをクリックします。
2. 「タイトルは必須です」「著者は必須です」というエラーメッセージが赤文字で表示されることを確認します。
3. タイトルと著者のフィールドの枠線が赤くなっていることを確認します。
4. タイトルフィールドに何か入力すると、タイトルのエラーメッセージが消えることを確認します。

### ステップ4: 書籍の登録

1. 以下のテスト用データを入力します: - タイトル: `リーダブルコード` - 著者: `Dustin Boswell` - 出版社: `オライリージャパン` - 出版日: `2012-06-23` - 評価: `★★★★☆ (4)` - ステータス: `読了` - メモ: `読みやすいコードを書くための実践的なテクニック集。`
2. 「登録する」ボタンをクリックします。
3. ボタンが一瞬「送信中...」に変わることを確認します。
4. トップページ（`http://localhost:3000`）にリダイレクトされることを確認します。
5. 「全 1 冊」と表示され、登録した書籍のカードが表示されることを確認します。

### ステップ5: カード表示の確認

1. 表示されたカードに以下の情報があることを確認: - タイトル「リーダブルコード」 - ステータスバッジ「読了」（緑色） - 著者「Dustin Boswell」 - 出版社「オライリージャパン」 - 出版日「2012-06-23」 - 星評価「★★★★☆ (4)」 - メモ「読みやすいコードを書くための実践的なテクニック集。」
2. カードにマウスを乗せると影が変化することを確認します。

### ステップ6: 複数冊の登録とグリッド表示の確認

1. さらに2〜3冊の書籍を登録します（ステータスを変えて登録すると、バッジの色分けも確認できます）。
2. 画面幅を変えて、グリッドレイアウトのレスポンシブ動作を確認します: - デスクトップ（1024px 以上）: 3列 - タブレット（768px〜1023px）: 2列 - スマホ（767px 以下）: 1列

### ステップ7: Supabase ダッシュボードでのデータ確認

1. Supabase ダッシュボード（`https://supabase.com/dashboard`）にアクセスします。
2. Table Editor で `books` テーブルを開きます。
3. 登録した書籍のレコードが正しく保存されていることを確認します。

---

## 7. トラブルシューティング

### 問題1: データが表示されない（一覧が常に空）

**症状**: 書籍を登録したはずなのに、トップページに書籍が表示されない。「全 0 冊」のまま。

**原因と対策**:

1. **Supabase の接続情報が正しくない**

`.env.local` ファイルの内容を確認してください。

`bash # .env.local NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbG...`

- URL の末尾にスラッシュ `/` が余計についていないか確認してください。
- `.env.local` を変更した場合は、開発サーバー（`npm run dev`）を **再起動** する必要があります（Ctrl+C で停止し、再度 `npm run dev`）。

2. **Row Level Security (RLS) の設定**

Supabase のテーブルに RLS が有効になっていると、ポリシーが設定されていない場合はデータを取得できません。

Supabase ダッシュボードで確認: - Authentication → Policies → `books` テーブル - SELECT に対するポリシーが設定されているか確認 - テスト段階では一時的に RLS を無効にすることもできます（ダッシュボードの Table Editor → `books` テーブル → RLS を OFF に）

3. **Supabase クライアントのインポートパスが間違っている**

`app/page.tsx`（Server Component）では `@/lib/supabase/server` からインポートし、`app/books/new/page.tsx`（Client Component）では `@/lib/supabase/client` からインポートする必要があります。逆にするとエラーになります。

4. **ブラウザの開発者ツールで確認する**

F12 → Console タブを開き、エラーメッセージがないか確認してください。また、Network タブで Supabase への API リクエストが成功しているか（ステータスコード 200）を確認してください。

**5. `permission denied for table books`（code: `42501`）と出る（2026年の Supabase の仕様変更）**

Supabase は 2026年に、「新しく作ったテーブルを、自動では Data API（supabase-js から使う窓口）に公開しない」という仕様に変わりました。2026年5月30日以降に作ったプロジェクトでは最初から、それより前のプロジェクトでも 2026年10月30日以降に作るテーブルから、この動きになります（すでにあるテーブルの許可はそのまま残ります）。この場合、第5章の RLS ポリシーを正しく作っていても、テーブルそのものを使う**許可（GRANT／グラント）**が無いため、一覧が空になり、サーバーのターミナルに次のエラーが出ます。

```
書籍の取得に失敗しました: permission denied for table books
```

1e のつまずきポイントの OK 版のように `error` を丸ごとログに出していれば、`hint`（直し方）まで表示されます。

```
書籍の取得に失敗しました: {
  code: '42501',
  details: null,
  hint: 'Grant the required privileges to the current role with: GRANT SELECT ON public.books TO anon;',
  message: 'permission denied for table books'
}
```

`hint` に書かれているとおり、Supabase ダッシュボードの **SQL Editor** で許可を与えます。

```
-- books テーブルを、ログインしていない利用者（anon ロール）からも読み書きできるようにする
-- この教材にはまだログイン機能が無いので、第5章の「全員に全操作を許可（開発用）」ポリシーと組み合わせて使う
grant select, insert, update, delete on public.books to anon;
-- ログイン済みの利用者（authenticated ロール）にも同じ許可を出しておく
grant select, insert, update, delete on public.books to authenticated;
```

- **GRANT** は「テーブルという建物に入るための入館証」、**RLS ポリシー**は「建物の中で、どの部屋（行）に入ってよいかのルール」です。supabase-js から使うには、**両方**がそろっている必要があります。
- プロジェクト作成時の設定によっては、従来どおり自動で許可が付いている場合もあります。エラーが出ていなければ、この SQL は実行しなくて構いません。

> **ヒント（API キーの名前について）**：最近の Supabase ダッシュボードでは、API キーが「Publishable key（`sb_publishable_` で始まる）」と「Secret key」という名前で表示されます。Publishable key は従来の anon key の代わりにそのまま使えるので、`.env.local` の `NEXT_PUBLIC_SUPABASE_ANON_KEY` の値として貼り付けて構いません。大切なのは、`.env.local` の**変数名**と、`lib/supabase/client.ts`・`server.ts` の `process.env.〇〇` の名前が一字一句そろっていることです。

## 問題2: フォーム送信でエラーが出る

**症状**: 「登録する」ボタンを押すと、エラーメッセージ（「書籍の登録に失敗しました」）が表示される。

**原因と対策**:

1. **テーブルのカラム名が一致していない**

`insert` に渡すオブジェクトのキー名が、データベースのカラム名と一致していない可能性があります。ブラウザの Console にエラー詳細が表示されているはずなので確認してください。

`// よくある間違い // NG: キャメルケース publishedDate: "2024-01-01" // OK: スネークケース（DBのカラム名に合わせる） published_date: "2024-01-01"`

2. **NOT NULL 制約違反**

データベースの `title` や `author` カラムに NOT NULL 制約がかかっている場合、空文字列や null を送信するとエラーになります。BookForm のバリデーションで必須チェックを行っていますが、バリデーションをバイパスして送信した場合に発生します。

3. **RLS ポリシーの INSERT が許可されていない**

Supabase のテーブルに RLS が有効になっていて、INSERT のポリシーが設定されていない場合、データを挿入できません。SELECT と同様に、INSERT のポリシーも設定する必要があります。

4. **Supabase のクライアント用ライブラリを使っているか確認**

`app/books/new/page.tsx` は Client Component なので、`@/lib/supabase/client` を使います。`@/lib/supabase/server` を使うと `cookies()` などのサーバー専用APIにアクセスしようとしてエラーになります。

**5. メモの列名がテーブルと違う（`memo` と `notes`）**

この章のコードは `memo` という列名で INSERT します。第5章の SQL のとおりに作ったテーブルなら列名は `memo` ですが、この教材の古い版や自分で作ったテーブルで列名が `notes` などになっていると、`Could not find the 'memo' column of 'books' in the schema cache`（code: `PGRST204`）というエラーになります。Supabase ダッシュボードの Table Editor で列名を確かめ、どちらかにそろえてください。テーブル側を `memo` に合わせる場合は、SQL Editor で次の1行を実行します。

```
-- books テーブルの notes 列の名前を memo に変える（中のデータはそのまま残る）
alter table books rename column notes to memo;
```

**▼ エラーコードで原因を見分ける**

Supabase の `error` には `code`（エラーコード）が入っています。`message` の英文は版によって少し変わることがありますが、`code` は変わりにくいので、原因を見分けるときは `code` を手がかりにするのが確実です。この章で出会いやすいものをまとめます。

| code | message の例 | 主な原因 | 対処 |
| --- | --- | --- | --- |
| `23502` | `null value in column "title" of relation "books" violates not-null constraint` | 必須の列（title・author）が空 | 必須チェックと `trim()` を確認 |
| `23514` | `new row for relation "books" violates check constraint "books_rating_check"` | 評価が1〜5以外、ステータスが3種類以外 | 評価の数値変換（4a の Number の落とし穴）を確認 |
| `22P02` | `invalid input syntax for type integer: ""` | 数値の列に空文字などを送った | 空文字を `null` に変換する |
| `22007` | `invalid input syntax for type date: ""` | 日付の列に空文字を送った | 空文字を `null` に変換する（4b の書き方） |
| `42501` | `new row violates row-level security policy for table "books"` | INSERT を許す RLS ポリシーが無い | 第5章の INSERT ポリシーを確認 |
| `42501` | `permission denied for table books` | テーブルへの GRANT が無い | 問題1の「5」を参照 |
| `PGRST204` | `Could not find the 'memo' column of 'books' in the schema cache` | 列名の書きまちがい | 列名を Table Editor で確認 |
| `PGRST205` | `Could not find the table 'public.bookz' in the schema cache` | テーブル名の書きまちがい | `hint` の候補を確認 |

### 動かして確かめる: エラーコードから画面に出す文を選ぶ

登録ページ（4b）では、どんな失敗でも同じ文（「書籍の登録に失敗しました…」）を出していました。`code` で分岐すれば、利用者にもう少し具体的な案内を出せます。

> **▼ このコードがやること（先に日本語で）**: Supabase が返す `error` の `code` を見て、画面に出す日本語の文を選ぶ関数 `toUserMessage` を作ります。実際に Supabase が返すのと同じ形のエラーを5つ用意して、それぞれどの文になるかを表示します。

```ts
// supabase-js が返す error（PostgrestError）の4つの項目
type SupabaseError = {
  code: string;
  message: string;
  details: string | null;
  hint: string | null;
};

// エラーコード（code）を見て、画面に出す日本語メッセージを選ぶ
// message の文面はバージョンで変わることがあるので、分岐には code を使う
function toUserMessage(error: SupabaseError): string {
  switch (error.code) {
    case "23502":
      return "必須項目（タイトル・著者）が空です。";
    case "23514":
      return "評価やステータスの値が、許可された範囲の外です。";
    case "22P02":
    case "22007":
      return "数値や日付の形式が正しくありません。";
    case "42501":
      return "データを保存する権限がありません。";
    case "PGRST204":
    case "PGRST205":
      return "テーブル名または列名がデータベースと一致していません。";
    default:
      return "書籍の登録に失敗しました。時間をおいて再度お試しください。";
  }
}

// Supabase が実際に返すエラーの例
//  （code と message は PostgreSQL / PostgREST の実物と同じ文面。details は長いので省略）
const samples: SupabaseError[] = [
  {
    code: "23502",
    message:
      'null value in column "title" of relation "books" violates not-null constraint',
    details: "Failing row contains (...)",
    hint: null,
  },
  {
    code: "23514",
    message:
      'new row for relation "books" violates check constraint "books_rating_check"',
    details: "Failing row contains (...)",
    hint: null,
  },
  {
    code: "22P02",
    message: 'invalid input syntax for type integer: ""',
```

```
    details: null,
    hint: null,
  },
  {
    code: "42501",
    message: 'new row violates row-level security policy for table "books"',
    details: null,
    hint: null,
  },
  {
    code: "PGRST204",
    message: "Could not find the 'memo' column of 'books' in the schema cache",
    details: null,
    hint: null,
  },
];

for (const error of samples) {
  console.log(`[${error.code}] ${error.message}`);
  console.log(`  → 画面に出す文: ${toUserMessage(error)}`);
}
```

▼ 実行結果（`node error-message-demo.ts`）

```
[23502] null value in column "title" of relation "books" violates not-null constraint
  → 画面に出す文: 必須項目（タイトル・著者）が空です。
[23514] new row for relation "books" violates check constraint "books_rating_check"
  → 画面に出す文: 評価やステータスの値が、許可された範囲の外です。
[22P02] invalid input syntax for type integer: ""
  → 画面に出す文: 数値や日付の形式が正しくありません。
[42501] new row violates row-level security policy for table "books"
  → 画面に出す文: データを保存する権限がありません。
[PGRST204] Could not find the 'memo' column of 'books' in the schema cache
  → 画面に出す文: テーブル名または列名がデータベースと一致していません。
```

## ▼ コードを1つずつ分解して解説

**解説1: error の4つの項目**

```
type SupabaseError = {
  code: string;
  message: string;
  details: string | null;
  hint: string | null;
};
```

- supabase-js の `error` には、主に `code`（エラーの種類を表す短い記号）・`message`（英語の説明）・`details`（問題のあった値などの詳細）・`hint`（直し方のヒント）の4つが入っています。
- 原因を調べるときは「`hint` → `code` → `details` → `message`」の順に見ると近道です。`hint` に直し方そのもの（GRANT の SQL や、正しいテーブル名の候補）が書いてあることがよくあります。

**解説2: switch で code ごとに文を選ぶ**

```
switch (error.code) {
  case "22P02":
  case "22007":
    return "数値や日付の形式が正しくありません。";
```

- `switch` は「値に応じて行き先を選ぶ」分岐です。`case` を2つ続けて書くと「どちらでも同じ処理」という意味になります。
- `default` は「どの `case` にも当てはまらなかったとき」の行き先です。知らないコードが来ても、必ず何かしらの文が返ります。

**解説3: 画面に出す文と、ログに出す情報を分ける**

- 利用者向けの文には、英語のエラー文や列名などの内部情報をそのまま出さず、「何をすればよいか」が分かる日本語にします。
- 原因調べに必要な情報は、`console.error("書籍の登録に失敗しました:", error)` のように `error` を丸ごとログに出しておきます。4b の登録ページで使う場合は、`setSubmitError("書籍の登録に失敗しました。…")` の部分を `setSubmitError(toUserMessage(error))` に置き換えるだけです。

## 問題3: 型エラーが出る

**症状**: TypeScript のコンパイルエラーが発生する。

**原因と対策**:

1. **`Book` 型のインポート漏れ**

`Cannot find name 'Book'.`

`Book` 型を使うファイルで、正しくインポートしているか確認してください。

`tsx import { type Book } from "@/components/BookCard";`

`type` キーワードを使っているのは「型だけをインポートする」という意味です。TypeScript 専用の構文で、コンパイル後の JavaScript からは削除されます。なくても動きますが、付けておくとインポートの意図が明確になります。

2. **`BookStatus` 型のインポート漏れ**

`Cannot find name 'BookStatus'.`

`StatusBadge.tsx` からエクスポートしている型なので:

`tsx import { type BookStatus } from "@/components/StatusBadge";`

3. **Supabase の型定義との不一致**

Supabase CLI で型を自動生成している場合、自動生成された型と手動で定義した `Book` 型に差異があるとエラーになることがあります。自動生成された型を使う方法は後の章で解説しますが、現段階では手動で定義した型で問題ありません。

4. **`null` と `undefined` の混在**

`Type 'undefined' is not assignable to type 'string | null'.`

Supabase から返るデータは `null`（`undefined` ではない）です。型定義で `string | undefined` ではなく `string | null` を使ってください。

5. **`createClient` の名前衝突**

サーバー用とクライアント用で同じ名前 `createClient` をインポートしていますが、ファイルが異なるので衝突しません。ただし、1つのファイルで両方使おうとすると衝突します。そのような場合はエイリアスを使います:

`tsx import { createClient as createServerClient } from "@/lib/supabase/server"; import { createClient as createBrowserClient } from "@/lib/supabase/client";`

---

## 8. 練習問題（解答・解説つき）

この章の内容が身に付いたかを確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んでください。コードを書く問題は、実際に VS Code や Node.js で試すとより理解が深まります。

### 問題1: CRUD の対応（基本）

次の (a)〜(c) は CRUD のどれにあたりますか？ また、supabase-js ではどのメソッドを使いますか？

- (a) 本の一覧を、新しく登録した順に表示する
- (b) 登録フォームから新しい本を追加する
- (c) 登録済みの本の評価を 3 から 5 に変える

### 問題2: 一覧がいつも0冊になる（await）

次のトップページは、本を何冊登録しても「全 0 冊」と表示されます。原因を答え、正しく直してください。

```tsx
export default async function HomePage() {
  const supabase = await createClient();
  const { data: books } = supabase.from("books").select("*");
  return <BookList books={books ?? []} />;
}
```

### 問題3: カードが1枚も出ない（map）

Supabase からデータは取れている（`console.log(books)` で配列が表示される）のに、一覧にカードが1枚も表示されません。どこを直せばよいですか？

```tsx
{books.map((book) => {
  <BookCard key={book.id} book={book} />;
})}
```

### 問題4: 評価を選ばずに登録すると失敗する（型変換）

ある人が `BookForm` の `handleRatingChange` を次のように書きました。評価を「選択してください」のまま登録すると、登録に失敗します。なぜ失敗するのか、どう直せばよいかを答えてください。

```tsx
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  setFormData((prev) => ({
    ...prev,
    rating: Number(e.target.value),
  }));
};
```

### 問題5: バリデーションの結果を予想する

4a の「動かして確かめる」で作った `validateBookForm` に、次の値を渡したときの戻り値を予想してください（`empty` は同じサンプルの初期値です）。

```tsx
validateBookForm({
  ...empty,
  title: "   ",
  author: "A".repeat(51),
  memo: "   ",
});
```

## 問題6: error.tsx が表示されるのはどれ？

次の (a)〜(c) のうち、3節で作った `app/error.tsx` の画面が表示されるのはどれですか？（複数選んでも構いません）

- (a) トップページ（`HomePage`）の中で `throw new Error("取得失敗")` が実行された
- (b) 登録ページの `handleSubmit`（「登録する」ボタンを押したあとの処理）の中で例外が起きた
- (c) トップページで Supabase が `error` を返し、`if (error)` の中で「読み込めませんでした」を `return` した

## 問題7: 登録できたのに一覧に出ない

「登録する」を押すと一覧ページに戻り、Supabase の Table Editor を見ると本は確かに保存されています。それなのに、一覧に新しい本が表示されません。考えられる原因を2つ挙げてください。

## 解答と解説

### 問題1の解答

| | CRUD | supabase-js |
|---|---|---|
| (a) | R（Read） | `.select("*").order("created_at", { ascending: false })` |
| (b) | C（Create） | `.insert({...})` |
| (c) | U（Update） | `.update({ rating: 5 }).eq("id", 本のid)` |

**解説:** 「表示する＝R」「新しく作る＝C」「直す＝U」「消す＝D」です。(a) の「新しい順」は `order` で `ascending: false`（降順）を指定します。(c) の `update` は第8章で詳しく扱いますが、「どの行を直すか」を `.eq("id", ...)` で必ず絞り込む点を覚えておきましょう。絞り込みを忘れると、全部の本の評価が書き換わってしまいます。

### 問題2の解答

`supabase.from("books").select("*")` の前に **`await` が無い**のが原因です。正しくは次のとおりです。

```
export default async function HomePage() {
  const supabase = await createClient();
  // await を付けて、結果（data と error）が届くまで待つ
  const { data: books, error } = await supabase.from("books").select("*");
  if (error) {
    console.error("書籍の取得に失敗しました:", error);
  }
  return <BookList books={books ?? []} />;
}
```

**解説:** `await` が無いと、右辺は結果ではなく「あとで結果が入る入れ物（Promise のなかま）」のままです。その入れ物には `data` というプロパティが無いので `books` は `undefined` になり、`books ?? []` で空配列になって「0冊」と

表示されます。しかも supabase-js は `await` されるまで通信を始めないので、Supabase への問い合わせ自体が行われていません（0.3節の補足）。VS Code では「`data` というプロパティは存在しません」という赤線が出るはずなので、赤線は必ず確認しましょう。あわせて `error` も受け取っておくと、失敗にも気づけます。

### 問題3の解答

`=>` の右側を `{ }` で囲んでいるのに `return` が無いため、`map` の結果が `[undefined, undefined, ...]` になっています。`( )` で囲んで JSX をそのまま返します。

```
{books.map((book) => (
  <BookCard key={book.id} book={book} />
))}
```

**解説:** 1d の「動かして確かめる」で見たとおり、`=> { }` はブロックなので `return` が必要、`=> ( )` は式なので自動で返ります。`undefined` は画面に何も描かず、エラーも出ないので気づきにくいバグです。「`=>` のすぐ右が `{` なら `return` を探す」と覚えましょう。

### 問題4の解答

「選択してください」の値は空文字 `""` で、`Number("")` は `0` になります。books テーブルには「評価は1〜5」という CHECK 制約があるため、評価 0 の行は拒否され、`23514`（CHECK 制約違反）のエラーで登録に失敗します。空文字を先に `null` へ変換します。

```
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const value = e.target.value;
  setFormData((prev) => ({
    ...prev,
    // 空文字（選択してください）なら null、それ以外は数値に
    rating: value === "" ? null : Number(value),
  }));
};
```

**解説:** 4a の「動かして確かめる」の実行結果のとおり、`Number("")` は `NaN` ではなく `0` です。`<select>` や `<input>` の値はいつも文字列なので、数値の列に保存する前には「空かどうか」→「数値に変換」の順で処理します。

### 問題5の解答

```
{ title: 'タイトルは必須です', author: '著者は50文字以内で入力してください' }
```

**解説:** タイトルの `"　"`（全角スペース2つ）は `trim()` で空文字になるので「必須です」です。著者の `"A".repeat(51)` は51文字なので上限（50文字）を1文字超えています。メモの `" "` は `trim()` すると0文字で、メモは任意項目なのでエラーになりません。実際に Node.js で実行すると、この結果が表示されます。

### 問題6の解答

**(a) だけ**です。

**解説:** `error.tsx` が受け止めるのは「画面を描いている最中」に投げられた例外です。(a) はトップページを描く途中の `throw` なので `error.tsx` に切り替わります。(b) のボタンを押したあとの処理（イベントハンドラ）の中のエラーは `error.tsx` に届かないので、4b のように `try / catch` と `submitError` で自分で表示します。(c) は例外を投げずに自分で別の表示を返しているので、`error.tsx` は出番がありません（図7-9）。

### 問題7の解答

例えば次の2つが考えられます。

1. 登録ページで `router.push("/")` のあとに **`router.refresh()` を呼んでいない**。一覧ページの古い表示（キャッシュ）が使われ、Server Component が取り直しをしていない。
2. 一覧の**取得（SELECT）に失敗している**のに、それを画面に出していない。SELECT を許す RLS ポリシーや GRANT が無いと `error` が返りますが、1e の元のコードは失敗しても「0冊」として表示を続けるため、取得できていないことに気づけません。

**解説:** 「保存（INSERT）」と「表示（SELECT）」は別々の処理なので、片方だけ成功することがあります。Table Editor にデータがあるなら保存は成功しているので、表示側（`router.refresh()`・SELECT の権限・`error` のログ）を順に疑いましょう。7節の「問題1」とエラーコードの表も役に立ちます。

---

## まとめ

この章では、以下の機能を実装しました。

- **一覧表示**: Server Component でSupabase からデータを取得し、BookCard を使ってグリッド表示
- **ステータス・評価の視覚化**: StatusBadge（色分けバッジ）、RatingStars（星評価）
- **ローディング状態**: loading.tsx とスケルトン UI
- **エラーハンドリング**: error.tsx とリトライ機能
- **新規登録**: BookForm でフォーム入力、バリデーション、Supabase への INSERT、リダイレクト

次の章では、**詳細表示（Read）**、**編集（Update）**、**削除（Delete）** を実装し、CRUD の残りの機能を完成させます。